



Università Campus Bio-Medico di Roma

Project Work di Master Universitario

di I livello in

Cybersecurity Management

La cybersecurity nel Web Semantico: un approccio ontologico alla cybersecurity basato su MITRE D3FEND

Studente: Roberto Polli

Relatore: Prof. Mario Garofano

Anno Accademico 2025/2026

Data di discussione: 23 settembre 2026

Abstract

TL;DR (it) Design & D3FEND è un'applicazione open source che permette di annotare i documenti di progettazione IT scritti in Markdown e Mermaid.js ricavandone poi dei Knowledge Graph basati sull'ontologia MITRE D3FEND, utili ad individuare in anticipo le tecniche offensive applicabili e le relative contromisure.

(en) Design & D3FEND is an open source application that allows to annotate IT design documents written in Markdown and Mermaid.js to derive Knowledge Graphs based on the MITRE D3FEND ontology, making it possible to identify the applicable offensive techniques and countermeasures before the implementation phase.

(it) Contesto: Knowledge Graph (KG) quali MITRE D3FEND o il Data Privacy Vocabulary (DPV) sono delle preziose basi di conoscenza per la progettazione di sistemi informatici sicuri. Il loro utilizzo da parte dei professionisti IT rimane tuttavia limitato, anche a causa della scarsa diffusione delle competenze legate alle tecnologie del Semantic Web (e.g., ontologie, RDF, SPARQL).

Obiettivo e approccio: Questo elaborato affronta tale problema, con l'obiettivo di rendere accessibili le informazioni contenute nei Knowledge Graph, integrandole negli strumenti e nei processi abitualmente utilizzati dai professionisti IT, anche in assenza di una solida conoscenza dei concetti ontologici.

Risultati: Presentiamo quindi Design & D3FEND, un'applicazione open source che permette di annotare i documenti di progettazione IT scritti in Markdown e Mermaid.js, ricavandone poi dei Knowledge Graph basati su D3FEND. Le architetture così descritte possono essere analizzate mettendo in relazione i loro componenti con la conoscenza contenuta in D3FEND, consentendo di individuare le tecniche offensive applicabili e le contromisure prima della fase di implementazione.

Parallelamente allo sviluppo dell'applicazione sono state sviluppate due estensioni per Visual Studio Code, D3FEND Language e GDPR Language, che permettono di

accedere alle informazioni contenute nei KG D3FEND, DPV ed EU GDPR direttamente dall'Integrated Development Environment (IDE), nonché delle mappature di esempio tra D3FEND e DPV per facilitare l'integrazione dei due KG.

Conclusioni: La valorizzazione dei KG passa dalla costruzione di strumenti che ne permettano un più ampio utilizzo. L'approccio di Design & D3FEND estende a tutti gli ambiti IT la possibilità di implementare strategie di difesa *by design* basate sull'analisi dei KG.

L'approccio è validato partendo dalla progettazione minimale di una Web API per cui - incrociando i dataset MITRE (D3FEND, CAPEC, CWE) con quello della OWASP API Security Top 10 2023 - è possibile individuare automaticamente 5 su 7 dei rischi della OWASP API Security Top 10 2023 associabili alle tecniche offensive censite in D3FEND.

(en) Knowledge Graphs (KG) such as MITRE D3FEND or Data Privacy Vocabulary are valuable datasets for the design of secure IT systems. Their use by IT professionals remains limited due to the lack of skills in Semantic Web technologies (e.g, RDF, SPARQL).

This paper faces this problem with the aim of making the information contained in Knowledge Graphs accessible, integrating them into the tools and processes commonly used by IT professionals, even without a solid knowledge of ontological concepts.

We present Design & D3FEND, an open source application to annotate IT design documents written in Markdown and Mermaid.js, to derive Knowledge Graphs based on the D3FEND ontology. The architectures described in this way can be analyzed by relating their components to the knowledge contained in D3FEND, making it possible to identify the applicable offensive techniques and countermeasures before the implementation phase.

Along with the development of the application, two Visual Studio Code extensions have been developed, D3FEND Language and GDPR Language, which allow to access the information contained in the D3FEND, DPV and EU GDPR KGs directly

from the Integrated Development Environment (IDE), as well as example mappings between D3FEND and DPV to facilitate the integration of the two KGs.

The approach is validated starting from the minimal design of a Web API, for which - by cross-referencing the MITRE datasets (D3FEND, CAPEC, CWE) with the OWASP API Security Top 10 2023 dataset - it is possible to automatically identify 5 out of 7 of the OWASP API Security Top 10 2023 risks that are associable with the offensive techniques described in D3FEND.

Keywords Cybersecurity, Ontology, IT Architecture, SPARQL

Indice

1	Introduzione	11
1.1	Contesto e motivazione	11
1.2	Obiettivi	13
1.2.1	Contributo originale	14
1.3	Struttura dell'elaborato	15
1.4	Convenzioni editoriali	15
1.4.1	Notazione RDF	17
1.5	Definizioni e Acronimi	18
1.5.1	Definizioni	18
1.5.2	Acronimi	19
1.6	Limitazioni	22
1.7	Prerequisiti	22
1.7.1	Grafi e rappresentazione della conoscenza	22
1.8	Il Web Semantico	26
1.8.1	Resource Description Framework (RDF): machine-readable knowledge	26
1.8.2	Vocabolari Controllati	30
1.8.3	Labeled Property Graph e RDF 1.2	32
1.8.4	Simbolismo	35
2	Knowledge Graph per la Cybersecurity	36
2.1	Cybersecurity	36
2.1.1	D3FEND e il CSF 2.0	38

2.2	MITRE D3FEND	39
2.2.1	D3FEND e i controlli NIST SP 800-53	41
2.2.2	D3FEND ed altri dataset del MITRE	44
2.2.3	Proprietà	46
2.2.4	Classi	48
2.2.5	Classi e Proprietà Tattiche	53
2.2.6	Relazioni tra d3f:OffensiveTechnique e d3f:DigitalArtifact .	57
2.2.7	Individuazione delle d3f:DefensiveTechnique	64
2.2.8	Associazioni tra tecniche e oggetti digitali che le implemen- tano	67
2.2.9	Estensioni: SPARTA, ATLAS, ATT&CK Mobile e ATT&CK ICS	67
2.3	OWASP TOP 10	69
2.4	Altre ontologie per la cybersecurity	72
2.5	Data Privacy Vocabulary (DPV)	73
2.5.1	Panoramica DPV ed EU-GDPR	74
2.5.2	Casi d'uso	77
2.5.3	Attori ed entità	78
2.5.4	Data e PersonalData	79
2.5.5	Contesto e Luogo del trattamento	80
2.5.6	Misure di sicurezza	80
3	Diagrammi architetturali	84
3.1	Altre modalità di rappresentazione delle architetture	85
3.1.1	D3FEND CAD	85

3.1.2	UML	86
3.1.3	C4 Model e Mermaid Architecture Diagram	87
3.1.4	Esempio di diagramma architetturale di AWS	87
4	Progettazione sicura con grafi architetturali basati su D3FEND	91
4.1	Usare la sintassi Mermaid Flowchart per rappresentare i grafi archi- tetturali	92
4.1.1	Rappresentazione dei grafi	92
4.1.2	Rappresentazione dei nodi	92
4.1.3	Rappresentazione delle relazioni	93
4.1.4	Relazioni transitive	94
4.1.5	Rappresentazione di d3f:DefensiveTechnique	95
4.1.6	Rappresentazione di flussi informativi	96
4.2	Modellare un’infrastruttura: il caso d’uso di una web application . .	97
4.2.1	Componenti implicite	98
4.2.2	Risorse computazionali, networking e storage	100
4.3	Ulteriori Esempi	102
4.3.1	Architettura minimale	102
4.3.2	Individuare le tecniche offensive applicabili	105
5	Implementazioni	107
5.1	D3FEND Language	107
5.2	GDPR Language	109
5.3	Design & D3FEND	110
5.3.1	Architettura	111
5.3.2	Interfaccia	113

5.4	Caso d'uso: individuare le tecniche ATT&CK applicabili ad un'architettura annotata con D3FEND	117
5.4.1	Ricerca diretta degli artefatti impattati	119
5.4.2	Ricerca per sottoclasse	119
5.4.3	Ricerca per superclasse	121
5.4.4	Ricerca per intorno di classe	122
5.4.5	Ricerca delle misure di sicurezza	123
5.4.6	Conclusioni	124
5.4.7	Mermaid.js Diagram	125
5.4.8	Knowledge Graph	126
6	Conclusioni e sviluppi futuri	128
6.1	Valutazione critica	129
6.2	Limiti	129
6.3	Sviluppi implementativi	130
6.3.1	Integrazione con dataset esterni	130
6.3.2	DevSecOps e Integrazione con IaC	131
6.4	Sviluppi di ricerca applicata	132
6.4.1	Identificazione della superficie d'attacco	132
6.4.2	Integrazione con vocabolari giuridici	132
6.4.3	Piattaforma integrata per la progettazione sicura	133
	Bibliografia	134

Elenco delle figure

1	Sottoclassi rappresentate come archi tratteggiati	16
2	Sottoclassi rappresentate come sottoinsiemi	17
3	Una frase come grafo, tra soggetto, predicato e oggetto	23
4	Una voce enciclopedica su Python rappresentata come grafo	24
5	Estratto della voce su Python di DBpedia	25
6	Estratto della voce su Guido van Rossum di Wikidata	25
7	Collegamento tra i grafi di DBpedia e Wikidata	26
8	Rappresentazione LPG di una relazione di amicizia	33
9	Rappresentazione RDF di una relazione di amicizia	34
10	Struttura orientativa di D3FEND	41
11	Modellazione dei controlli di sicurezza in D3FEND	42
12	D3FEND e NIST SP 800-53	43
13	Relazioni tra D3FEND, Security Control e CSF 2.0	44
14	D3FEND e CWE	45
15	Proprietà principali di D3FEND e relative sottoproprietà	47
16	Relazioni tattiche in D3FEND	47
17	Proprietà tattiche in D3FEND	48
18	Classi principali di D3FEND e relative sottoclassi	49
19	Classi principali di D3FEND rappresentate come sotto-diagrammi	50
20	Sottoclassi di d3f:AnalyticTechnique	50
21	Gerarchia della classe d3f:Artifact	51
22	Un utente che accede a un sistema digitale	51
23	Sottoclassi dirette di d3f:OffensiveTechnique	56

24	Relazioni di esempio tra d3f:OffensiveTechnique e d3f:DigitalArtifact	57
25	Tecniche offensive associate a d3f:Database	61
26	Tecniche offensive applicabili a d3f:Database tramite i suoi vicini .	63
27	Oggetti digitali che implementano tecniche difensive	67
28	Modelli di minaccia esterni come sottoclassi di d3f:ExternalThing .	68
29	Panoramica DPV ed EU-GDPR	75
30	Attori ed entità in DPV	78
31	Sottoclassi di dpv:Data	79
32	Sottoclassi di dpv:TechnicalOrganisationalMeasure	81
33	Architettura di una piattaforma su AWS	88
34	Flusso informativo dell'utente attraverso la CDN e il WAF	88
35	Disambiguazione dei flussi informativi della WebUI.	90
36	Diagramma minimale di una Web Application	91
37	Diagramma annotato di una Web Application	94
38	Relazione transitiva rappresentata come archi	94
39	Relazione transitiva rappresentata come sotto-diagramma	94
40	Diagramma minimale di una Web Application con le tecniche di- fensive	95
41	Una data pipeline	96
42	Diagramma minimale di una Web Application con i flussi informativi	97
43	Diagramma di intorno di una Web Application	98
44	Diagramma di intorno di una Web Application con i meccanismi di accesso	99
45	Risorse contenute nella Web Application	101
46	Un utente che accede a una Web Application	102

47	Un utente che accede a una Web Application, con dispositivi e collocazione fisica	103
48	Un utente che accede a una Web Application, con la collocazione come sotto-diagramma	104
49	Una Web Application con una query al database	105
50	Web Application annotata con classi e proprietà D3FEND	105
51	Architettura di D3FEND Language	108
52	Autocompletamento di entità D3FEND	108
53	Tooltip su un'entità D3FEND	109
54	Gerarchia di un'entità D3FEND	109
55	Hover su un'entità DPV o EU GDPR	110
56	Architettura di Design & D3FEND	113
57	Interfaccia di Design & D3FEND	114
58	d3f:OffensiveTechnique e d3f:DefensiveTechnique associate ad un oggetto	115
59	Interfaccia di Design & D3FEND - preview dei diagrammi Mermaid.js	115
60	Interfaccia di Design & D3FEND - contenuto del KG generato . . .	116
61	Interfaccia di Design & D3FEND - interfaccia SPARQL	117
62	Percorsi verso i rischi A07 e A08 della OWASP API Security Top 10 2023	125
63	Diagramma con alias CPE per sistema operativo e applicazione . . .	131
64	Oggetti digitali impliciti in un file Terraform	133

1 Introduzione

1.1 Contesto e motivazione

Il Web Semantico fornisce un insieme di standard del World Wide Web Consortium (W3C) per rappresentare le informazioni come conoscenza in un formato machine-readable; queste specifiche includono il Resource Description Framework (RDF) (Cyganiak, Wood, e Lanthaler 2014), il Web Ontology Language (OWL) (W3C OWL Working Group 2012) e lo SPARQL Protocol and RDF Query Language (SPARQL) (Harris e Seaborne 2013).

La conoscenza è quindi rappresentata come un grafo di nodi interconnessi, anche detto Knowledge Graph (KG) (Hogan et al. 2021).

Nell'ultimo decennio, il numero di progetti che utilizzano il Web Semantico in ambito cybersecurity è aumentato. L'esempio più importante è il progetto MITRE D3FEND (Detection, Denial, and Disruption Framework Empowering Network Defense) (MITRE Corporation 2021), che fornisce un framework per descrivere e analizzare le tecniche difensive ed offensive in ambito cybersecurity (`d3f:DefensiveTechnique` ed `d3f:OffensiveTechnique`) usando una terminologia indipendente dai vendor.

Un altro esempio di progetti che utilizzano il Web Semantico è quello relativo ai cataloghi legali, come EUR-Lex e Normattiva, dove le leggi sono rappresentate come Uniform Resource Identifier (URI) (Berners-Lee, Fielding, e Masinter 2005). In questo contesto, troviamo anche progetti orientati alla conformità normativa, come il Data Privacy Vocabulary (DPV) (Pandit, Esteves, Golpayegani, et al. 2026) e la sua estensione per il GDPR (European Union 2016).

In questo elaborato, esploreremo la “democratizzazione” dell'uso dei KG per la cybersecurity attraverso interfacce che permettono agli utenti di migliorare la postura di sicurezza dei loro sistemi utilizzando una base di conoscenza condivisa.

Oggi i diagrammi architetturali sono oggetti grafici non interrogabili, e l'individuazione delle tecniche offensive richiede un'analisi manuale dipendente dall'esperien-

za del progettista, dove le basi di conoscenza a supporto (e.g., dataset MITRE, DPV) non sono accessibili a chi non ha familiarità con RDF e SPARQL.

Ci chiediamo quindi: è possibile derivare in modo sistematico e riproducibile, senza intervento manuale, l'insieme delle tecniche offensive applicabili a un'architettura IT, a partire da una sua descrizione testuale annotata con entità di un KG di cybersecurity?

Nel 2023 è stato presentato un primo concept di riuso dei cybersecurity KG tramite sintassi Mermaid.js (Polli 2023), concept che ha poi portato ad alcuni contributi al progetto MITRE D3FEND¹.

Presenteremo degli strumenti basati su KG per supportare la progettazione sicura di infrastrutture IT in cui le architetture non vengono “disegnate” con strumenti grafici (e.g., PowerPoint, draw.io), ma rappresentate tramite dei diagrammi semantici in cui le componenti vengono associate ai vari KG - sia di tipo tecnico (e.g., D3FEND) che giuridico (e.g., DPV, GDPR) - anche tramite specifiche mappature tra le entità di D3FEND e DPV.

Renderemo accessibili i contenuti dei KG tramite delle estensioni per Visual Studio Code (D3FEND Language, GDPR Language), mentre supporteremo la progettazione sicura di infrastrutture IT tramite un'applicazione web (Design & D3FEND) (Polli 2026) che permette di creare dei diagrammi architetturelari semantici e navigare visualmente i KG derivati. Tutte le implementazioni sono open source e disponibili su GitHub.

Abbiamo scelto questo approccio perché si basa su strumenti usati quotidianamente dagli specialisti IT, come Mermaid.js (Mermaid Contributors 2014) e Visual Studio Code: integrare i KG nei processi di progettazione e sviluppo richiede quindi uno sforzo di apprendimento minimo.

Infine, vedremo come derivare in modo sistematico le tecniche offensive applicabili alle componenti di un'architettura, già a partire dalla fase di progettazione.

Questo lavoro non è senza sfide, a causa della difficoltà di rappresentare secondo

¹Si veda la sezione “Committing Contributors” della pagina <https://d3fend.mitre.org/contribute/>. Consultato 26 agosto 2026.

prospettive diverse - ed a diversi livelli di dettaglio - le infrastrutture informatiche, nonché della varietà delle scelte di modellazione delle entità e delle loro relazioni.

1.2 Obiettivi

L'obiettivo del project work è quello di individuare, a partire da un'architettura IT di esempio, le `d3f:OffensiveTechnique` che impattano sulle sue componenti. L'obiettivo è raggiunto se è possibile:

1. esprimere un diagramma architetturale in una notazione testuale (Mermaid.js) le cui componenti e relazioni sono annotate con entità di D3FEND e DPV, e tradurlo in un grafo RDF interrogabile, tramite il supporto delle applicazioni sviluppate;
2. individuare le `d3f:DefensiveTechnique` applicabili alle componenti annotate in modalità assistita e tramite query SPARQL;
3. derivare dal grafo così ottenuto, tramite query SPARQL e senza intervento manuale, l'elenco delle `d3f:OffensiveTechnique` impattanti sulle componenti annotate;
4. associare all'architettura i rischi indicati nella OWASP API Security Top 10 2023 (OWASP Foundation 2023).

Gli strumenti sviluppati per raggiungere questo obiettivo sono:

- due estensioni per Visual Studio Code (D3FEND Language, GDPR Language) con funzionalità di auto-completamento e navigazione tra le entità e le relazioni di KG;
- l'applicazione “Design & D3FEND” per la creazione e la navigazione visuale di diagrammi architetture rappresentati come KG.

Il lavoro si è svolto in modo agile, iterando le seguenti fasi sui diversi diagrammi architetture di esempio presenti in <https://github.com/ioggstream/d3sign/> (Polli 2026):

1. analisi delle ontologie di riferimento (D3FEND, DPV, ontologie GDPR) per individuare le classi e le relazioni utili a descrivere componenti IT;
2. definizione di una notazione testuale per i diagrammi architetturali semantici e delle regole di traduzione in RDF;
3. implementazione degli strumenti (estensioni Visual Studio Code, applicazione web);
4. validazione degli esempi, con verifica dei criteri 1-3 elencati sopra;

a cui si aggiunge uno step di validazione finale descritto in sez. 5.4, *Caso d'uso: individuare le tecniche ATT&CK applicabili ad un'architettura annotata con D3FEND* per il quale sono riportati il diagramma annotato, le query e i risultati ottenuti in termini di copertura dei rischi (OWASP Foundation 2023).

1.2.1 Contributo originale

Questo lavoro contribuisce all'attuale panorama della progettazione sicura introducendo:

1. una notazione testuale per rappresentare diagrammi architetturali semantici, con le relative regole di traduzione in RDF (vedi sez. 4, *Progettazione sicura con grafi architetturali basati su D3FEND*);
2. una mappatura tra `d3f:DefensiveTechnique` e concetti DPV in sez. 2.5, *Data Privacy Vocabulary (DPV)*;
3. una catena di derivazione tra dataset OWASP (API Security Top 10 2023 (OWASP Foundation 2023), Web Application Security (OWASP Foundation 2025)) e `d3f:DigitalArtifact` basata su dataset ufficiali (OWASP, CWE, CAPEC);
4. l'applicazione open source Design & D3FEND per la progettazione sicura basata su grafi architetturali (Polli 2026).

Rispetto al prototipo del 2023, questo lavoro introduce:

- un approccio metodologico rigoroso, derivato dall’analisi dei vari dataset del MITRE associati (vedi sopra i punti 1, 3)
- l’integrazione con KG giuridici e di conformità (vedi sez. 2.5, *Data Privacy Vocabulary (DPV)* e sez. 2.2.1, *D3FEND e i controlli NIST SP 800-53*);
- una separazione netta tra editing e rendering, che permette di utilizzare l’applicazione anche per visualizzare KG generati indipendentemente da Mermaid.js.

Rispetto a D3FEND CAD (vedi il confronto in sez. 3.1.1, *D3FEND CAD*), questo lavoro permette di creare KG in modo indiretto, a partire dalla documentazione di progetto, rendendo l’intera infrastruttura versionabile, diffabile, dove il KG può essere eventualmente integrato in una CI.

1.3 Struttura dell’elaborato

1. Introduzione: ontologie e rappresentazione della conoscenza.
2. Knowledge Graph per la cybersecurity: D3FEND, Data Privacy Vocabulary, altre ontologie.
3. Diagrammi architetturali: rappresentazione delle infrastrutture Information Technology (IT) e delle loro componenti.
4. Progettazione sicura con grafi architetturali basati su D3FEND.
5. Implementazioni: estensioni Visual Studio Code, applicazione web Design & D3FEND e caso d’uso di validazione dei criteri.
6. Conclusioni e sviluppi futuri.

1.4 Convenzioni editoriali

I termini tecnici di uso comune (e.g., Knowledge Graph, triple, namespace) sono mantenuti in inglese.

Ogni acronimo è riportato per esteso alla prima occorrenza, seguito dall’acronimo tra parentesi; l’elenco completo è in sez. 1.5, *Definizioni e Acronimi*.

I riferimenti bibliografici sono riportati per la prima volta in linea. Le volte successive, qualora sia utile un riferimento ad una sezione specifica, viene indicata anche la sezione - comprensiva di un collegamento ipertestuale. I riferimenti bibliografici che non hanno un anno di pubblicazione (e.g., <https://d3fend.mitre.org/cad/docs/>) riportano la data di ultimo accesso.

Per la terminologia si distingue costantemente tra:

- **grafo**: la rappresentazione della conoscenza in RDF (il Knowledge Graph);
- **diagramma**: una rappresentazione visuale, resa con Mermaid.js (Mermaid Contributors 2014).

Le immagini dei diagrammi sono create tramite Mermaid.js.

A seconda delle necessità, le gerarchie di classi e le relazioni ontologiche possono essere rappresentate visivamente come:

- classi collegate da archi tratteggiati, per indicare la relazione di sottoclasse (`rdfs:subClassOf`);

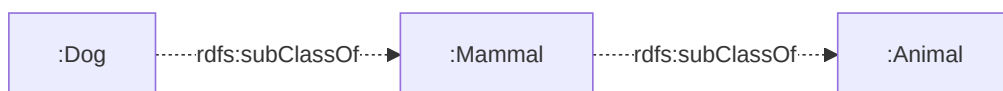


Figura 1: Sottoclassi rappresentate come archi tratteggiati

- classi nidificate per indicare la relazione di sottoclasse (`rdfs:subClassOf`);

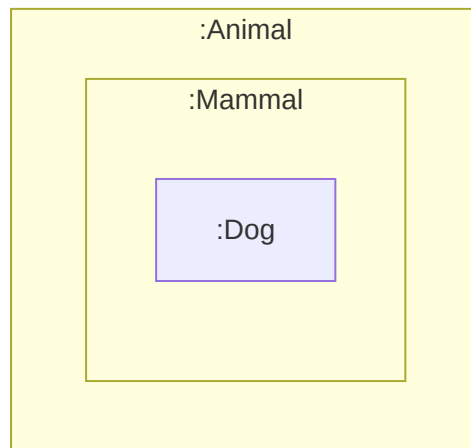


Figura 2: Sottoclassi rappresentate come sottoinsiemi

I blocchi di codice usano un font a 8pt con interlinea singola. Le righe più lunghe di 72 caratteri sono mandate a capo secondo la strategia “single backslash” di RFC 8792 (Watsen et al. 2020). Eventuali comandi shell sono preceduti dal prompt \$ a delimitare l’invocazione e seguiti dall’eventuale output.

Le note a piè di pagina sono utilizzate a riferimenti a risorse online; le espansioni degli acronimi sono inline e raccolte in sez. 1.5, *Definizioni e Acronimi* I rimandi interni sono riferimenti alle sezioni dell’elaborato.

1.4.1 Notazione RDF

I termini di un vocabolario sono indicati con la notazione Compact URI (cURIE) (Birbeck e McCarron 2010) (e.g., `d3f:WebApplication`, `rdfs:subClassOf`); gli Internationalized Resource Identifier (IRI) (Dürst e Suignard 2005) assoluti sono racchiusi tra `<e>` (e.g., `<http://d3fend.mitre.org/ontologies/d3fend.owl#>`).

Il contesto di default - sia per RDF che per SPARQL - include i seguenti prefissi:

```
@prefix d3f: <http://d3fend.mitre.org/ontologies/d3fend.owl#> .  
@prefix rdfs: <http://www.w3.org/2000/01/rdf-schema#> .  
@prefix owl: <http://www.w3.org/2002/07/owl#> .  
@prefix xsd: <http://www.w3.org/2001/XMLSchema#> .
```

1.5 Definizioni e Acronimi

Questa sezione non è una bibliografia: raccoglie i termini e gli acronimi utilizzati nell'elaborato, ciascuno con il riferimento alla fonte che lo definisce.

1.5.1 Definizioni

- code list: vocabolario controllato che elenca i valori ammessi per un campo o un attributo (e.g., i codici dei paesi di ISO 3166 (International Organization for Standardization s.d.b)).
- diagramma: rappresentazione visuale di un grafo o di un'architettura, resa con Mermaid.js (Mermaid Contributors 2014).
- grafo: rappresentazione della conoscenza come insieme di nodi collegati da archi; in questo elaborato è sempre un grafo RDF (Cyganiak, Wood, e Lanthaler 2014).
- MITRE: nome proprio dell'organizzazione che sviluppa D3FEND e ATT&CK; non è un acronimo (MITRE Corporation s.d.c).
- namespace: contesto, identificato da un IRI, che disambigua i termini di un vocabolario (Cyganiak, Wood, e Lanthaler 2014).
- ontologia: vocabolario controllato che descrive un dominio della conoscenza con un linguaggio formale, tipicamente OWL (W3C OWL Working Group 2012).
- RDF Schema: estensione di RDF che introduce le classi (`rdfs:Class`) e le relazioni di sottoclasse e sottoproprietà (Brickley e Guha 2014).
- serializzazione: formato testuale con cui un grafo RDF viene scritto e scambiato: Turtle (Beckett et al. 2014), TriG (Carothers e Seaborne 2014), JSON-LD (Kellogg, Champin, e Longley 2020), RDF/XML (Gandon e Schreiber 2014).
- thesaurus: vocabolario controllato che organizza i concetti in una gerarchia di relazioni *broader/narrower*, tipicamente espressa in SKOS (Miles e Bechhofer 2009).

- tripla: l'asserzione elementare di RDF, nella forma soggetto-predicato-oggetto (Cyganiak, Wood, e Lanthaler 2014).
- vocabolario controllato: insieme di termini identificati da URI e definiti in modo esplicito, così da poter essere riutilizzati senza ambiguità (Cyganiak, Wood, e Lanthaler 2014).

1.5.2 Acronimi

- API: Application Programming Interface (National Institute of Standards and Technology s.d.a)
- ATLAS: Adversarial Threat Landscape for Artificial-Intelligence Systems (MITRE Corporation s.d.d)
- ATT&CK: Adversarial Tactics, Techniques, and Common Knowledge (MITRE Corporation s.d.e)
- AWS: Amazon Web Services (Amazon Web Services, Inc. s.d.)
- C4: Context, Containers, Components, Code (Brown 2018)
- CAD: Cyber Attack-Defense Utility for Capturing Engagements and Understandable Scenarios (MITRE Corporation 2026)
- CAPEC: Common Attack Pattern Enumeration and Classification (MITRE Corporation s.d.a)
- CDN: Content Delivery Network (National Institute of Standards and Technology s.d.a)
- CI: Continuous Integration (National Institute of Standards and Technology s.d.a)
- CIA: Confidentiality, Integrity, Availability (International Organization for Standardization e International Electrotechnical Commission 2018)
- CPE: Common Platform Enumeration (National Institute of Standards and Technology s.d.a)
- CSF: Cybersecurity Framework (National Institute of Standards and Technology 2024)
- cURLe: Compact URI (Birbeck e McCarron 2010)

- CVE: Common Vulnerabilities and Exposures (The CVE Program s.d.)
- CWE: Common Weakness Enumeration (MITRE Corporation s.d.b)
- D3FEND: Detection, Denial, and Disruption Framework Empowering Network Defense (MITRE Corporation 2021)
- DPIA: Data Protection Impact Assessment (European Union 2016)
- DPV: Data Privacy Vocabulary (Pandit, Esteves, Golpayegani, et al. 2026)
- EU: European Union (European Union 2016)
- FNCDP: Framework Nazionale per la Cybersecurity e la Data Protection (CIS Sapienza e Laboratorio Nazionale di Cybersecurity CINI 2025)
- GDPR: General Data Protection Regulation (European Union 2016)
- HTTP: Hypertext Transfer Protocol (Fielding, Nottingham, e Reschke 2022)
- IaaS: Infrastructure as a Service (Mell e Grance 2011)
- IaC: Infrastructure as Code (National Institute of Standards and Technology s.d.a)
- ICS: Industrial Control Systems (MITRE Corporation s.d.e)
- IDE: Integrated Development Environment (National Institute of Standards and Technology s.d.a)
- IEC: International Electrotechnical Commission (International Organization for Standardization e International Electrotechnical Commission 2018)
- IRI: Internationalized Resource Identifier (Dürst e Suignard 2005)
- ISO: International Organization for Standardization (International Organization for Standardization s.d.a)
- IT: Information Technology (National Institute of Standards and Technology s.d.a)
- JSON: JavaScript Object Notation (Bray 2017)
- JSON-LD: JSON for Linked Data (Kellogg, Champin, e Longley 2020)
- JWT: JSON Web Token (Jones, Bradley, e Sakimura 2015)
- KG: Knowledge Graph (Hogan et al. 2021)
- LAN: Local Area Network (National Institute of Standards and Technology s.d.a)
- LCSH: Library of Congress Subject Headings (Library of Congress s.d.)

- LPG: Labeled Property Graph (Neo4j, Inc. s.d.)
- NIST: National Institute of Standards and Technology (National Institute of Standards and Technology s.d.a)
- NPE: Non-Person Entity (MITRE Corporation 2021)
- NSA: National Security Agency (National Security Agency s.d.)
- OWL: Web Ontology Language (W3C OWL Working Group 2012)
- PDF: Portable Document Format (National Institute of Standards and Technology s.d.a)
- RDF: Resource Description Framework (Cyganiak, Wood, e Lanthaler 2014)
- RFC: Request for Comments²
- SKOS: Simple Knowledge Organization System (Miles e Bechhofer 2009)
- SP: Special Publication (National Institute of Standards and Technology 2020)
- SPA: Single Page Application (Mozilla Developer Network s.d.)
- SPARQL: SPARQL Protocol and RDF Query Language (Harris e Seaborne 2013)
- SPARTA: Space Attack Research and Tactic Analysis (The Aerospace Corporation s.d.)
- SQL: Structured Query Language (ISO/IEC, s.d.)
- UCO: Unified Cyber Ontology (Unified Cyber Ontology s.d.)
- UI: User Interface (National Institute of Standards and Technology s.d.a)
- UML: Unified Modeling Language (Object Management Group 2017)
- URI: Uniform Resource Identifier (Berners-Lee, Fielding, e Masinter 2005)
- URL: Uniform Resource Locator (Berners-Lee, Fielding, e Masinter 2005)
- W3C: World Wide Web Consortium (World Wide Web Consortium s.d.)
- WAF: Web Application Firewall (National Institute of Standards and Technology s.d.a)
- WAN: Wide Area Network (National Institute of Standards and Technology s.d.a)
- XML: Extensible Markup Language (Bray et al. 2008)

²La serie di documenti Request for Comments è pubblicata da RFC Editor, <https://www.rfc-editor.org/>. Consultato 1 settembre 2026.

1.6 Limitazioni

In questo documento non affronteremo i seguenti temi:

- la rappresentazione delle proprietà delle relazioni tra i nodi (e.g., i Labeled Property Graph (LPG) (Neo4j, Inc. s.d.) o RDF 1.2 (Hartig et al. 2026)). Ci limiteremo ad illustrare le problematiche di fondo e le complessità implementative;
- il tema dell'analisi del rischio, che richiederebbe un lavoro a parte. Il KG fornisce però un punto di partenza consistente per questo tipo di analisi;
- le estensioni di D3FEND legate ai temi dello spazio, dell'OT e del mondo mobile descritti in sez. 2.2.9, *Estensioni: SPARTA, ATLAS, ATT&CK Mobile e ATT&CK ICS*.
- i collegamenti tra il NIST Cybersecurity Framework (CSF) 2.0 (National Institute of Standards and Technology 2024), D3FEND, e il Framework Nazionale per la Cybersecurity e la Data Protection (FNCDP) (CIS Sapienza e Laboratorio Nazionale di Cybersecurity CINI 2025).

Le limitazioni relative ai risultati presentati sono invece indicate in sez. 6.2, *Limiti*.

1.7 Prerequisiti

1.7.1 Grafi e rappresentazione della conoscenza

L'Encyclopédie (Diderot e d'Alembert 1751) è stata uno dei primi tentativi moderni di organizzare la conoscenza, e di renderla disponibile al pubblico.

Le sue voci possono essere sostanzialmente viste come una collezione di frasi, ciascuna della forma



Figura 3: Una frase come grafo, tra soggetto, predicato e oggetto

Un suo erede moderno è Wikipedia, che ha reso la conoscenza disponibile a tutti in un formato digitale. La pagina dell'Encyclopédie su Wikipedia è disponibile all'indirizzo <https://en.wikipedia.org/wiki/Encyclop%C3%A9die>.

Ogni termine è identificato da un IRI assoluto. L'IRI estende il concetto di URI.

Possiamo vedere che tutti i termini di Wikipedia in inglese sono prefissati da <https://en.wikipedia.org/wiki/> e quelli in italiano da <https://it.wikipedia.org/wiki/>: Wikipedia pubblica diverse definizioni di ogni termine, a seconda della lingua.

In realtà, il prefisso identifica il nome del vocabolario (Wikipedia in inglese o in italiano), mentre il suffisso identifica il termine.

Ad esempio, un'ipotetica voce enciclopedica su Python potrebbe essere:

Python is a programming language designed by Guido van Rossum and named after the Monty Python comedy group.

Possiamo rappresentare questo testo come un grafo, dove l'oggetto di una frase può essere il soggetto di un'altra frase.

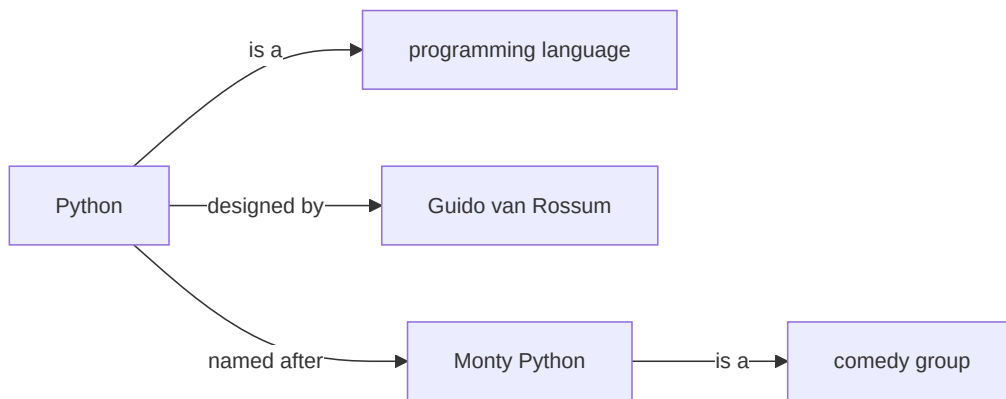


Figura 4: Una voce enciclopedica su Python rappresentata come grafo

Ci sono anche altre enciclopedie sul web, come DBpedia. La voce su Python di DBpedia è disponibile all'URL:

- [https://dbpedia.org/page/Python_\(programming_language\)](https://dbpedia.org/page/Python_(programming_language))

mentre quella di Wikidata, all'URL:

- <https://www.wikidata.org/wiki/Q28865>

Poiché possiamo rappresentare le frasi come grafi, è naturale rappresentare e organizzare la conoscenza in una struttura a grafo, dove ogni nodo è un concetto e gli archi sono le relazioni tra di essi.

Ecco un grafo che rappresenta un estratto della voce su Python di DBpedia:

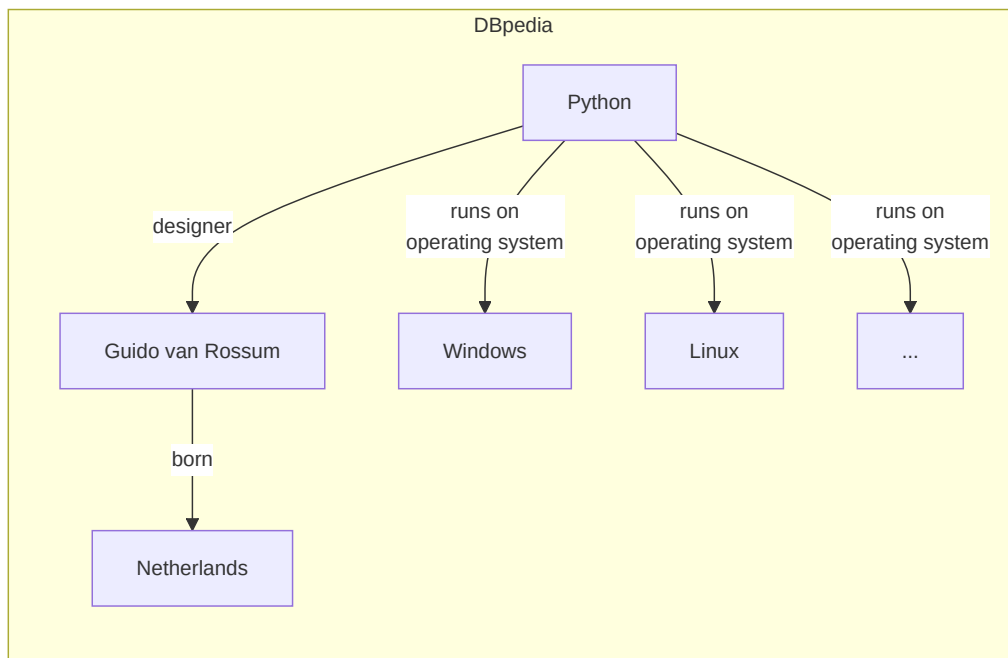


Figura 5: Estratto della voce su Python di DBpedia

Un altro esempio di grafo è quello della voce su Guido van Rossum di Wikidata:

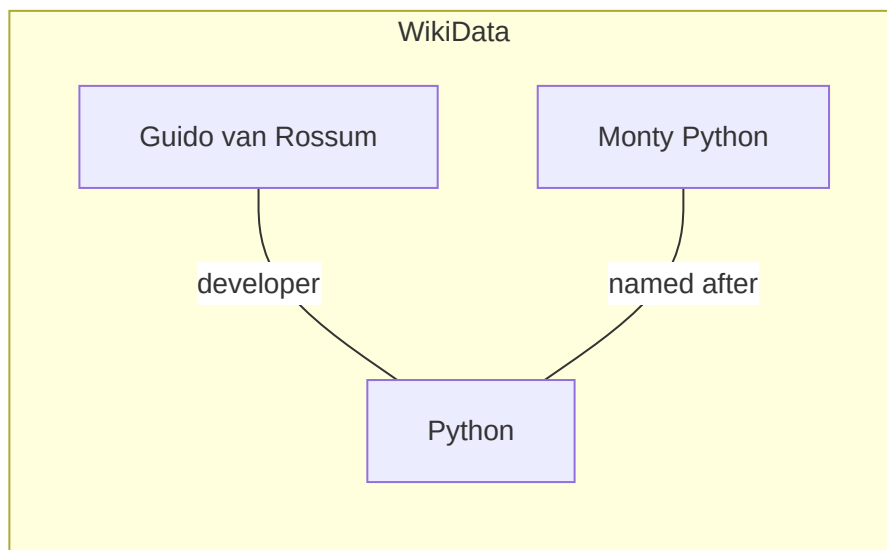


Figura 6: Estratto della voce su Guido van Rossum di Wikidata

1.8 Il Web Semantico

Possiamo utilizzare ulteriori conoscenze - sfruttando i riferimenti incrociati esistenti tra DBpedia e Wikidata, oppure utilizzando le nostre conoscenze - per collegare i due grafi.

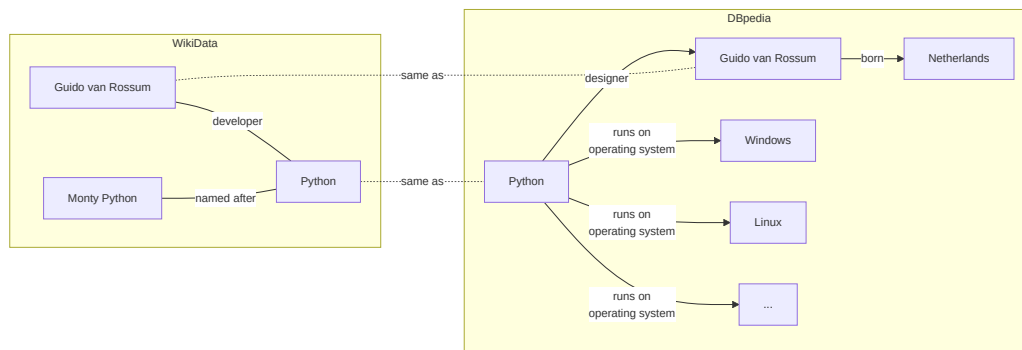


Figura 7: Collegamento tra i grafi di DBpedia e Wikidata

Questa è la base del Web Semantico, dove la conoscenza è rappresentata in un formato machine-readable.

Ma è anche la base del Web stesso (ad esempio, vedi Web Linking (Nottingham 2017)).

1.8.1 Resource Description Framework (RDF): machine-readable knowledge

Sia DBpedia che Wikipedia utilizzano RDF: il primo in modo diretto, il secondo tramite il progetto Wikidata che funge da storage per i suoi dati strutturati (Wikidata s.d.).

RDF è un linguaggio formale standardizzato dal W3C per rappresentare la conoscenza sul web in modo machine-readable utilizzando triple soggetto-predicato-oggetto.

RDF ha diversi formati di serializzazione, come Turtle (Beckett et al. 2014) e la sua estensione TriG (Carothers e Seaborne 2014), [JSON for Linked Data \(JSON-LD\)](#) (Kellogg, Champin, e Longley 2020), e l'Extensible Markup Language (XML) (Gandon e Schreiber 2014).

In questo elaborato useremo principalmente il formato Turtle e la sua estensione TriG, dove una frase è espressa come una tripla:

subject predicate object .

Soggetti e predicati sono identificati da IRI (Cyganiak, Wood, e Lanthaler 2014, sez. 3.2), in modo che siano sempre univoci, mentre gli oggetti possono essere IRI o letterali (come stringhe, numeri, date, ecc.).

Gli IRI sono racchiusi tra < e >.

Esempio: la voce su Python di DBpedia può essere espressa in Turtle come segue:

```
===== NOTE: '\' line wrapping per RFC 8792 =====  
<http://dbpedia.org/resource/Python_(programming_language)>  
  <http://dbpedia.org/ontology/designer> <http://dbpedia.org/resource/Guido_van_Rossum> .
```

RDF usa prefissi (namespace prefixes) per abbreviare gli IRI utilizzando la sintassi cURLe (Birbeck e McCarron 2010). In Turtle questo avviene tramite la direttiva @prefix:

```
@prefix dbr: <http://dbpedia.org/resource/> .  
@prefix dbo: <http://dbpedia.org/ontology/> .  
  
dbr:Python_(programming_language)  
  dbo:designer dbr:Guido_van_Rossum .
```

Chi ha familiarità con lo Structured Query Language (SQL) (ISO/IEC, s.d.) può vedere un grafo come una tabella, dove ogni riga è una frase.

Subject	Predicate	Object
dbr:Python	rdf:type	dbo:ProgrammingLanguage
dbr:Python	dbo:designer	dbr:gvr
dbr:Python	dbo:runsOn	dbr:win
dbr:Python	dbo:runsOn	dbr:linux

Il prefisso identifica la fonte della definizione del termine (che chiameremo *vocabulary*), mentre il suffisso identifica il termine.

Gli elementi principali di RDF sono:

- element (IRI, blank node e literal);

- triple (subject-predicate-object);
- graph (insieme di triple).
- vocabulary (grafo contenente termini e le loro definizioni).

Ulteriori informazioni su RDF sono disponibili in (Cyganiak, Wood, e Lanthaler 2014). In particolare si vedano le seguenti sezioni:

- Graph data model (Cyganiak, Wood, e Lanthaler 2014, sez. 1.1)
- Graphs (Cyganiak, Wood, e Lanthaler 2014, sez. 3)

Possiamo vedere i predicati come delle relazioni tra i nodi. I predicati principali sono:

- `rdf:type`, o il suo alias `a` (Beckett et al. 2014, sez. 2.4), che indica l'appartenenza di una risorsa ad una classe; e.g.,

```
dbr:Python rdf:type dbo:ProgrammingLanguage .
```

- `rdfs:subClassOf`, che indica che tutti gli elementi di una classe sono anche elementi di un'altra classe; e.g.,

```
dbo:ProgrammingLanguage rdfs:subClassOf dbo:Software .
```

- `rdfs:subPropertyOf`, che indica che tutti gli elementi di una proprietà sono anche elementi di un'altra proprietà; e.g.,

```
dbo:designer rdfs:subPropertyOf dbo:creator .
```

Per evitare la ripetizione di soggetto e predicato, Turtle (Beckett et al. 2014) permette di raggruppare:

- le triple con lo stesso soggetto separando i predicati con punto e virgola (;);
- le triple con lo stesso soggetto e predicato in un'unica tripla, separando gli oggetti con una virgola (,); e.g.,

```
dbr:Python
  a dbo:ProgrammingLanguage ;
  dbo:designer dbr:Guido_van_Rossum ;
  dbo:runsOn dbr:Windows, dbr:Linux .
```

I predicati che rappresentano una relazione tra due risorse possono avere delle proprietà specifiche, come la riflessività, la simmetria e la transitività.

Ad esempio, i predicati `rdfs:subClassOf` e `rdfs:subPropertyOf` sono transitivi (Brickley e Guha 2014, sez. 3.5), quindi un file contenente

```
:Snowball a db:Dog .  
db:Dog rdfs:subClassOf db:Mammal .
```

è semanticamente equivalente a un file contenente

```
:Snowball a db:Dog .  
db:Dog rdfs:subClassOf db:Mammal .  
:Snowball a db:Mammal .
```

Notazione Turtle e TriG Turtle permette di serializzare un singolo grafo come sequenza di triple.

TriG (Carothers e Seaborne 2014) è un'estensione di Turtle che permette di serializzare un intero dataset RDF come sequenza di grafi, come indicato in (Carothers e Seaborne 2014, sez. 1) e (Carothers e Seaborne 2014, sez. 5.3).

Alla sintassi di Turtle, TriG aggiunge i *graph statement* (Carothers e Seaborne 2014, sez. 2.2): blocchi di triple delimitati da graffe ed etichettati con un IRI che identifica il grafo, introdotti dalla parola chiave opzionale `GRAPH`. Se l'etichetta è omessa, le triple del blocco appartengono al “default graph”, insieme alle triple che non sono racchiuse in alcun blocco (Carothers e Seaborne 2014, sez. 2.1). Un documento Turtle è quindi anche un documento TriG le cui triple sono tutte nel “default graph”.

```
@prefix dbr: <http://dbpedia.org/resource/> .  
@prefix dbo: <http://dbpedia.org/ontology/> .  
  
# Triple fuori da ogni blocco: default graph.  
dbr:Python a dbo:ProgrammingLanguage .  
  
GRAPH <https://example.org/graphs/dbpedia> {  
  dbr:Python dbo:designer dbr:Guido_van_Rossum .  
}  
  
GRAPH <https://example.org/graphs/local> {  
  dbr:Python dbo:designer dbr:Someone_Else .  
}
```

L'etichetta del grafo mantiene distinte le triple provenienti da fonti diverse e permette di risalire alla loro provenienza dopo l'aggregazione. Questa proprietà verrà usata per unire in un'unica vista grafi di origine differente.

Per elencare i grafi presenti in un dataset RDF, si può usare la query SPARQL seguente:

```
SELECT DISTINCT ?g
WHERE {
  GRAPH ?g { }
```

1.8.2 Vocabolari Controllati

I Vocabolari Controllati (Controlled Vocabulary) nascono in ambito biblioteconomico e documentale: nel Web Semantico utilizzano gli URI per disambiguare dei termini. Ogni termine, che può essere un concetto, un oggetto o un'entità, è quindi associato ad una risorsa (vedi (URI Planning Interest Group, W3C/IETF s.d.)) ed identificato da un URI prefissato dal nome del vocabolario: questo fornisce un contesto (namespace).

Esempio: il termine “Dog” secondo il vocabolario <https://dbpedia.org/data/> è definito dalla stringa “The dog is a four legged animal” e può essere referenziato tramite l'URI <https://dbpedia.org/data/Dog>.

I Vocabolari Controllati vengono descritti utilizzando RDF e le sue estensioni, come RDF Schema e OWL, che sono caratterizzati da concetti come classi (e.g., `rdfs:Class`), sottoclassi (e.g., `rdfs:subClassOf`) ed elementi di teoria degli insiemi (e.g., `owl:disjointWith`, `owl:inverseOf`, `owl:intersectionOf`, ...).

Esistono diversi tipi di Vocabolari Controllati, tra cui:

- Thesaurus: un thesaurus è un vocabolario controllato che fornisce una struttura gerarchica di concetti, con relazioni tra i concetti come “broader” (più ampio) e “narrower” (più specifico). Un esempio di thesaurus è il Library of Congress Subject Headings (LCSH) (Library of Congress s.d.).

- Code List: una code list è un vocabolario controllato che fornisce un elenco di valori predefiniti per un determinato campo o attributo. Un esempio di code list è il codice della International Organization for Standardization (ISO) 3166 per i paesi (International Organization for Standardization s.d.b).
- Ontologie: un'ontologia è un vocabolario controllato che descrive un dominio specifico della conoscenza. Le ontologie sono spesso utilizzate per rappresentare la conoscenza in modo più dettagliato e preciso rispetto ai thesauri o alle code list, ed utilizzano OWL per definire le classi, le proprietà e le relazioni tra i concetti.

Esempio: un'ontologia minimale che definisce il concetto di “Cane” in formato Turtle.

```
@prefix db: <http://ontology.example/> .
@prefix owl: <http://www.w3.org/2002/07/owl#> .
@prefix rdfs: <http://www.w3.org/2000/01/rdf-schema#> .

db:Dog
  a owl:Class;
  rdfs:subClassOf db:Mammal;
  rdfs:label "dog"@en, "cane"@it;
  rdfs:comment "A dog is a four legged animal"@en
.
db:petName
  a owl:DatatypeProperty;
  rdfs:domain db:Dog;
  rdfs:label "name"
.
db:Mammal
  a owl:Class;
  rdfs:subClassOf db:Animal;
  owl:disjointWith db:Fish
.
```

RDF si focalizza sulla conoscenza e sulle relazioni semantiche: dall'ontologia di esempio sopra possiamo dedurre che un “cane” è un mammifero ma non un pesce, e che può avere un nome. Le sue estensioni, come RDF Schema (Brickley e Guha 2014) e OWL (W3C OWL Working Group 2012), forniscono il concetto di classe (`rdfs:Class`) per “dividere le risorse in gruppi”, nonché una serie di relazioni di base che possono o meno avere proprietà specifiche (e.g., riflessività, simmetria, transitività, ecc.).

Le ontologie sono sviluppate per descrivere con un linguaggio formale un dominio specifico, cercando di classificare le entità “nella loro essenza”; questo rimando alla

filosofia greca va contestualizzato, poiché l'ambito resta sempre il mondo reale: in pratica, le ontologie sono strumenti per descrivere lo spazio di un problema in modo riutilizzabile, a prova di obsolescenza e indipendente dalle possibili soluzioni implementative.

Da una prospettiva Domain-Driven (vedi (Evans 2003)), gli esperti di Semantic Web si concentrano sul dominio, mentre gli esperti tecnici si concentrano sui contesti limitati e dinamici. RDF può essere un utile strumento di Domain Modeling (Fowler 2002), a supporto della definizione di un Ubiquitous Language (Evans 2003, chapter 2).

1.8.3 Labeled Property Graph e RDF 1.2

RDF 1.1 non è l'unico formato per rappresentare la conoscenza in un grafo. Un'alternativa è rappresentare la conoscenza come un LPG, dove i nodi e gli archi possono avere etichette e proprietà: questo formato è utilizzato da database come Neo4j (Neo4j, Inc. s.d.).

Di seguito un LPG che rappresenta due persone connesse da una relazione di amicizia, con le loro proprietà.

```
CREATE (alice:Person {name: 'Alice', age: 30})
CREATE (bob:Person {name: 'Bob', age: 25})
CREATE (carol:Person {name: 'Carol', age: 35})
CREATE (alice)-[:FRIEND {since: 2020}]->(bob)
CREATE (alice)-[:FRIEND {since: 2021}]->(carol)
```

Questo grafo può essere visualizzato come segue:

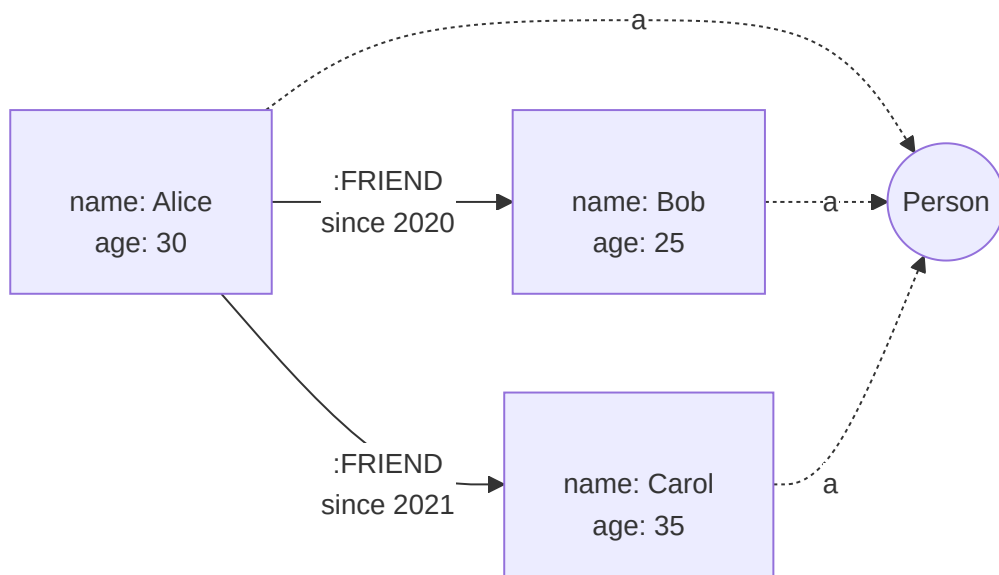


Figura 8: Rappresentazione LPG di una relazione di amicizia

Vediamo come la differenza tra RDF e LPG sia nella rappresentazione delle proprietà degli archi: in RDF non è possibile associare direttamente delle label agli archi.

Inoltre, le proprietà dei nodi non devono essere necessariamente nodi stessi (i.e., dire che

```
(alice:Person {name: 'Alice'})
```

non implica che il property name sia associato ad un URI).

Per ottenere un risultato simile è necessario creare un nodo intermedio che rappresenti la relazione e associare ad esso le proprietà desiderate.

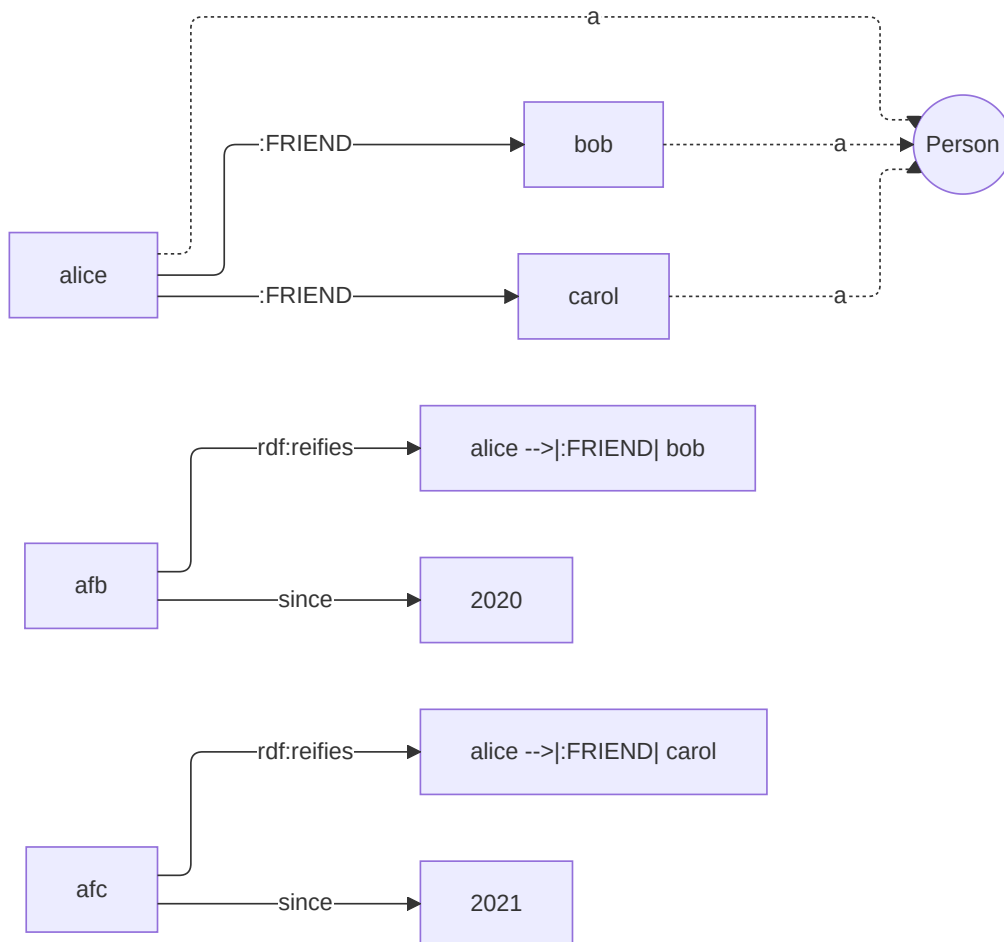


Figura 9: Rappresentazione RDF di una relazione di amicizia

Attualmente è in corso un processo di standardizzazione di RDF 1.2 (vedi (Hartig et al. 2026)) che include una sintassi di serializzazione.

```
:alice a :Person ;
      :name "Alice" ;
      :age 30 .
:bob a :Person ;
     :name "Bob" ;
     :age 25 .
:carol a :Person ;
       :name "Carol" ;
       :age 35 .

:alice :FRIEND :bob {
  :since 2020
} .
:alice :FRIEND :carol {
  :since 2021
} .
```

In formato esteso:

```

:alice a :Person ;
      :name "Alice" ;
      :age 30 .
:bob a :Person ;
     :name "Bob" ;
     :age 25 .
:carol a :Person ;
      :name "Carol" ;
      :age 35 .

:alice :FRIEND :bob .
:alice :FRIEND :carol .

_:afb :rdf:reifies <<(:alice :FRIEND :bob )>> ;
      :since 2020 .
_:afc :rdf:reifies <<(:alice :FRIEND :carol )>> ;
      :since 2021 .

```

In questo elaborato useremo solamente RDF 1.1.

1.8.4 Simbolismo

In questo elaborato useremo la seguente notazione per rappresentare le relazioni tra classi e proprietà:

- $C \sqsubseteq D$ indica che $:C \text{ rdfs:subClassOf* } :D$
- $C \sqsubseteq_1 D$ indica che $:C \text{ rdfs:subClassOf } :D$;
- $C \sqsubseteq \exists r.D$ indica che esiste la tripla $:C \text{ :r } :D$;
- $\text{Anc}(C)$ indica l'insieme degli antenati di C (incluso C stesso), come da SPAR-QL

```

SELECT ?ancestor
WHERE {
  :C rdfs:subClassOf* ?ancestor .
}

```

- $\text{Desc}(C)$ indica l'insieme dei discendenti di C (incluso C stesso);

```

SELECT ?descendant
WHERE {
  ?descendant rdfs:subClassOf* :C .
}

```

- $P \cap S$ indica l'intersezione tra insiemi;

2 Knowledge Graph per la Cybersecurity

2.1 Cybersecurity

Il glossario del NIST (National Institute of Standards and Technology s.d.a) definisce la [cybersecurity](#) come l'insieme delle attività di

prevenzione, protezione e ripristino dei sistemi informatici, delle comunicazioni elettroniche e delle informazioni in essi contenute, al fine di garantirne riservatezza, integrità e disponibilità.

ISO/IEC 27000 (International Organization for Standardization e International Electrotechnical Commission 2018) definisce in modo analogo il concetto di Information Security.

Per chiarezza riportiamo le definizioni dei tre concetti in inglese, direttamente dal glossario:

- [confidentiality](#) (riservatezza): Preserving authorized restrictions on information access and disclosure, including means for protecting personal privacy and proprietary information;
- [integrity](#) (integrità): Guarding against improper information modification or destruction, and includes ensuring information nonrepudiation and authenticity;
- [availability](#) (disponibilità): Ensuring timely and reliable access to and use of information.

Queste tre proprietà costituiscono la cosiddetta triade “CIA”, che individua quindi degli obiettivi da raggiungere.

Il NIST Cybersecurity Framework (CSF) 2.0 (National Institute of Standards and Technology 2024):

- fornisce indicazioni per la gestione dei rischi cyber, ossia collegati alla perdita di riservatezza, integrità e disponibilità dei sistemi e delle informazioni (National Institute of Standards and Technology 2024, sez. 5.2);
- descrive dei risultati tecnologicamente neutrali, classificati in sei Funzioni (Govern, Identify, Protect, Detect, Respond, Recover) (National Institute of Standards and Technology 2024, 31);
- organizza tali Funzioni in categorie e sotto-categorie con una serie di esempi di implementazione³. Essendo il CSF 2.0 tecnologicamente neutrale, gli esempi non sono prescrittivi e non esistono obblighi di adozione verso specifiche misure di sicurezza (National Institute of Standards and Technology 2024, 2).

Sempre il NIST, nel documento (National Institute of Standards and Technology 2020), definisce dei controlli di sicurezza e privacy⁴ che possono essere adottati per mitigare i rischi cyber. Questi controlli possono essere mappati sul CSF 2.0 tramite strumenti quali le Informative Reference (National Institute of Standards and Technology 2026). In particolare, il documento Cybersecurity-Framework-v2.0-to-SP-800-53-Rev-5-2-0 Informative Reference Details⁵ contiene una mappatura informativa tra i controlli NIST Special Publication (SP) 800-53 e le Funzioni CSF 2.0 di cui mostriamo un estratto:

Sottocategoria	Outcome CSF 2.0	Controllo SP 800-53
PR.PS-04	Log records are generated and made available for continuous monitoring	AU-11
PR.PS-04	Log records are generated and made available for continuous monitoring	AU-12
PR.PS-04	Log records are generated and made available for continuous monitoring	SA-15(13)
PR.PS-05	Installation and execution of unauthorized software are prevented	CM-07(02)

Sul NIST CSF 2.0 si basa il Framework Nazionale per la Cybersecurity e la Data Protection (FNCDP) (CIS Sapienza e Laboratorio Nazionale di Cybersecurity CINI

³<https://www.nist.gov/document/csf-20-implementation-examples-xlsx>. Consultato il 26 agosto 2026.

⁴https://csrc.nist.gov/glossary/term/security_control. Consultato il 26 agosto 2026.

⁵I dati della mappatura sono reperibili all'indirizzo <https://csrc.nist.gov/projects/olir/informative-reference-catalog/details?referenceId=186#/olir/home>. Consultato il 26 agosto 2026.

2025), che include traduzioni italiane e sottocategorie dedicate alla protezione dei dati personali.

Da un punto di vista semantico, sia la triade CIA che le funzioni CSF 2.0 possono essere considerate degli obiettivi da raggiungere ortogonali tra loro, in quanto ogni funzione può contribuire a garantire una o più tra confidentiality, integrity e availability.

In sez. 2.1.1, *D3FEND e il CSF 2.0* vedremo quali raccordi sono possibili tra la triade CIA, le funzioni CSF 2.0 e le `d3f:DefensiveTactic` di D3FEND.

2.1.1 D3FEND e il CSF 2.0

Né la triade CIA, né le funzioni CSF 2.0 hanno una controparte esplicita in D3FEND. Essendo degli obiettivi da raggiungere, possono essere considerati dei `d3f:Goal`.

Saltano all’occhio invece delle affinità lessicali con le `d3f:DefensiveTactic` (anch’esse `rdfs:subClassOf d3f:Goal`), per quanto anche qui le Funzioni CSF sono per definizione tecnologicamente neutrali, mentre le `d3f:DefensiveTactic` hanno dei riferimenti tecnologici specifici.

<code>d3f:DefensiveTactic</code>	Funzione CSF 2.0
<code>d3f:Model</code>	Identify
<code>d3f:Harden</code>	Protect
<code>d3f:Isolate</code>	Protect
<code>d3f:Detect</code>	Detect
<code>d3f:Deceive</code>	Detect, Respond
<code>d3f:Evict</code>	Respond
<code>d3f:Restore</code>	Recover

Il sito del MITRE fornisce invece una [mappatura esplicita](https://d3fend.mitre.org/mappings/nist/5/)⁶ tra alcune `d3f:DefensiveTechnique` e dei “Security and Privacy Controls” (National Institute of Standards and Technology 2020). Questa mappatura non fa però riferimento alle Funzioni CSF 2.0, che non compaiono nel dataset D3FEND: il raccordo con le Funzioni resta quindi quello, puramente lessicale, con le `d3f:DefensiveTactic` riportato sopra. La mappatura è discussa nella In sez. 2.2.1, *D3FEND e i controlli NIST SP 800-53*.

`d3f:DefensiveTactic`
`rdfs:subClassOf d3f:Goal ;`

⁶<https://d3fend.mitre.org/mappings/nist/5/>. Consultato il 26 agosto 2026.

```

    d3f:enabled-by d3f:DefensiveTechnique
.
d3f:Harden rdfs:subClassOf d3f:DefensiveTactic
.
d3f:DefensiveTechnique
  rdfs:seeAlso <https://csrc.nist.gov/glossary/term/security_control> ;
  d3f:enables d3f:DefensiveTactic
.

```

In sez. 4, *Progettazione sicura con grafi architetturali basati su D3FEND* le tattiche e le tecniche difensive vengono applicate ai grafi architetturali; il raccordo con i vocabolari giuridici è descritto invece in sez. 2.5.6, *Relazioni con D3FEND*.

2.2 MITRE D3FEND

Il MITRE è un'organizzazione no-profit statunitense, fondata nel 1958 come spin-off del MIT Lincoln Laboratory su iniziativa dell'U.S. Air Force⁷ per collegare accademia e industria nell'ambito della difesa aerea nazionale. Oggi fornisce consulenza e supporto a enti governativi e privati nell'ambito dell'ingegneria dei sistemi⁸.

Grazie alla sponsorizzazione della National Security Agency (NSA), il MITRE ha sviluppato D3FEND e ATT&CK, due ontologie per la cybersecurity.

D3FEND è un'ontologia che ha l'obiettivo di definire una terminologia cyber indipendente dai vendor e dalle tendenze del momento, e classificare:

- gli oggetti fisici e portatori di informazione;
- le tecniche di difesa e di attacco.

D3FEND si basa su tre classi di oggetti principali, tutte sottoclassi a livelli diversi di `d3f:D3FENDCore`.

ATT&CK invece è un knowledge base di tecniche di attacco basata su osservazioni reali, e fornisce un framework per la classificazione delle minacce informatiche (MITRE Corporation s.d.e) nonché per lo sviluppo di threat model specifici.

Il sito di D3FEND (MITRE Corporation 2021) pubblica un KG in formato RDF contenente sia le informazioni di D3FEND, sia quelle di ATT&CK.

⁷<https://www.mitre.org/who-we-are/our-story>. Consultato il 26 agosto 2026.

⁸<https://www.mitre.org/who-we-are>. Consultato il 26 agosto 2026.

Vediamo le classi principali, riportandone le definizioni originali:

- `d3f:DigitalArtifact`: “An information-bearing artifact (object) that is, or is encoded to be used with, a digital computer system. This concept is broad to include the literal instances of an artifact, or an implicit summarization of changes to or properties of other artifacts.”; sinonimi: Digital Asset;
- `d3f:OffensiveTechnique`: $\sqsubseteq$ `d3f:CyberTechnique`, priva di definizione nell’ontologia; nel seguito la intendiamo come una tecnica offensiva, che modifica in modo improprio un `d3f:DigitalArtifact`;
- `d3f:DefensiveTechnique`: $\sqsubseteq$ `d3f:CyberTechnique`, “A method which makes a computer system more difficult to attack.”; sinonimi: Countermeasure Technique, Technical Security Control, Defensive Capability Feature.

e sulle loro relazioni:

- `d3f:d3fend-object-property`: proprietà radice delle relazioni dell’ontologia, senza una definizione; nel seguito la intendiamo come una relazione tra due oggetti digitali;
- `d3f:accesses`: “x accesses y: The subject x takes the action of reading from, writing into, or executing the stored information in the object y. Reads, writes, and executes are specific cases of accesses.”;
- `d3f:modifies`: “x modifies y: A technique or agent x causes a digital object y to change; become different; or undertake a transformation. Afterwards, the data or state held by a digital object is changed.”;
- `d3f:verifies`: “x verifies y: The technique x confirms the truth of a digital artifact y.”.

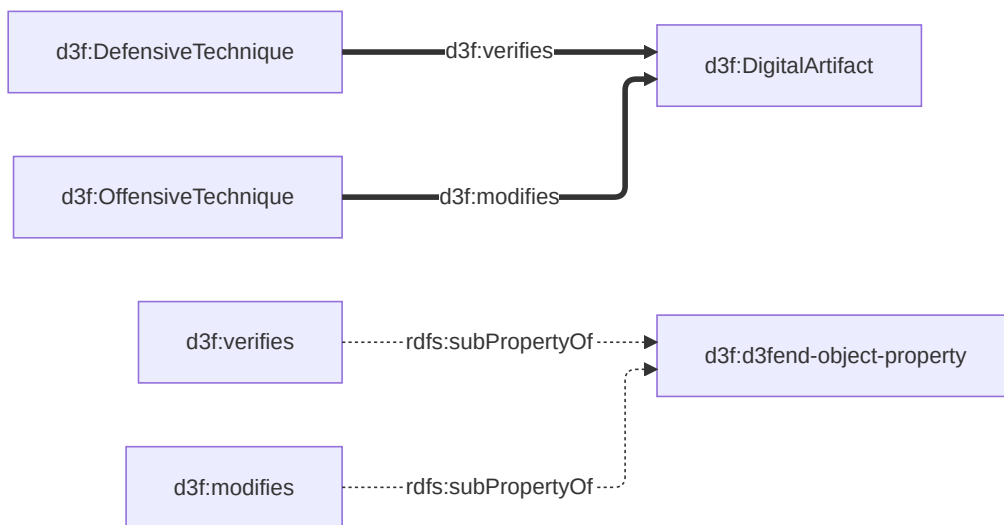


Figura 10: Struttura orientativa di D3FEND

D3FEND è un progetto in evoluzione: ad ogni versione si è arricchito di nuove classi e relazioni, integrando anche entità non digitali, quali `d3f:Agent`, `d3f:Person` e `d3f:PhysicalLocation`.

La versione 1.4 del dataset contiene:

DigitalArtifact	DefensiveTechnique	OffensiveTechnique
896	272	849

2.2.1 D3FEND e i controlli NIST SP 800-53

D3FEND modella anche i controlli secondo la struttura sintetizzata nel diagramma di seguito, ed include i controlli definiti in (National Institute of Standards and Technology 2020), a cui è associata la classe `d3f:NISTControl` $\sqsubseteq$ `d3f:ExternalControl`, raccolti nel catalogo `d3f:NIST_SP_800-53_R5` tramite la proprietà `d3f:member-of`.

Il collegamento con le tecniche difensive è espresso da sottoproprietà di `d3f:semantic-relation` (`d3f:broader`, `d3f:narrower`, `d3f:related`, `d3f:exactly`).

Per brevità il diagramma visualizza:

- le sottoclassi tramite sottoinsiemi, e non tramite link;
- le restrizioni come link in grassetto.

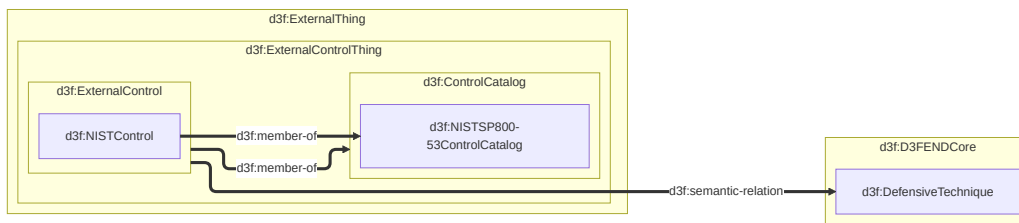


Figura 11: Modellazione dei controlli di sicurezza in D3FEND

Mostriamo un estratto da un controllo NIST SP 800-53, notando che esiste un ulteriore livello di indirizzione tra il documento NIST e il control catalog: questo è finalizzato a gestire ulteriori revisioni del documento:

```

===== NOTE: '\' line wrapping per RFC 8792 =====

d3f:NISTControl
  rdfs:subClassOf d3f:ExternalControl ;
  rdfs:subClassOf [ owl:onProperty d3f:member-of ;
                    owl:someValuesFrom d3f:NISTSP800-53ControlCatalog ]
.
d3f:NIST_SP_800-53_R5_AC-2_1 a d3f:NISTControl ;
  rdfs:label "AC-2(1)" ;
  d3f:control-name "Account Management | Automated System Account Manag\
ement" ;
  d3f:member-of d3f:NIST_SP_800-53_R5 ;
  d3f:broader d3f:AccountLocking, d3f:Multi-factorAuthentication
.

```

ed il corrispondente diagramma:

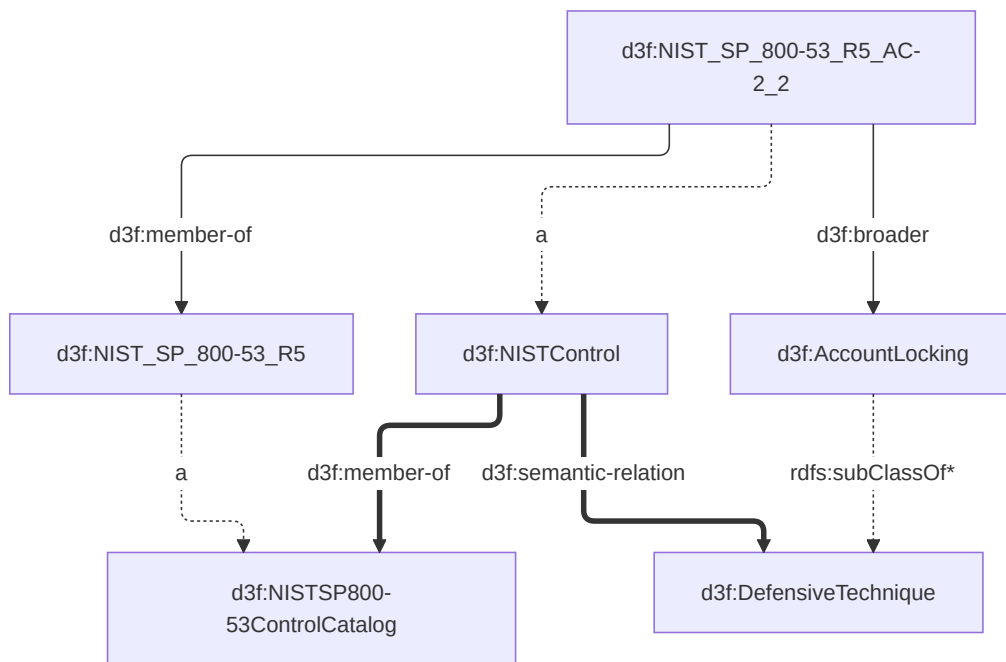


Figura 12: D3FEND e NIST SP 800-53

Riportiamo qui un estratto della mappatura.

control	controlName	relation	technique
AC-2(1)	Automated System Account Mgmt.	broader	Account Locking
AC-2(1)	Automated System Account Mgmt.	broader	Multi-factor Authentication
AC-2(4)	Automated Audit Actions	related	Domain Account Monitoring
AC-3	Access Enforcement	narrower	Executable Allowlisting
AC-3	Access Enforcement	narrower	Executable Denylisting
AC-4	Information Flow Enforcement	narrower	Inbound Traffic Filtering
AC-4	Information Flow Enforcement	narrower	Outbound Traffic Filtering
AC-6	Least Privilege	broader	Local File Permissions
AC-6	Least Privilege	broader	User Account Permissions
SI-3	Malicious Code Protection	broader	Network Traffic Analysis
SI-4	System Monitoring	broader	Operating System Monitoring

In sintesi, possiamo rappresentare così le relazioni tra D3FEND, Security Control e CSF 2.0.

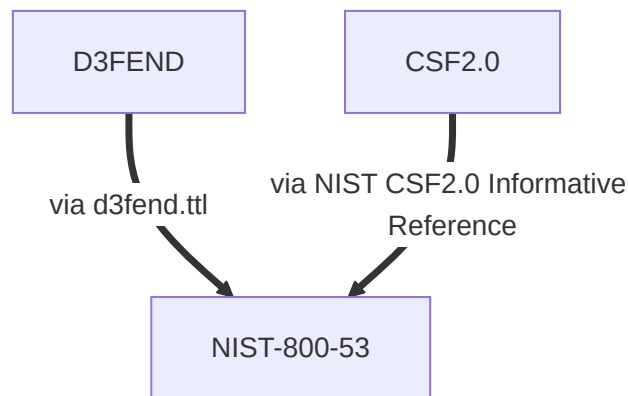


Figura 13: Relazioni tra D3FEND, Security Control e CSF 2.0

2.2.2 D3FEND ed altri dataset del MITRE

Il MITRE gestisce altri dataset, tra cui:

- CVE (Common Vulnerabilities and Exposures) (The CVE Program s.d.);
- CWE (Common Weakness Enumeration) (MITRE Corporation s.d.b);
- CAPEC (Common Attack Pattern Enumeration and Classification) (MITRE Corporation s.d.a);
- ATT&CK (Adversarial Tactics, Techniques, and Common Knowledge) (MITRE Corporation s.d.e).

Questi dataset sono integrati in D3FEND in misura diversa:

- le CWE sono modellate come `d3f:CWE-*` $\sqsubseteq$ `d3f:Weakness` $\sqsubseteq$ `d3f:d3f:D3FENDCore`, con `d3f:cwe-id` e `d3f:definition`;
- CAPEC è integrabile tramite un tool presente sul repository GitHub del MITRE (MITRE Corporation 2024), che genera un file RDF contenente le classi `d3f:CAPEC-*` $\sqsubseteq$ `d3f:CommonAttackPattern` $\sqsubseteq$ `d3f:CAPECThing` $\sqsubseteq$ `d3f:ExternalThreatModelThing` $\sqsubseteq$ `d3f:ExternalThing` collegate tramite `d3f:related` alle CWE e alle `d3f:OffensiveTechnique`.

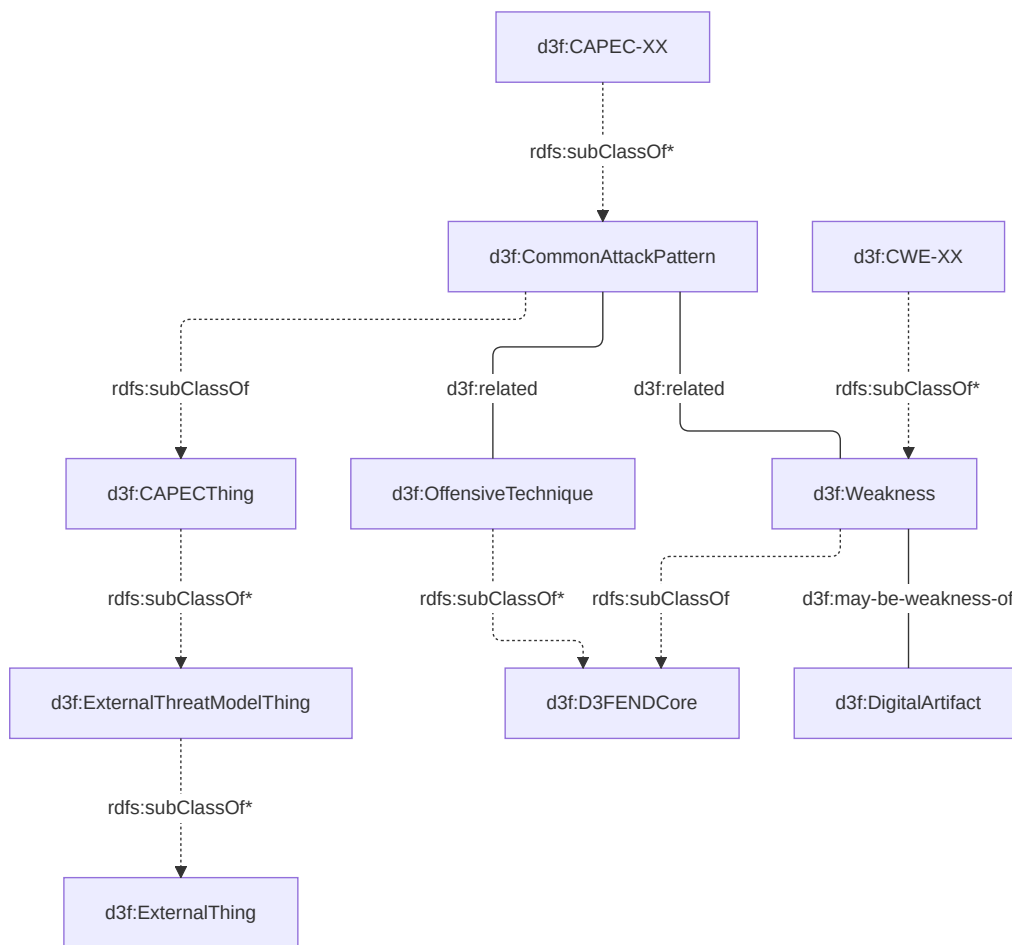


Figura 14: D3FEND e CWE

Il dataset D3FEND contiene anche delle estensioni collegate ad ATT&CK - ma che non sono oggetto di questo lavoro. Le riportiamo per completezza:

- SPARTA (Space Attack Research and Tactic Analysis) (The Aerospace Corporation s.d.), associati a `d3f:SPARTAThing`;
- ATLAS (Adversarial Threat Landscape for Artificial-Intelligence Systems) (MITRE Corporation s.d.d), associati a `d3f:ATLASThing`;
- ATT&CK Mobile, associati a `d3f:ATTACKMobileThing`;
- ATT&CK ICS (Industrial Control Systems) associati a `d3f:ATTACKICSThing`.

2.2.3 Proprietà

Ricordiamo che `d3f:d3fend-object-property` è la radice delle relazioni definite da D3FEND.

Sebbene sia laborioso, è fondamentale per la comprensione dei flussi informativi e per l'individuazione delle tecniche offensive applicabili utilizzare le proprietà più specifiche possibili (si veda pure (MITRE Corporation 2026)).

Di seguito alcune delle proprietà principali, con la loro alberatura. In alcuni casi, vengono definite anche:

- delle proprietà inverse (e.g., `d3f:accesses` e `d3f:accessed-by`);
- delle generalizzazioni possibilistiche (e.g., `d3f:may-access`, `d3f:may-modify`, `d3f:may-create`, `d3f:may-produce`).

Alcune proprietà sono:

- transitive (e.g., `d3f:contains/d3f:contained-by`, `d3f:process-ancestor`).
- simmetriche (e.g., `d3f:communicates-with`).

altre invece non hanno proprietà particolari (e.g., `d3f:reads`).

In particolare, la transitività è importante per poter rappresentare in modo consistente oggetti nidificati, e.g.:

```
d3f:DatabaseServer d3f:contains d3f:DatabaseApplication .  
d3f:FileSystem d3f:contains d3f:Directory .
```

Ecco qui un grafo che rappresenta le proprietà principali.

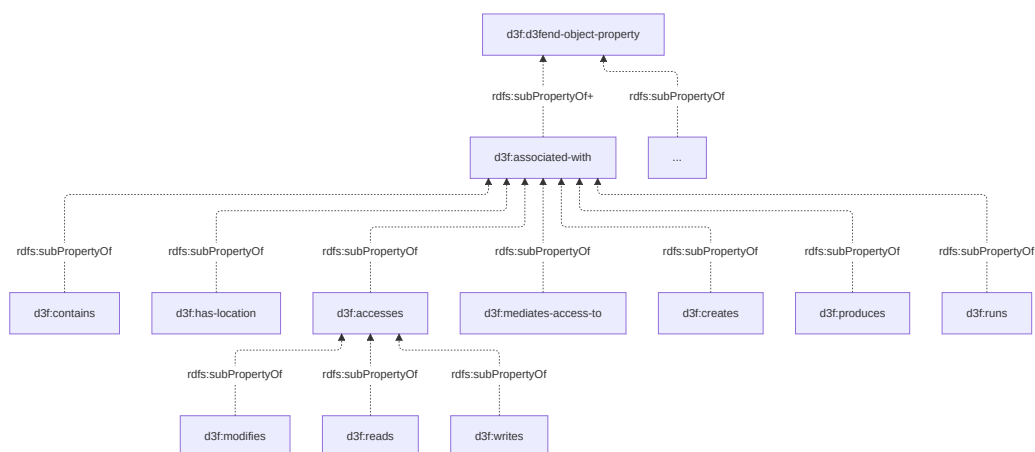


Figura 15: Proprietà principali di D3FEND e relative sottoproprietà

Proprietà Tattiche Le relazioni tattiche sono $\sqsubseteq$ `d3f:d3fend-tactical-verb-property` e associano principalmente - ma non solo - `d3f:CyberTechnique` e `d3f:DigitalArtifact`.

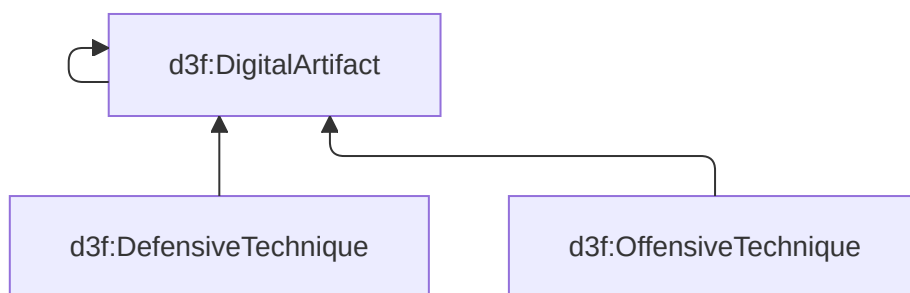


Figura 16: Relazioni tattiche in D3FEND

Dominio	Codominio	Frequenza
d3f:DefensiveTechnique	d3f:DigitalArtifact	"200"
d3f:DigitalArtifact	d3f:DigitalArtifact	"30"
d3f:OffensiveTechnique	d3f:DigitalArtifact	"11"
d3f:DefensiveTechnique	d3f:Action	"7"
d3f:DefensiveTechnique	d3f:Agent	"3"
d3f:DigitalArtifact	d3f:Action	"2"
d3f:DefensiveTechnique	d3f:Artifact	"2"
d3f:SPARTATechnique	d3f:Artifact	"1"
d3f:Action	d3f:Agent	"1"
d3f:DefensiveTechnique	d3f:OffensiveTechnique	"1"
d3f:SPARTATechnique	d3f:DigitalArtifact	"1"
d3f:Action	d3f:Action	"1"

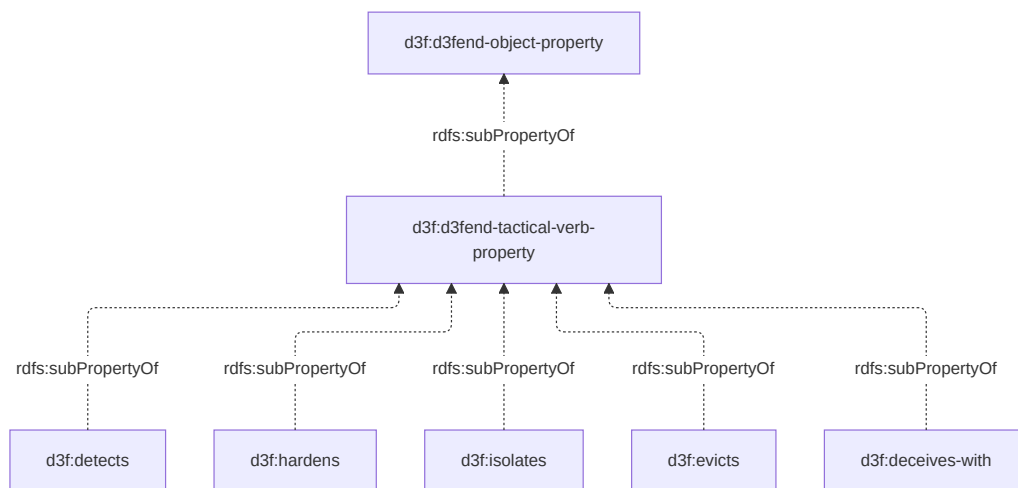


Figura 17: Proprietà tattiche in D3FEND

2.2.4 Classi

Rappresentiamo alcune delle classi principali, tutte sottoclassi di `d3f:D3FENDCore`:

- `d3f:Agent`
- `d3f:Artifact`
- `d3f:Plan`
- `d3f:Goal`
- `d3f:PhysicalLocation`

e le loro sottoclassi nel seguente diagramma.

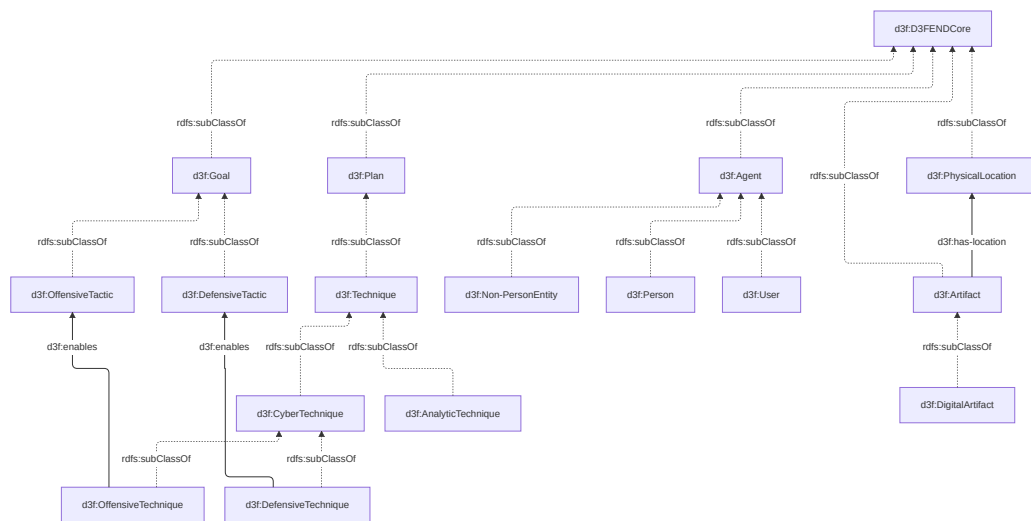


Figura 18: Classi principali di D3FEND e relative sottoclassi

Attualmente (versione 1.4) gli assiomi di disgiunzione tra le classi sono pochi, ma vanno ricordati quelli relativi ai concetti principali. Le future versioni potrebbero introdurre ulteriori vincoli.

```

[] a owl:AllDisjointClasses ;
  owl:members (
    d3f:Agent
    d3f:Artifact
    d3f:Event
    d3f:Goal
    d3f:Plan
    d3f:Weakness ) .

```

Poiché `rdfs:subClassOf` è transitiva, e le classi `d3f:Agent`, `d3f:Artifact`, `d3f:Plan`, `d3f:Goal` sono a due a due disgiunte, possiamo rappresentare le relazioni tra le classi principali e le loro sottoclassi utilizzando dei sotto-grafi, e generalizzando le restrizioni (in grassetto) tra le classi.

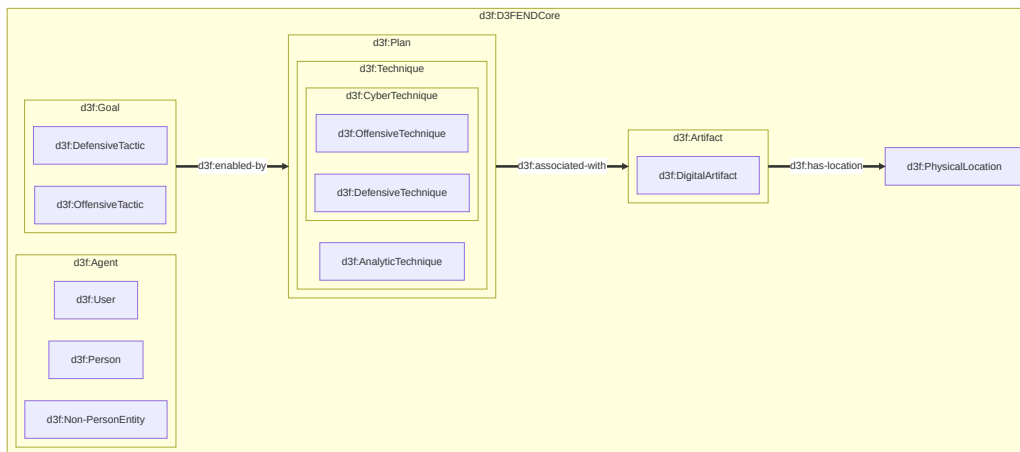


Figura 19: Classi principali di D3FEND rappresentate come sotto-diagrammi

La classe `d3f:AnalyticTechnique` rappresenta una tecnica di analisi dei dati. Non essendo una sottoclasse di `d3f:CyberTechnique`, le `d3f:AnalyticTechnique` non verranno considerate come tecniche difensive in questo documento, nonostante ad oggi l'identificativo viene associato tramite la property `d3f:d3fend-id`.

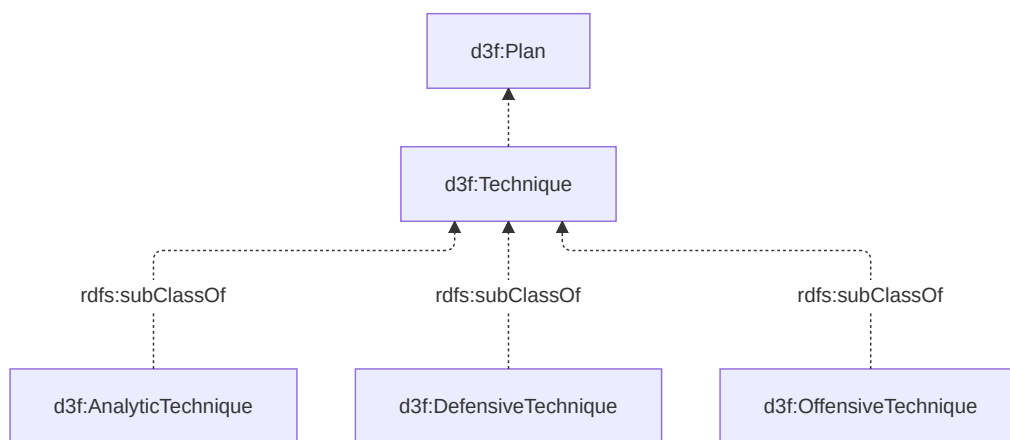


Figura 20: Sottoclassi di `d3f:AnalyticTechnique`

Di seguito, l'estratto di una `d3f:AnalyticTechnique`:

```
===== NOTE: '\' line wrapping per RFC 8792 =====
d3f:BayesianEstimation a owl:Class,
    owl:NamedIndividual ;
    rdfs:label "Bayesian Estimation" ;
    rdfs:subClassOf d3f:BayesianMethod ;
    d3f:d3fend-id "D3A-BE" ;
    d3f:definition "A Bayes estimator or a Bayes action is an estimator\
or decision rule that minimizes the posterior expected value of a loss\
```

```
function (i.e., the posterior expected loss)." ;
    d3f:kb-article """## References
Wikipedia. (n.d.). Bayes estimator. [Link] (https://en.wikipedia.org/wiki/Bayes_estimator)""" .
```

Artifact La classe `d3f:Artifact` rappresenta un oggetto nella sua interezza. Qui alcune sue sottoclassi principali.

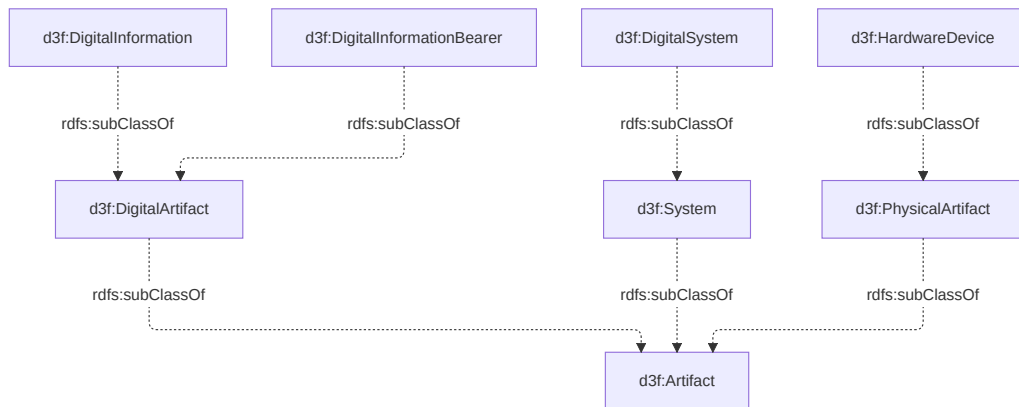


Figura 21: Gerarchia della classe `d3f:Artifact`

Questo pone delle complessità nella rappresentazione dell'interazione con gli utenti e l'associazione tra `d3f:Artifact` e `d3f:PhysicalLocation`.

Premesso che i vincoli (`owl:Restriction`) di D3FEND sono laschi, e che esistono pochi vincoli di disgiunzione, vediamo che la rappresentazione di un utente che accede ad un sistema digitale può essere abbastanza complessa se si va nei dettagli.

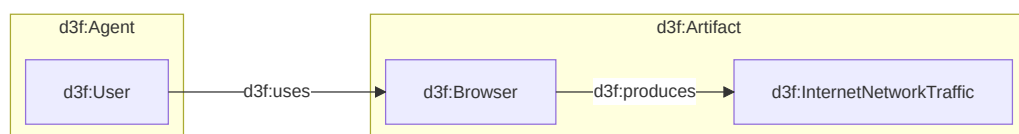


Figura 22: Un utente che accede a un sistema digitale

Vediamo qui alcune delle sottoclassi “concrete” di `d3f:DigitalInformation`:

- `d3f:Command`
- `d3f:DigitalMedia`
- `d3f:Software`

e di `d3f:DigitalInformationBearer`:

- `d3f:ComputerPlatform`
- `d3f:DigitalMessage`
- `d3f:SoftwarePackage`
- `d3f:Network`
- `d3f:UserAccount`
- `d3f:Storage`

Intorno di una classe Sul sito di D3FEND, la pagina dedicata ai `d3f:DigitalArtifact` include una sezione “neighbor” con i nodi dell’ontologia collegati alla classe. Definiamo qui il concetto di intorno di una classe come l’insieme contenente:

- sia la classe stessa;
- sia tutte le classi ad essa collegate tramite una relazione D3FEND e che condividono un antenato comune (e.g., `d3f:Artifact`).

Ossia

$$\mathcal{N}(C) = \{C\} \cup \{X \mid \exists R \sqsubseteq_1 \text{D3FENDCore} : C \sqsubseteq R \wedge X \sqsubseteq R \\ \wedge \exists r \sqsubseteq \text{d3fend-object-property} : C \sqsubseteq \exists r.X \vee X \sqsubseteq \exists r.C\}$$

A differenza della definizione sul sito di D3FEND, quella qui proposta è più restrittiva, ed esclude le classi collegate appartenenti ad altri sotto-rami dell’ontologia (e.g., `d3f:Event` per `d3f:DigitalArtifact`).

Per `d3f:Database` la query SPARQL è:

```
SELECT DISTINCT ?neighbour WHERE {  
  # la classe stessa appartiene al proprio intorno  
  { VALUES ?neighbour { d3f:Database } }  
  UNION  
  {  
    VALUES ?class { d3f:Database }  
  
    # .. esiste un antenato core comune (e.g., d3f:Artifact)
```

```

?root rdfs:subClassOf d3f:D3FENDCore .
?class rdfs:subClassOf* ?root .
?neighbour rdfs:subClassOf* ?root .

{?class ?rel ?neighbour .}
UNION
{?neighbour ?rel ?class .}

# .. ad essa collegare tramite una relazione D3FEND.
?rel rdfs:subPropertyOf* d3f:d3fend-object-property .
}
}

```

Mostrando il risultato della query per la classe `d3f:Database`, troviamo:

```

d3f:Database d3f:may-contain d3f:StoredProcedure .
d3f:Database d3f:contains d3f:DatabaseRecord .

d3f:DatabaseService d3f:manages d3f:Database .
d3f:EmbeddedDatabaseApplication d3f:manages d3f:Database .
d3f:DatabaseFile d3f:contains d3f:Database .
d3f:DatabaseQuery d3f:queries d3f:Database .

```

Questo concetto di “intorno” aiuta a navigare ed esplorare D3FEND.

2.2.5 Classi e Proprietà Tattiche

Questa sezione approfondisce le relazioni tra classi e proprietà tattiche introdotte in sez. 2.2.3, *Proprietà Tattiche*.

Prendiamo un estratto di `d3f:AccessMediation` $\sqsubseteq$ `d3f:DefensiveTechnique`, e vediamo le sue proprietà principali:

- `d3f:enables d3f:Isolate`: la tecnica di difesa consente di isolare un oggetto digitale;
- `d3f:d3fend-id "D3-AMED"`: l’identificativo della tecnica di difesa, utile per la ricerca nel grafo. In D3FEND ogni `d3f:DefensiveTechnique` ha un identificativo prefissato da D3-.

Per evitare di basare decisioni su termini ambigui o denominazioni commerciali con significati che variano nel tempo, le tecniche difensive riportano una ricca collezione di riferimenti bibliografici e link a risorse esterne.

===== NOTE: '\' line wrapping per RFC 8792 =====

```
d3f:AccessMediation
  rdfs:subClassOf d3f:DefensiveTechnique ;
  d3f:d3fend-id "D3-AMED" ;
  d3f:enables d3f:Isolate ;
  d3f:definition "Access mediation is the process of granting or denyin\
g specific requests to: 1) obtain and use information and related infor\
mation processing services; and 2) enter specific physical facilities .\
.." ;
  d3f:kb-reference d3f:Reference-CNNSI-4009 ;
  rdfs:seeAlso <http://dbpedia.org/resource/Access_control>,
    <https://ieeexplore.ieee.org/stamp/stamp.jsp?tp=&arnumber=1451869\
>,
    <https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.201-2.pdf>,
    <https://nvlpubs.nist.gov/nistpubs/SpecialPublications/NIST.SP.80\
0-160v1r1.pdf>,
    <https://www.cs.cmu.edu/afs/cs/usr/bsy/security/CSC-STD-001-83.tx\
t> .
```

D3FEND censisce oltre 270 d3f:DefensiveTechnique:

```
SELECT
  (COUNT(DISTINCT ?s) AS ?c)
WHERE {
  ?s rdfs:subClassOf* d3f:DefensiveTechnique .
}
```

Analogamente, vediamo le proprietà principali di una d3f:OffensiveTechnique mostrando un estratto di d3f:T1110.001 come esempio:

- d3f:accesses d3f:Password: la tecnica offensiva accede ad un oggetto digitale di tipo d3f:Password;
- d3f:modifies d3f:AuthenticationLog: la tecnica offensiva modifica un oggetto digitale di tipo d3f:AuthenticationLog;
- d3f:produces d3f:Authentication: la tecnica offensiva produce un oggetto digitale di tipo d3f:Authentication;
- rdfs:subClassOf d3f:T1110: quindi
$$d3f:T1110.001 \sqsubseteq d3f:T1110 \sqsubseteq d3f:OffensiveTechnique;$$
- d3f:attack-id "T1110.001": l'identificativo della tecnica offensiva. Per escludere dalla ricerca i risultati contenuti nelle estensioni descritte in sez. 2.2.9,

Estensioni: SPARTA, ATLAS, ATT&CK Mobile e ATT&CK ICS è necessario filtrare per `rdfs:subClassOf* d3f:OffensiveTechnique` nelle query SPARQL. In D3FEND ogni sottoclasse concreta di `d3f:OffensiveTechnique` ha un identificativo prefissato da T.

```
===== NOTE: '\' line wrapping per RFC 8792 =====

d3f:T1110.001
  rdfs:label "Password Guessing" ;
  rdfs:subClassOf d3f:T1110;
  d3f:accesses d3f:Password ;
  d3f:attack-id "T1110.001" ;
  d3f:definition "Adversaries with no prior knowledge of legitimate cre\
dentials within the system or environment may guess passwords..." ;
  d3f:modifies d3f:AuthenticationLog ;
  d3f:produces d3f:Authentication
.
d3f:T1110
  rdfs:subClassOf d3f:CredentialAccessTechnique ;
  rdfs:label "Brute Force" ;
  d3f:attack-id "T1110"
.
```

Si noti che l'associazione tra `d3f:OffensiveTechnique` e `d3f:DigitalArtifact` non implica necessariamente un impatto su ogni possibile istanza di un dato `d3f:DigitalArtifact`, ma semplicemente una scelta di modellazione.

D3FEND censisce oltre 800 `d3f:OffensiveTechnique`:

```
SELECT
  (COUNT(DISTINCT ?s) AS ?c)
WHERE {
  ?s rdfs:subClassOf* d3f:OffensiveTechnique .
}
```

di cui 15 sono sottoclassi dirette di `d3f:OffensiveTechnique`.

```
SELECT ?a WHERE {
  ?a rdfs:subClassOf* d3f:OffensiveTechnique ;
  rdfs:label ?label .
  FILTER NOT EXISTS { ?a d3f:attack-id [] . }
}
```

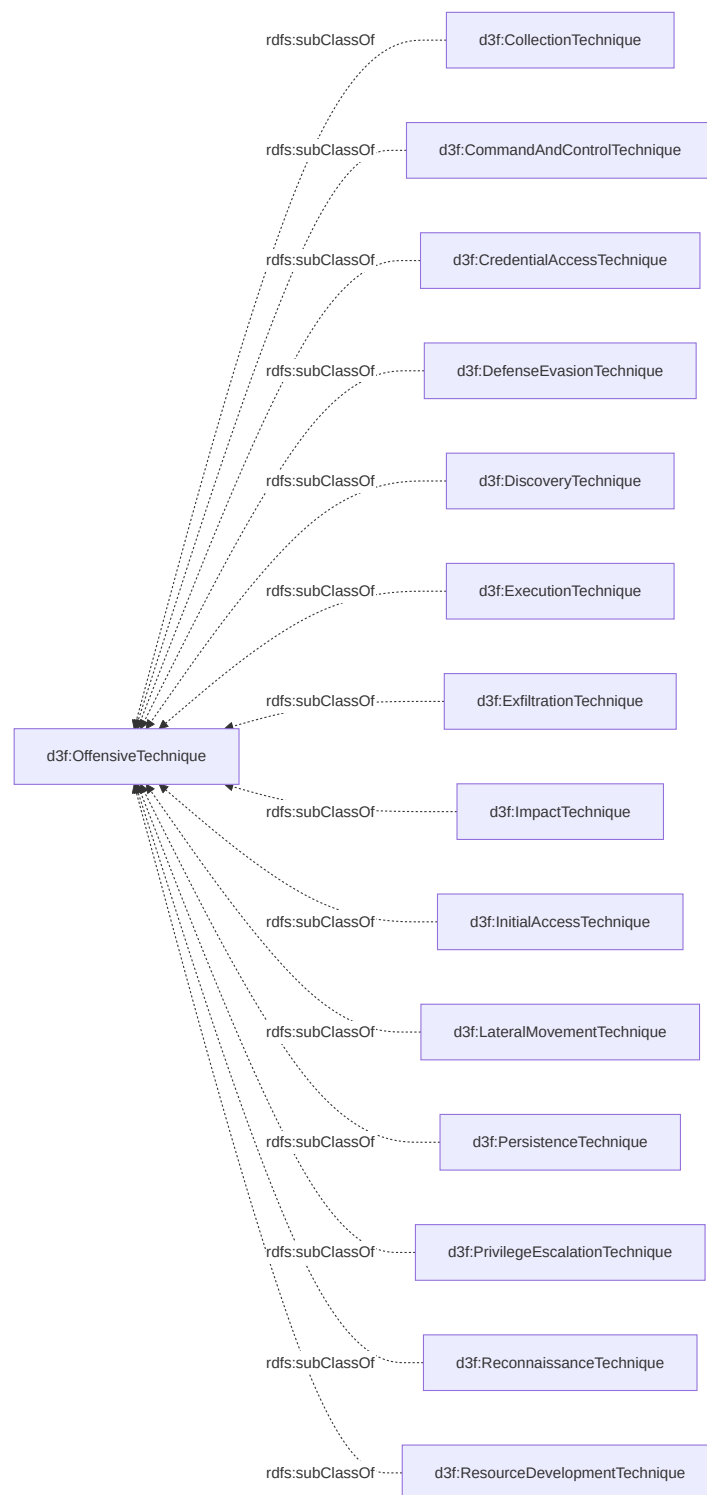


Figura 23: Sottoclassi dirette di d3f:OffensiveTechnique

2.2.6 Relazioni tra d3f:OffensiveTechnique e d3f:DigitalArtifact

Una delle domande ci siamo posti in sez. 1.2, *Obiettivi* è se sia possibile individuare le d3f:OffensiveTechnique applicabili a un'infrastruttura digitale descritta con D3FEND.

Prima di mostrare cosa viene visualizzato nella sezione “Related Offensive Techniques” di un d3f:DigitalArtifact sul sito di D3FEND, visualizziamo alcune relazioni di esempio.

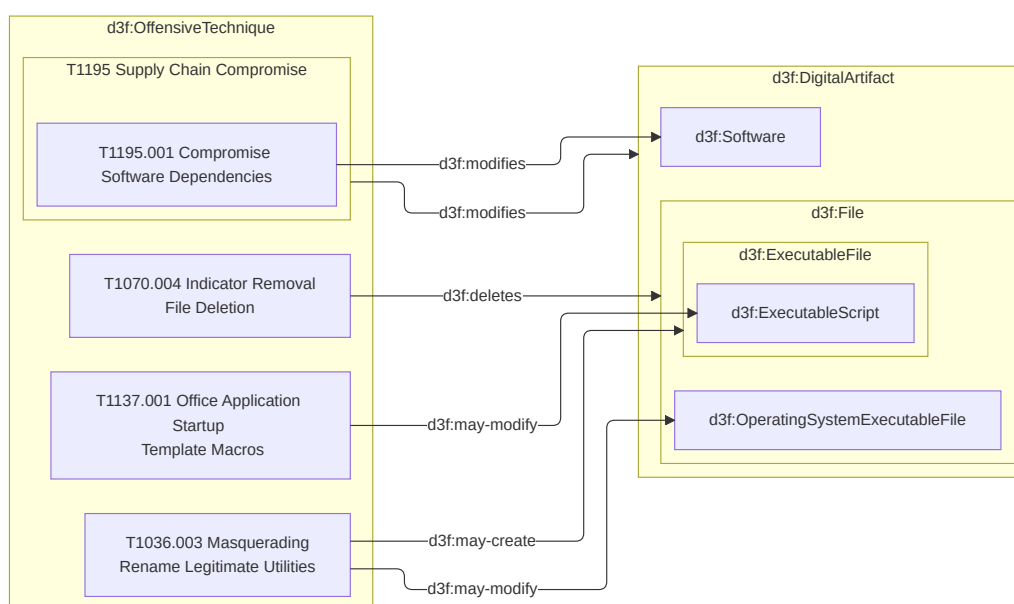


Figura 24: Relazioni di esempio tra d3f:OffensiveTechnique e d3f:DigitalArtifact

Notiamo - vedendo l'esempio d3f:T1195 - che una d3f:OffensiveTechnique può avere più relazioni con diversi rami di d3f:DigitalArtifact.

Per determinare l'insieme delle tecniche offensive applicabili ad un d3f:DigitalArtifact possiamo seguire diverse strade.

Una nota terminologica In questa stesura non usiamo il termine “superficie di attacco”, che il glossario del NIST (National Institute of Standards and Technology s.d.a) alla voce [attack surface](#) (National Institute of Standards and Technology 2020) definisce come:

The set of points on the boundary of a system, a system component, or an environment where an attacker can try to enter, cause an effect on, or extract data from.

Sebbene questa possa essere individuata a partire dal KG che rappresenta il sistema, in questo documento ci focalizziamo sull'individuazione delle `d3f:OffensiveTechnique` che insistono sui nodi - a prescindere dalla loro collocazione nell'architettura.

Useremo pertanto il termine **tecniche offensive applicabili**, l'insieme delle `d3f:OffensiveTechnique` applicabili ad $x \sqsubseteq \text{d3f:DigitalArtifact}$.

I limiti di questa costruzione sono discussi in sez. 2.2.6, *Limiti dell'analisi delle tecniche applicabili*.

Tecniche applicabili determinate dalle relazioni dirette La prima, che indichiamo con $T(x)$, consiste nel considerare tutte le `d3f:OffensiveTechnique` che hanno una relazione diretta con il `d3f:DigitalArtifact` scelto, senza attraversare la gerarchia delle classi. Limitandoci ai dati del nostro esempio, per `d3f:File` otteniamo:

```
T(d3f:File) = {  
  d3f:T1070.004, # d3f:deletes d3f:File  
}
```

Ignoriamo quindi sia il fatto che `d3f:File` $\sqsubseteq$ `d3f:DigitalArtifact`, che tutte le sottoclassi di `d3f:File`, come `d3f:ExecutableFile` e `d3f:OperatingSystemExecutableFile`.

Questa è una strada più conservativa, ma meno dispersiva, e si presta bene durante la fase di progettazione, in cui si vuole individuare il tipo di `d3f:DigitalArtifact` più adatto a modellare un oggetto digitale.

La query corrispondente è:

```
SELECT DISTINCT  
  ?label ?affects ?artifact  
WHERE {  
  BIND(d3f:File AS ?artifact)  
  # Prendi tutte le tecniche offensive...  
  ?attack rdfs:subClassOf* d3f:OffensiveTechnique .  
  ?attack rdfs:label ?label .
```

```

    # ... che impattano su ...
    ?attack ?affects ?artifact .
}
ORDER BY ?artifact

```

Tecniche applicabili determinate dall'unione sulle superclassi La seconda, che indichiamo con $T_{\sqsubseteq}(x) = T(Anc(x))$, consiste nel considerare tutte le `d3f:OffensiveTechnique` che hanno una relazione con $x \sqsubseteq d3f:File$. Limitandoci ai dati del nostro esempio, dato che `d3f:DigitalArtifact` $\sqsubseteq$ `d3f:File`, otteniamo:

```

Ta(d3f:File) = {
    d3f:T1070.004, # d3f:deletes d3f:File
    d3f:T1195      # d3f:modifies d3f:DigitalArtifact
}

```

In SPARQL:

```

SELECT DISTINCT
  ?label ?affects ?parent ?artifact
WHERE {
  BIND(d3f:File AS ?artifact)
  # Prendi tutte le tecniche offensive...
  ?attack rdfs:subClassOf* d3f:OffensiveTechnique .
  ?attack rdfs:label ?label .

  # ...
  ?attack ?affects ?parent .

  # ... che le relaziona con una superclasse di d3f:File.
  ?artifact rdfs:subClassOf* ?parent .
}
ORDER BY ?artifact

```

Usando la stessa logica, vediamo - omettendo i dettagli di tutte le 21 tecniche ritornate dalla query - che

```

T(d3f:ExecutableFile) = {
    d3f:T1036.003, # d3f:may-create d3f:ExecutableFile
    d3f:T1070.004, # d3f:deletes d3f:File
    d3f:T1195      # d3f:modifies d3f:DigitalArtifact
}

```

Questa strada ci porta quindi ad avere un insieme monotonicamente crescente, man mano che ci spostiamo verso le sottoclassi di `d3f:DigitalArtifact`: poiché `d3f:ExecutableFile` $\sqsubseteq$ `d3f:File`, $T_{\sqsubseteq}(d3f:ExecutableFile)$ includerà anche le tecniche offensive di `d3f:File` o `d3f:DigitalArtifact`.

Più in generale:

$$x \sqsubseteq y \Rightarrow T_{\sqsubseteq}(x) \supseteq T_{\sqsubseteq}(y)$$

Tecniche applicabili determinate dall'unione sulle sottoclassi Un'altra strada, che indichiamo con $T_{\sqsubseteq}(x) = T(Desc(x))$, consiste nel caratterizzare un `d3f:DigitalArtifact` come l'unione delle sue sottoclassi, in cui un attacco su una sottoclasse è considerato un possibile attacco sulla superclasse scelta. Questo approccio viene utilizzato quando si considerano gli attacchi verso una superclasse come troppo generici.

Limitandoci ai dati del nostro esempio, per trovare le `d3f:OffensiveTechnique` che impattano su `d3f:File`, inizieremo col domandarci quali sono tutte le possibili declinazioni - e quindi le sottoclassi - di `d3f:File`: `d3f:ExecutableFile`, `d3f:OperatingSystemExecutableFile`, `d3f:ExecutableScript`.

A questo punto, si considerano tutte le `d3f:OffensiveTechnique` che hanno una relazione con una di queste sottoclassi, ottenendo:

```
T(d3f:File) = {
  d3f:T1036.003, # d3f:may-create d3f:ExecutableFile
  d3f:T1070.004, # d3f:deletes d3f:File
  d3f:T1137.001, # d3f:may-modify d3f:ExecutableScript
}
```

La query SPARQL corrispondente è:

```
SELECT DISTINCT
  ?label ?affects ?child ?artifact
WHERE {
  BIND(d3f:File AS ?artifact)
  # Prendi tutte le tecniche offensive...
  ?attack rdfs:subClassOf* d3f:OffensiveTechnique .
  ?attack rdfs:label ?label .
  # ... che impattano su ...
  ?attack ?affects ?child .
  # ... una sottoclasse di d3f:File.
  ?child rdfs:subClassOf* ?artifact .
}
ORDER BY ?artifact
```

Si noti che la direzione di `rdfs:subClassOf*` è invertita rispetto alla query precedente, dove tanto più una classe è specifica, tanto più le tecniche offensive diminuiscono:

$$x \sqsubseteq y \Rightarrow T_{\sqsubseteq}(x) \subseteq T_{\sqsubseteq}(y)$$

Questo è l’approccio utilizzato nella sezione “Related Offensive Techniques” del sito di D3FEND per associare `d3f:DigitalArtifact` e `d3f:OffensiveTechnique`. Mostriamo come esempio la sezione “Related Offensive Techniques” di `d3f:Database`⁹ recuperabili anche tramite la query SPARQL:

```
SELECT DISTINCT
  ?label ?affects ?artifact
WHERE {
  ?attack d3f:attack-id ?id .
  ?attack rdfs:label ?label .
  ?attack ?affects ?artifact .
  ?artifact rdfs:subClassOf* d3f:Database .
}
ORDER BY ?artifact
```

di cui rappresentiamo un estratto come grafo.

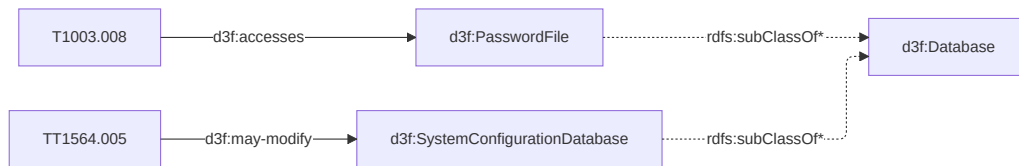


Figura 25: Tecniche offensive associate a `d3f:Database`

Anche in questo caso l’elenco può essere verboso, e a prima vista poco rilevante rispetto a quello che ci si aspetterebbe dal nome della classe `d3f:Database`.

label	affects	artifact
Code Repositories	d3f:reads	d3f:CodeRepository
Credentials from Web Browsers	d3f:may-access	d3f:In-memoryPasswordStore
Securityd Memory	d3f:accesses	d3f:In-memoryPasswordStore
Keychain	d3f:accesses	d3f:MacOSKeychain
System Owner/User Discovery	d3f:may-access	d3f:PasswordFile
/etc/passwd and /etc/shadow	d3f:accesses	d3f:PasswordFile
Credentials from Password Stores	d3f:accesses	d3f:PasswordStore
System Language Discovery	d3f:queries	d3f:SystemConfigurationDatabase
Hidden File System	d3f:may-modify	d3f:SystemConfigurationDatabase
Hidden Window	d3f:may-modify	d3f:SystemConfigurationDatabase
Credentials in Registry	d3f:accesses	d3f:SystemConfigurationDatabase
Elevated Execution with Prompt	d3f:creates	d3f:SystemConfigurationDatabase
Component Object Model Hijacking	d3f:modifies	d3f:SystemConfigurationDatabase
Image File Execution Options Injection	d3f:modifies	d3f:SystemConfigurationDatabase

⁹Vedi <https://d3fend.mitre.org/dao/artifact/d3f:Database/>. Consultato il 26 agosto 2026.

label	affects	artifact
Windows Service	d3f:modifies	d3f:SystemConfigurationDatabase
MMC	d3f:may-modify	d3f:SystemConfigurationDatabase
Rogue Domain Controller	d3f:modifies	d3f:SystemConfigurationDatabase
Add-ins	d3f:may-modify	d3f:SystemConfigurationDatabase
Query Registry	d3f:accesses	d3f:SystemConfigurationDatabase
LSA Secrets	d3f:may-access	d3f:SystemPasswordDatabase
Security Account Manager	d3f:may-access	d3f:SystemPasswordDatabase
Modify Registry	d3f:modifies	d3f:WindowsRegistry

Per D3FEND infatti il concetto di `d3f:Database` è generico ed include tutte le sotto-classi elencate in tabella, incluse `d3f:WindowsRegistry` e `d3f:MacOSKeychain`.

E' fondamentale quindi:

- individuare il corretto livello di specificità dei `d3f:DigitalArtifact`;
- valutare l'utilizzo di query maggiormente mirate.

Quando si utilizza $T_{\sqsubseteq}(x)$, bisogna tenere presente che questa non individua tecniche a largo raggio quali la `d3f:T1195` - "Supply Chain Compromise" poiché D3FEND le associa a `d3f:DigitalArtifact` generici.

Tecniche applicabili determinate dall'intorno Ricordiamo che l'Intorno di una classe sez. 2.2.4, *Intorno di una classe* è l'insieme delle classi appartenenti allo stesso concetto core (e.g., `d3f:Artifact` $\sqsubseteq_1$ `d3f:D3FENDCore`) che sono in relazione non gerarchica con essa.

Indichiamo con $T_I(x) = T(I(x))$ le `d3f:OffensiveTechnique` che hanno una relazione con una classe appartenente all'intorno di `d3f:Database`.

```
SELECT DISTINCT
  ?attack ?label ?attacks ?neighbour ?rel ?rel1 ?class
WHERE {
  BIND (d3f:Database AS ?class)

  ?class rdfs:subClassOf* ?root .
  ?root rdfs:subClassOf d3f:D3FENDCore .

  {
    ?class ?rel ?neighbour .
    ?rel rdfs:subPropertyOf* d3f:d3fend-object-property .
  }
  UNION
  {
    ?neighbour ?rel1 ?class .
    ?rel1 rdfs:subPropertyOf* d3f:d3fend-object-property .
  }
}
```

```

}
?neighbour rdfs:subClassOf* ?root .

?attack rdfs:subClassOf* d3f:OffensiveTechnique ;
  ?attacks ?neighbour ;
  rdfs:label ?label
.
}

```

troviamo un risultato più in linea con l’esperienza comune:

?attack	?label	?attacks	?neighbour	?rel	?rel1	?class
d3f:T1555.003	“Credentials from Web Browsers”	d3f:accesses	d3f:Database File		d3f:contains	d3f:Database
d3f:T1555	“Credentials from Password Stores”	d3f:may-access	d3f:Database File		d3f:contains	d3f:Database
d3f:T1190	“Exploit Public-Facing Application”	d3f:injects	d3f:Database Query		d3f:queries	d3f:Database
d3f:T1505.001	“SQL Stored Procedures”	d3f:creates	d3f:Stored Procedure	d3f:may- contain		d3f:Database

Ossia

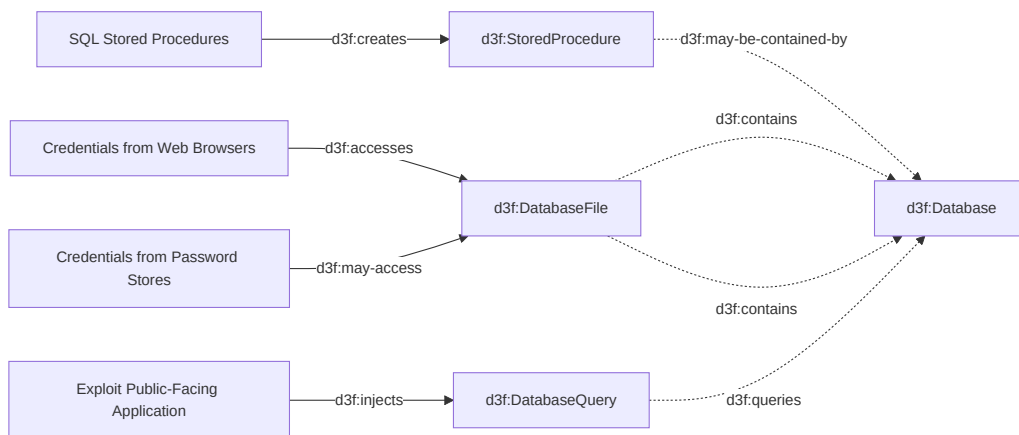


Figura 26: Tecniche offensive applicabili a d3f:Database tramite i suoi vicini

Questa vista è fondamentale per migliorare la modellazione di un’infrastruttura digitale, poiché ci permette di far emergere delle componenti trascurate in fase di progettazione.

Limiti dell’analisi delle tecniche applicabili

- La scelta tra i vari tipi di T dipende sia dall’evoluzione di D3FEND che dal contesto. Se possiamo dire che

$$T(x) \subseteq T_{\sqsubseteq}(x) \cap T_{\sqsupseteq}(x)$$

a seconda di x potresti avere che $T_{\sqsubseteq}(x)$ sia vuoto o che $T_{\sqsupseteq}(x)$ contenga solo il minimo - ossia $T(d3f:DigitalArtifact)$.

- Relazioni offensive e difensive tra classi e istanze: $T(x)$ è associato ad una classe $A \sqsubseteq d3f:DigitalArtifact$, mentre i nodi di un diagramma architetturale sono invece istanze. Due istanze dello stesso tipo hanno perciò la stessa $T(x)$, indipendentemente da come vengono dispiegate in architettura.
- Il fatto che una tecnica impatti su un `d3f:DigitalArtifact` non implica che lo faccia su tutte le sue istanze: impossibile fare questo tipo di analisi senza conoscere i dettagli del prodotto, della versione e delle vulnerabilità note. Questo tema può essere oggetto di ulteriori sviluppi, come discusso in 6.3.1.

2.2.7 Individuazione delle `d3f:DefensiveTechnique`

L'individuazione delle `d3f:DefensiveTechnique` e le query associate seguono il medesimo approccio indicato sopra.

Riportiamo per sintesi quindi esclusivamente le query SPARQL relative alle strategie per superclassi e sottoclassi.

Poiché una corretta modellazione passa per l'inclusione degli oggetti rilevanti relativi all'intorno, non includiamo qui le tecniche difensive relative all'intorno.

`d3f:DefensiveTechnique` dirette

```
SELECT DISTINCT
  ?label ?protects ?artifact
WHERE {
  BIND(d3f:File AS ?artifact)
  # Prendi tutte le tecniche difensive dirette...
  ?measure rdfs:subClassOf* d3f:DefensiveTechnique .
  ?measure rdfs:label ?label .
  # ... che proteggono ...
  ?measure ?protects ?artifact .
```

```
}
ORDER BY ?artifact
```

La query ritorna 11 risultati. Nelle tabelle di questa sezione omettiamo la colonna `?artifact`, costante per il BIND:

label	protects
File Analysis	d3f:analyzes
File Integrity Monitoring	d3f:analyzes
File Eviction	d3f:deletes
File Encryption	d3f:encrypts
Content Filtering	d3f:filters
Remote File Access Mediation	d3f:isolates
Content Modification	d3f:modifies
Content Quarantine	d3f:quarantines
Restore File	d3f:restores
Local File Permissions	d3f:restricts
Decoy File	d3f:spoofs

d3f:DefensiveTechnique per superclassi

```
SELECT DISTINCT
  ?label ?protects ?cursor ?artifact
WHERE {
  BIND(d3f:File AS ?artifact)
  ?measure rdfs:subClassOf* d3f:DefensiveTechnique .
  ?measure rdfs:label ?label .
  ?measure ?protects ?cursor .

  ?artifact rdfs:subClassOf* ?cursor .
}
ORDER BY ?artifact
```

Anche questa query ritorna 11 risultati, gli stessi della precedente: nessuna `d3f:DefensiveTechnique` dichiara una relazione verso le superclassi di `d3f:File`, per cui `?cursor` resta vincolato a `d3f:File`.

label	protects	cursor
File Analysis	d3f:analyzes	d3f:File
File Integrity Monitoring	d3f:analyzes	d3f:File
File Eviction	d3f:deletes	d3f:File
File Encryption	d3f:encrypts	d3f:File
Content Filtering	d3f:filters	d3f:File
Remote File Access Mediation	d3f:isolates	d3f:File
Content Modification	d3f:modifies	d3f:File
Content Quarantine	d3f:quarantines	d3f:File
Restore File	d3f:restores	d3f:File
Local File Permissions	d3f:restricts	d3f:File
Decoy File	d3f:spoofs	d3f:File

A differenza di quanto avviene per le `d3f:OffensiveTechnique`, dove la risalita sulla gerarchia aggiunge le tecniche legate a `d3f:Resource` e `d3f:DigitalArtifact`, qui la strategia per superclassi non arricchisce il risultato diretto.

d3f:DefensiveTechnique per sottoclassi Come per le `d3f:OffensiveTechnique`, il sito D3FEND pubblica l'elenco delle `d3f:DefensiveTechnique` per unione di sottoclassi, che ritorna un risultato molto numeroso.

```
SELECT DISTINCT
  ?label ?protects ?cursor ?artifact
WHERE {
  BIND(d3f:File AS ?artifact)
  ?measure rdfs:subClassOf* d3f:DefensiveTechnique .
  ?measure rdfs:label ?label .
  ?measure ?protects ?cursor .

  ?cursor rdfs:subClassOf* ?artifact .
}
ORDER BY ?artifact
```

La query ritorna 27 risultati: alle 11 tecniche dirette si aggiungono quelle relative a `d3f:Email` (6), `d3f:ExecutableFile` (4), `d3f:DocumentFile` (3), `d3f:CertificateFile`, `d3f:OperatingSystemFile` e `d3f:UserInitConfigurationFile` (1 ciascuna):

label	protects	cursor
Certificate Analysis	d3f:analyzes	d3f:CertificateFile
Dynamic Analysis	d3f:analyzes	d3f:DocumentFile
Emulated File Analysis	d3f:analyzes	d3f:DocumentFile
Data Inventory	d3f:inventories	d3f:DocumentFile
Homoglyph Detection	d3f:analyzes	d3f:Email
Sender MTA Reputation Analysis	d3f:analyzes	d3f:Email
Sender Reputation Analysis	d3f:analyzes	d3f:Email
Email Removal	d3f:deletes	d3f:Email
Email Filtering	d3f:filters	d3f:Email
Restore Email	d3f:restores	d3f:Email
Dynamic Analysis	d3f:analyzes	d3f:ExecutableFile
Emulated File Analysis	d3f:analyzes	d3f:ExecutableFile
Executable Allowlisting	d3f:blocks	d3f:ExecutableFile
Executable Denylisting	d3f:blocks	d3f:ExecutableFile
File Analysis	d3f:analyzes	d3f:File
File Integrity Monitoring	d3f:analyzes	d3f:File
File Eviction	d3f:deletes	d3f:File
File Encryption	d3f:encrypts	d3f:File
Content Filtering	d3f:filters	d3f:File
Remote File Access Mediation	d3f:isolates	d3f:File
Content Modification	d3f:modifies	d3f:File
Content Quarantine	d3f:quarantines	d3f:File
Restore File	d3f:restores	d3f:File
Local File Permissions	d3f:restricts	d3f:File
Decoy File	d3f:spoofs	d3f:File
System File Analysis	d3f:analyzes	d3f:OperatingSystemFile
User Session Init Config Analysis	d3f:analyzes	d3f:UserInitConfigurationFile

Si noti che Dynamic Analysis e Emulated File Analysis compaiono due volte, essendo legate sia a `d3f:DocumentFile` che a `d3f:ExecutableFile`.

2.2.8 Associazioni tra tecniche e oggetti digitali che le implementano

Alcuni `d3f:DigitalArtifact` sono associabili a tecniche difensive, quali:

- `d3f:CodeAnalyzer`, `d3f:TestExecutionTool`
- `d3f:NetworkTrafficAnalysisSoftware`
- `d3f:Firewall`, `d3f:SecurityToken`
- `d3f:IntrusionDetectionSystem`

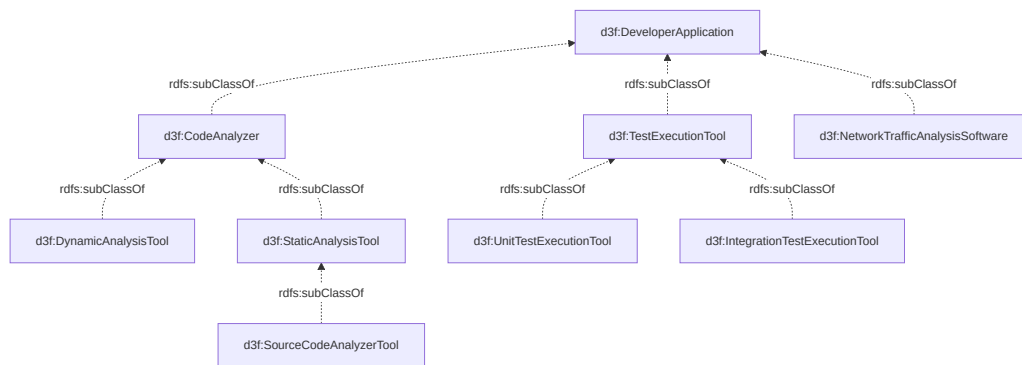


Figura 27: Oggetti digitali che implementano tecniche difensive

ma D3FEND non definisce una relazione specifica per associare un `d3f:DigitalArtifact` ad una `d3f:DefensiveTechnique`, quale ad esempio `d3f:d3fend-tactical-verb-property`, poiché `d3f:enables` associa invece `d3f:DefensiveTechnique` e `d3f:Goal`.

2.2.9 Estensioni: SPARTA, ATLAS, ATT&CK Mobile e ATT&CK ICS

Il dataset D3FEND contiene anche delle estensioni, quali:

- SPARTA (Space Attack Research and Tactic Analysis) (The Aerospace Corporation s.d.), associati a `d3f:SPARTAThing`;
- ATLAS (Adversarial Threat Landscape for Artificial-Intelligence Systems) (MITRE Corporation s.d.d), associati a `d3f:ATLASThing`;

- ATT&CK Mobile, associati a `d3f:ATTACKMobileThing`;
- ATT&CK ICS (Industrial Control Systems) associati a `d3f:ATTACKICSThing`.

Descriviamo sinteticamente queste estensioni, modellate come sottoclassi di `d3f:ExternalThing`, come illustrato nel diagramma seguente:

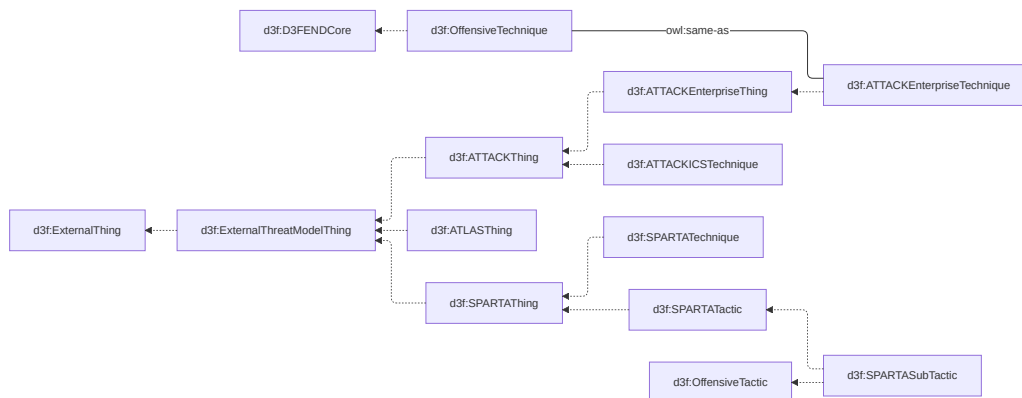


Figura 28: Modelli di minaccia esterni come sottoclassi di `d3f:ExternalThing`

Sebbene non vengano trattate in questo documento, notiamo che il pattern di modellazione segue quello delle `d3f:OffensiveTechnique` come si evince dalla seguente query SPARQL:

```

SELECT ?root ?a ?label WHERE {
  VALUES ?root {
    d3f:SPARTATechnique
    d3f:ATLASTechnique
    d3f:ATTACKMobileTechnique
    d3f:ATTACKICSTechnique
  }

  ?a rdfs:subClassOf* ?root ;
  rdfs:label ?label .
  FILTER NOT EXISTS { ?a d3f:attack-id [] . }
}
ORDER BY ?root ?label

```

che rappresentiamo sotto forma di tabella, raggruppando le tecniche per estensione e per tattica.

Notiamo inoltre che tutte le `d3f:OffensiveTechnique` sono anche `rdfs:subClassOf` di `d3f:ATTACKEnterpriseTechnique`, che identifica la collezione di tecniche offensive presenti in D3FEND.

La colonna Enterprise riporta le sottoclassi dirette di `d3f:OffensiveTechnique`, a titolo di riferimento.

Tattica	Enterprise	ATLAS	ICS	Mobile	SPARTA
AIAttackStaging		yes			
AIModelAccess		yes			
Collection	yes	yes	yes	yes	
CommandAndControl	yes	yes	yes	yes	
CredentialAccess	yes	yes		yes	
DefenseEvasion	yes	yes		yes	yes
Discovery	yes	yes	yes	yes	
Evasion			yes		
Execution	yes	yes	yes	yes	yes
Exfiltration	yes	yes		yes	yes
Impact	yes	yes	yes	yes	yes
ImpairProcessControl			yes		
InhibitResponseFunction			yes		
InitialAccess	yes	yes	yes	yes	yes
LateralMovement	yes	yes	yes	yes	yes
Persistence	yes	yes	yes	yes	yes
PrivilegeEscalation	yes	yes	yes	yes	
Reconnaissance	yes	yes			yes
ResourceDevelopment	yes	yes			yes

Nel complesso, il dataset D3FEND contiene quindi le seguenti quantità di tecniche offensive per framework:

Framework	Astratte	Concrete	Totale
ATT&CK Enterprise	15	835	850
SPARTA	10	216	226
ATT&CK Mobile	13	166	179
ATLAS	17	140	157
ATT&CK ICS	13	95	108

2.3 OWASP TOP 10

OWASP (Open Worldwide Application Security Project) è una *nonprofit foundation* che fornisce risorse, strumenti e standard gratuiti e open-source per la sicurezza informatica.

OWASP pubblica due documenti di riferimento che elencano i principali rischi di sicurezza per le applicazioni web e le API:

- [OWASP Top 10 Web Application Security Risks] (OWASP Foundation 2025) (aggiornato nel 2025), che elenca i principali rischi di sicurezza per le applicazioni web;

- [OWASP API Security Top 10] (OWASP Foundation 2023) (aggiornato nel 2023), con un focus sugli ecosistemi API.

Ogni *rischio OWASP* è associato ad una o più CWE - oggetto già presente su D3FEND come `d3f:Weakness`. Incrociando questa informazione col dataset CAPEC (MITRE Corporation s.d.a), possiamo quindi associare ad ogni *rischio OWASP* una o più `d3f:OffensiveTechnique` e i relativi `d3f:DigitalArtifact` impattati.

Per questo lavoro abbiamo mappato in RDF i rischi OWASP Top 10 API 2023 e le CWE nel file [data/owasp-cwe.ttl](#).

SPARQL ci permette di creare quindi una tabella con queste informazioni:

```
===== NOTE: '\' line wrapping per RFC 8792 =====

prefix top10: <https://owasp.org/Top10/2025/>
prefix top10_api: <https://owasp.org/API-Security/editions/2023/en/>

SELECT
  ?risk
  ?owasp
  (count(distinct ?cwe) as ?ncwe)
  (count(distinct ?t) as ?attacks)
  (count(distinct ?a) as ?artifacts)
WHERE {
  VALUES ?risk { top10:Risk top10_api:Risk}

  ?owasp a ?risk .

  # Includi anche i rischi OWASP che non hanno CWE associate
  OPTIONAL {
    ?owasp d3f:associated-with ?cwe .

    # Un artefatto può essere associato direttamente ad una CWE
    # tramite d3f:weakness-of o d3f:may-be-weakness-of.
    OPTIONAL {
      ?cwe (d3f:weakness-of|d3f:may-be-weakness-of) ?a .
      ?a rdfs:subClassOf* d3f:DigitalArtifact .
    }

    # Oppure e' necessario passare per CAPEC e le tecniche offensive as\
sociate.
    OPTIONAL {
      ?capec a d3f:CommonAttackPattern ;
        d3f:related ?t ;
        d3f:related ?cwe .

      ?t rdfs:subClassOf* d3f:OffensiveTechnique .

      # .. che impattano su un d3f:DigitalArtifact.
      # Il predicato è una variabile: ?verb, non ?attacks, che è gi\
à
      # il nome di una proiezione.
      OPTIONAL {
        ?t ?affects ?a .
        ?a rdfs:subClassOf* d3f:DigitalArtifact .
      }
    }
  }
}
```

```

    }
  }
}
GROUP BY ?risk ?owasp
ORDER BY ?risk ?owasp

```

Documento	Rischio	CWE	Attacchi	Artefatti
OWASP Top 10:2025	A01:2025 - Broken Access Control	40	80	71
OWASP Top 10:2025	A02:2025 - Security Misconfiguration	16	11	14
OWASP Top 10:2025	A03:2025 - Software Supply Chain Failures	7	0	0
OWASP Top 10:2025	A04:2025 - Cryptographic Failures	32	8	5
OWASP Top 10:2025	A05:2025 - Injection	37	12	13
OWASP Top 10:2025	A06:2025 - Insecure Design	39	43	33
OWASP Top 10:2025	A07:2025 - Authentication Failures	36	38	31
OWASP Top 10:2025	A08:2025 - Software or Data Integrity Failures	14	30	21
OWASP Top 10:2025	A09:2025 - Security Logging and Alerting Failures	5	4	7
OWASP Top 10:2025	A10:2025 - Mishandling of Exceptional Conditions	24	0	1
OWASP API Security Top 10 Risk	A01: Broken Object Level Authorization	2	12	15
OWASP API Security Top 10 Risk	A02: Broken Authentication	2	7	7
OWASP API Security Top 10 Risk	A03: Broken Object Property Level Authorization	2	0	0
OWASP API Security Top 10 Risk	A04: Unrestricted Resource Consumption	3	6	1
OWASP API Security Top 10 Risk	A05: Broken Function Level Authorization	1	12	15
OWASP API Security Top 10 Risk	A06: Unrestricted Access to Sensitive Business Flows	0	0	0
OWASP API Security Top 10 Risk	A07: Server Side Request Forgery	1	0	1
OWASP API Security Top 10 Risk	A08: Security Misconfiguration	7	2	2
OWASP API Security Top 10 Risk	A09: Improper Inventory Management	1	0	0
OWASP API Security Top 10 Risk	A10: Unsafe Consumption of APIs	3	30	30

Notiamo che:

- la OWASP Top 10 Web Application 2025 ha molte più CWE associate rispetto alla OWASP API Security Top 10 2023, e risulta quindi più facile trovare tecniche offensive e artefatti digitali associati;
- tutti i rischi della OWASP Top 10 Web Application 2025 sono associati ad almeno una CWE, mentre nella OWASP API Security Top 10 2023 il rischio A06: Unrestricted Access to Sensitive Business Flows non è associato a nessuna CWE;

- i seguenti rischi restano invece senza d3f:OffensiveTechnique associate: A03:2025 - Software Supply Chain Failures, A10:2025 - Mishandling of Exceptional Conditions, A03: Broken Object Property Level Authorization, A06: Unrestricted Access to Sensitive Business Flows, A09: Improper Inventory Management. Sono tutti possibili punti per sviluppare il KG di D3FEND e CAPEC, anche se l'assenza di CWE su A06 rappresenta un limite strutturale - collegabile anche alla vaghezza tecnica del concetto di "Sensitive Business Flows".

Un maggior raccordo di questi rischi con D3FEND - sebbene possibile oggetto di sviluppi futuri - non è negli obiettivi di questo lavoro.

In sez. 5.4, *Caso d'uso: individuare le tecniche ATT&CK applicabili ad un'architettura annotata con D3FEND* utilizzeremo la mappatura relativa alla OWASP API Security Top 10 2023 per validare la metodologia utilizzata in sez. 4.2, *Modellare un'infrastruttura: il caso d'uso di una web application* per modellare una web application minimale.

2.4 Altre ontologie per la cybersecurity

In questo lavoro si è deciso di usare l'ontologia D3FEND perché:

- è mantenuta da una fonte autorevole per tutto il dataset di riferimento in ambito cybersecurity, quali (MITRE Corporation s.d.b), (MITRE Corporation s.d.a), (MITRE Corporation s.d.e), ed è in costante evoluzione - con una community attiva capace di integrare contributi esterni;
- ha come obiettivo la rappresentazione delle tecniche difensive nel dominio della cybersecurity, quindi il supporto a metodologie di security by design;
- è pubblicata come KG usando degli standard quali RDF ed OWL;

Menzioniamo comunque, tra le varie ontologie per la cybersecurity, anche la Unified Cyber Ontology (UCO)¹⁰ - un lavoro community-driven per modellare concetti

¹⁰<https://unifiedcyberontology.org/> Unified Cyber Ontology (UCO). Consultato il 26 agosto 2026.

e relazioni in ambito cybersecurity. Questa ontologia ha un focus concettuale, e - a differenza di D3FEND - non pubblica dataset specifici, lasciando agli implementatori il compito di creare e gestire il proprio patrimonio informativo. In questo, UCO si differenzia da D3FEND, che invece fornisce non solo un dataset completo per ATT&CK, ma anche un modello di integrazione per altri dataset pubblicati dal MITRE.

2.5 Data Privacy Vocabulary (DPV)

Il Data Privacy Vocabulary (DPV) (Pandit, Esteves, Golpayegani, et al. 2026) è un vocabolario controllato nato in seno al W3C per rappresentare in modo machine-readable i metadati relativi al processamento di dati (anche personali) e supportare iniziative legislative come il General Data Protection Regulation (GDPR) dell'Unione Europea.

DPV definisce concetti sufficientemente generici da poter essere estesi per rappresentare scenari specifici di trattamento dei dati personali: questa estensione avviene tramite `rdfs:subClassOf` e `rdfs:subPropertyOf`, e permette di rappresentare in modo più preciso contesti giuridici specifici.

L'ultima versione dei vocabolari è la seguente:

- il vocabolario DPV (Pandit, Esteves, Golpayegani, et al. 2026) è disponibile all'URL <https://w3c-cg.github.io/dpv/2.3/dpv/>; l'estensione EU-GDPR (Pandit, Esteves, Koulirakis, et al. 2026) è disponibile all'URL <https://w3c-cg.github.io/dpv/2.3/legal/eu/gdpr/>.

Il documento (Pandit, Esteves, e Krog s.d.) fornisce una panoramica dei concetti principali di DPV e della sua estensione EU-GDPR.

In questo documento esploreremo l'utilizzo di DPV e della sua estensione EU-GDPR per rappresentare le informazioni relative al trattamento dei dati personali in un diagramma architetturale (vedi Sezione 2.3).

Un elemento di frizione tra DPV e D3FEND è relativo alla modellazione: mentre D3FEND è basato su OWL, per cui l'alberatura delle classi è rappresentata tramite `rdfs:subClassOf`, in DPV le relazioni tra i concetti sono rappresentate sia come `rdfs:subClassOf`, sia come `skos:broader`:

```
d3f:Person a owl:Class ;
  rdfs:subClassOf d3f:Agent .

dpv:DataSubject a rdfs:Class ;
  rdfs:subClassOf dpv:HumanSubject ;
  skos:broader dpv:HumanSubject .

dpv:HumanSubject a rdfs:Class ;
  rdfs:subClassOf dpv:LegalEntity ;
  skos:broader dpv:LegalEntity .

dpv:NaturalPerson a rdfs:Class ;
  rdfs:subClassOf dpv:Entity ;
  skos:broader dpv:Entity .

# dpv:User does not have a rdfs:subClassOf relation.
dpv:User a rdfs:Class, skos:Concept, dpv:HumanSubject ;
  skos:broader dpv:HumanSubject .

d3f:User a owl:Class ;
  rdfs:subClassOf d3f:Agent .
```

Analogamente a quanto fatto con D3FEND, è stata sviluppata un'estensione per Visual Studio Code che mostra direttamente nell'editor le informazioni pubblicate da DPV ed EU-GDPR.

Inoltre, creeremo un grafo sperimentale per connettere il KG di defend con quello di DPV ed EU-GDPR, in modo da poter interrogare il KG partendo da diverse prospettive.

DPV include un modulo Technology che non utilizzeremo: ci baseremo invece su D3FEND che contiene una tassonomia più utile durante la fase progettuale.

2.5.1 Panoramica DPV ed EU-GDPR

DPV si sviluppa intorno al concetto di trattamento (`dpv:Process`) e dei concetti ad esso collegati, da cui derivano le altre classi principali:

- `dpv:Process` - un'azione o un processo che coinvolge il trattamento dei dati personali. E' la classe principale attorno a cui ruotano le altre classi di DPV;
- `dpv:Purpose` - la finalità del trattamento dei dati personali;
- `dpv:Data` - il tipo di dato trattato;
- `dpv:Processing` - il tipo di trattamento dei dati personali;
- `dpv:Entity` - un'entità coinvolta nel trattamento dei dati personali, sia fisica che giuridica;
- `dpv:LegalBasis` - la base giuridica del trattamento dei dati personali;
- `dpv:Rule` - regole e vincoli relativi al trattamento dei dati personali;

L'estensione EU-GDPR specializza ulteriormente i concetti partendo dalle definizioni presenti nell'art. 4 del GDPR. Si vedano ulteriori dettagli in <https://w3c-cg.github.io/dpv/2.3/legal/eu/gdpr/#mapping>.

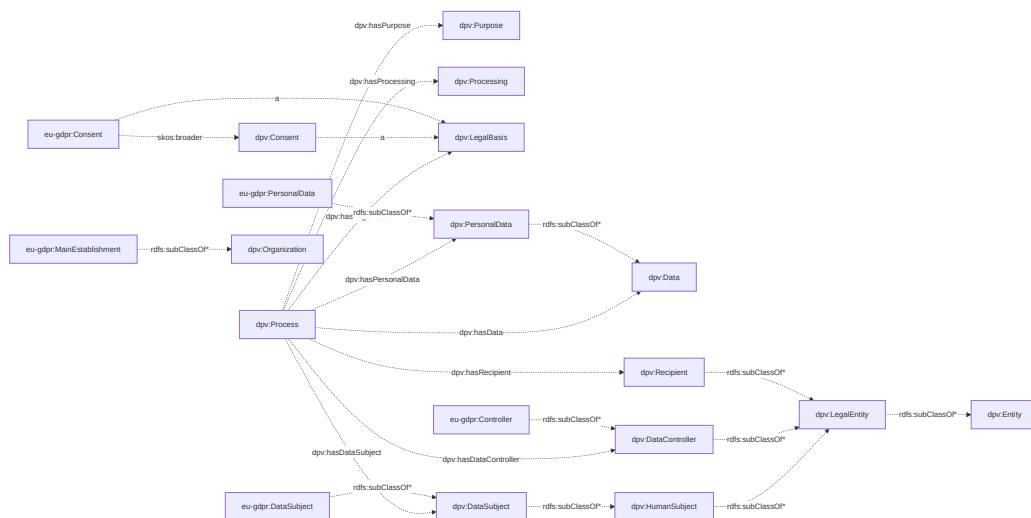


Figura 29: Panoramica DPV ed EU-GDPR

Mappatura tra DPV ed EU-GDPR Il documento (Pandit, Esteves, Koulierakis, et al. 2026) non modella tutti gli articoli di GDPR, focalizzandosi su quelli relativi alla definizione dei concetti utili alla modellazione dei trattamenti.

Concetti	Articoli
Legal bases	6 (all of 6-1-a...f, with sub-variants), 7-3, 9 (9-2-a...j)
Data subject rights	12 (justifications only), 13, 14, 15, 16, 17, 18, 19, 20, 21, 22 (22-3-a/b/c)
Third-country transfers	45-3, 46-2-a...f, 46-3-a/b, 49-1-a...g, 49-2
Remedies	77, 78, 79

Non c'è quindi una risorsa per ogni articolo della norma, poiché un articolo potrebbe essere rappresentato da più concetti.

Il documento articola la mappatura tra DPV ed EU-GDPR secondo le seguenti sezioni:

- §3 Liceità del trattamento: la base giuridica del trattamento dei dati personali: include Art. 6 (Liceità del trattamento), Art. 9 (Trattamento di categorie particolari di dati personali) e artt. 44-50 (Capo V, Trasferimenti di dati personali verso paesi terzi o organizzazioni internazionali) del GDPR;
- §4 Principi applicabili al trattamento di dati personali: i principi generali, come definiti nell'art. 5 del GDPR;
- §5 Diritti dell'interessato: i diritti degli interessati, come definiti negli artt. 12-23 del GDPR;
- §6 Trasferimenti verso paesi terzi: condizioni per il trasferimento di dati personali verso paesi terzi, Art. 46 GDPR;
- §7 Valutazione d'impatto sulla protezione dei dati (DPIA): valutazione d'impatto sulla protezione dei dati, Art. 35 GDPR;
- §8 Data breach: violazioni dei dati personali, Art. 33-34 GDPR;
- §9 Titolare del trattamento, stabilimento principale e Authorities: definizioni di titolare del trattamento e stabilimento principale, Art. 4 GDPR;
- §10 Conformità;
- §11 Codici di condotta e certificazioni: codici di condotta e certificazioni, (Capo IV, Sezione 5) Art. 40-43 GDPR;
- §12 Altri concetti: altri concetti, quali quelli legati alle tipologie di dato (e.g., `eu-gdpr:BiometricData`), o alle tipologie di trattamento (e.g., `eu-gdpr:Profiling`).

In questo documento affronteremo solo una parte di questi concetti, strumentalmente più utili a rappresentare le architetture IT e i flussi di dati personali.

Ulteriori classi di DPV

- `dpv:Context` - informazioni ulteriori sul contesto del trattamento dei dati personali;
- `dpv:TechnicalOrganisationalMeasure` - misure tecniche e organizzative adottate nel trattamento dei dati personali;
- `dpv:RiskConcept` - concetti relativi al rischio.

2.5.2 Casi d'uso

In questa sezione riportiamo alcuni casi d'uso di EU-GDPR basati su quelli contenuti in (Pandit, Esteves, Koulierakis, et al. 2026).

Esempio 1: il trattamento (`dpv:Process`) di un dato personale (`dpv:PersonalData`) lecito (`dpv:LegalBasis`) in base all'articolo 6, paragrafo 1, lettera b del GDPR (`eu-gdpr:A6-1-b`) che viene effettuato da un fornitore di servizi cloud (`ex:MyCloudProvider`) sulla base di un contratto con il titolare del trattamento (`ex:MyOrganization`).

```
@prefix dpv: <https://w3id.org/dpv#>
@prefix eu-gdpr: <https://w3id.org/dpv/legal/eu/gdpr#>
@prefix ex: <https://example.org/>

ex:Treatment_1 a dpv:Process ;
  dpv:hasData eu-gdpr:PersonalData ;
  dpv:hasDataProcessor ex:MyCloudProvider ;
  dpv:hasLegalBasis eu-gdpr:A6-1-b ;
  dpv:hasDataController ex:MyOrganization .

ex:MyOrganization a eu-gdpr:Controller .
ex:MyCloudProvider a eu-gdpr:Processor .
```

Esempio 2: il trattamento di un dato biometrico (`eu-gdpr:BiometricData`) per fini di autenticazione (`dpv:IdentityAuthentication`) in base al consenso dell'interessato (`eu-gdpr:A6-1-a`).

```
@prefix dpv: <https://w3id.org/dpv#>
@prefix eu-gdpr: <https://w3id.org/dpv/legal/eu/gdpr#>
@prefix ex: <https://example.org/>

ex:Treatment_2 a dpv:Process ;
  dpv:hasData eu-gdpr:BiometricData ;
  dpv:hasPurpose dpv:IdentityAuthentication ;
  dpv:hasLegalBasis eu-gdpr:A6-1-a ;
  dpv:hasDataSubject ex:User .

ex:User a dpv:NaturalPerson .
```

2.5.3 Attori ed entità

Le entità coinvolte nel trattamento dei dati personali sono rappresentate da `dpv:Entity`, che può essere una persona fisica (`dpv:NaturalPerson`) o giuridica (`dpv:LegalEntity`).

Nota: attualmente `dpv:DataSubject` non è una sottoclasse di `dpv:NaturalPerson`.

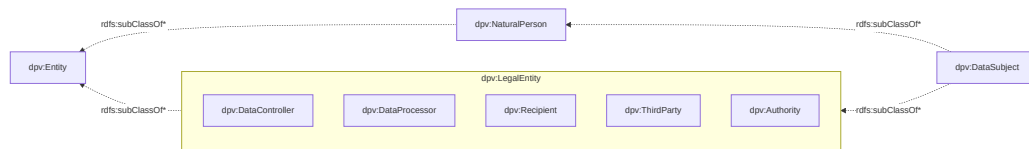


Figura 30: Attori ed entità in DPV

Relazioni con D3FEND Il concetto D3FEND più vicino a `dpv:Entity` è quello di `d3f:Agent`, che rappresenta un'entità capace di agire intenzionalmente. `dpv:Agent` invece pone l'accento sulla capacità di agire in nome e per conto di un'altra entità. I due concetti non sono direttamente collegati.

Ai fini di rappresentare le architetture IT e i flussi di dati personali, possiamo limitarci a considerare le seguenti corrispondenze tra le sottoclassi di `dpv:Entity` e le sottoclassi di `d3f:Agent`:

- `d3f:Person` `skos:closeMatch` `dpv:NaturalPerson` $\sqsubseteq$ `dpv:Entity`
- `d3f:Organization` `skos:closeMatch` `dpv:Organisation` $\sqsubseteq$ `dpv:LegalEntity`
- `d3f:ServiceProvider` `skos:closeMatch` `dpv:ServiceProvider` $\sqsubseteq$ `dpv:LegalEntity`

Ci sono poi delle differenze sostanziali:

- `d3f:User` può o meno essere umano, quindi è un concetto più ampio di `dpv:User`
 $\sqsubseteq$ `dpv:HumanSubject`
- `d3f:Agent` è un concetto più ampio di `dpv:Agent`

2.5.4 Data e PersonalData

Il concetto di dato (`dpv:Data`) viene declinato in diverse sottoclassi, tra cui notiamo quelle di:

- dato personale (`dpv:PersonalData`);
- dato confidenziale (`dpv:ConfidentialData`);
- dato sensibile (`dpv:SensitiveData`).

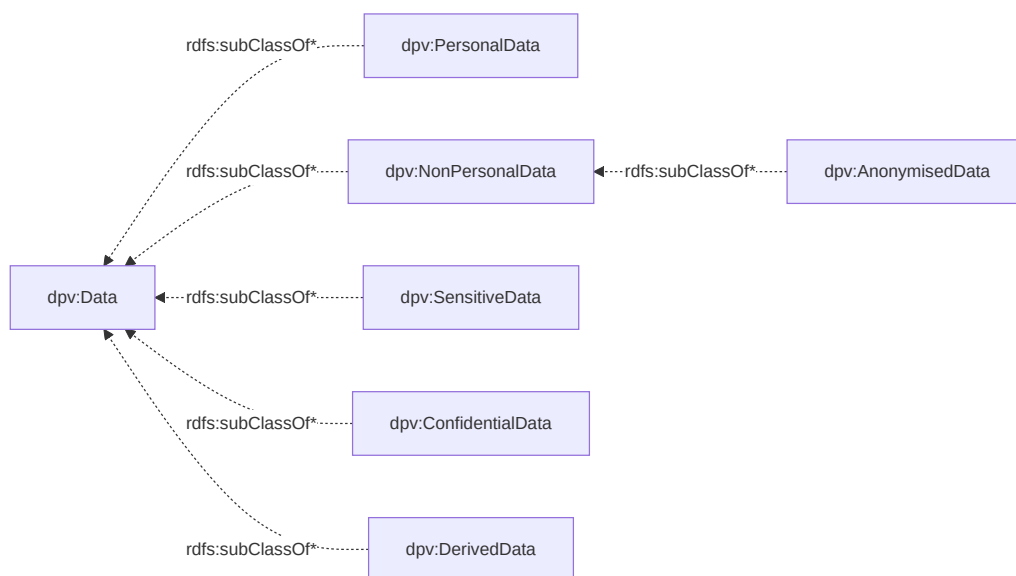


Figura 31: Sottoclassi di `dpv:Data`

Quando si è in un contesto GDPR, è possibile utilizzare direttamente la sottoclasse `eu-gdpr:PersonalData` che fa riferimento all'art. 4, paragrafo 1 del GDPR.

Relazioni con D3FEND Non esiste una corrispondenza diretta tra DPV e D3FEND, poiché D3FEND non entra nel merito della tipologia giuridica dei dati trattati, ma della sola parte tecnologica.

Ad esempio, un file Portable Document Format (PDF) che contiene una prescrizione medica (`dpv:Prescription`) può essere rappresentato come un `d3f:DocumentFile`, ma va indicato esplicitamente il fatto che contiene dati personali (`dpv:PersonalData`) e che tipo di dato è (`dpv:Prescription`).

```
:prescription.pdf a d3f:DocumentFile,  
  eu-gdpr:PersonalData,  
  pd:Prescription .
```

2.5.5 Contesto e Luogo del trattamento

DPV permette di rappresentare il contesto del trattamento dei dati personali, come ad esempio il luogo in cui avviene il trattamento. Questo è utile per definire in D3FEND degli elementi legati alla localizzazione dei dati (e.g., dei datacenter) utilizzando una tassonomia dei luoghi di trattamento.

2.5.6 Misure di sicurezza

In DPV le misure di sicurezza sono rappresentate da `dpv:TechnicalOrganisationalMeasure`, che presenta diversi tipi di misure, a livello:

- legale (`dpv:LegalMeasure`),
- organizzativo (`dpv:OrganisationalMeasure`),
- tecnico (`dpv:TechnicalMeasure`) e fisico (`dpv:PhysicalMeasure`),

e più generalmente:

- di mitigazione del rischio (`dpv:RiskMitigationMeasure`).

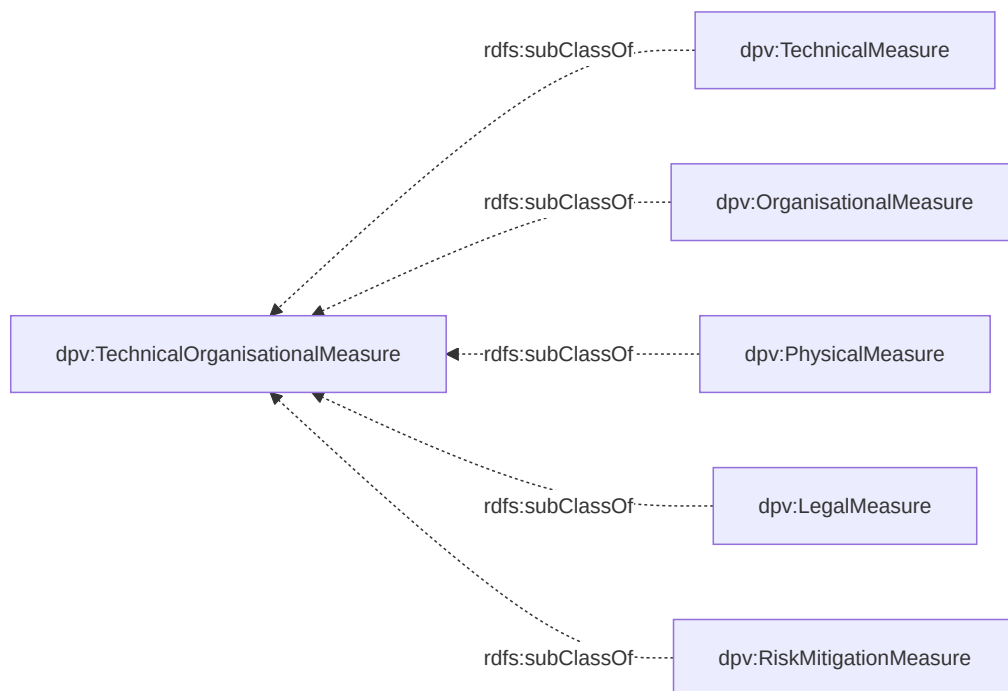


Figura 32: Sottoclassi di dpv:TechnicalOrganisationalMeasure

Relazioni con D3FEND In D3FEND le contromisure sono rappresentate da `d3f:DefensiveTechnique` che rimanda alla voce security control (National Institute of Standards and Technology s.d.b) del glossario NIST e - ricordiamo -

```

===== NOTE: '\ ' line wrapping per RFC 8792 =====

d3f:DefensiveTechnique d3f:enables d3f:DefensiveTactic .

d3f:DefensiveTactic ~rdfs:subClassOf d3f:Model, d3f:Harden, d3f:Detect,\
d3f:Isolate, d3f:Deceive, d3f:Evict, d3f:Restore .

```

La relazione tra le misure di sicurezza di DPV e le contromisure di D3FEND non è diretta, poiché DPV si concentra sul trattamento dei dati personali, mentre D3FEND sulle tecniche di difesa, indipendentemente dal tipo di dato trattato.

La strada più semplice è quella di associare ad ogni `d3f:DefensiveTechnique` di un livello sufficientemente generico i corrispondenti concetti giuridici di DPV, come nell'estratto seguente.

```

===== NOTE: '\ ' line wrapping per RFC 8792 =====

@prefix al: <http://github.com/ioggstream/d3sign#> .

```

```

al:map-restore-database-gdpr-32-1-c a al:Mapping ;
al:d3fend-class d3f:RestoreDatabase ;
al:legal-concept dpv:DataBackupProtocols ;
al:obligation ob:gdpr-art32-1-c ;
al:strength al:Partial ;
al:rationale "Article 32(1)(c) asks for the ability to restore av\
ailability and access. Restoring the database that holds the personal d\
ata is that ability; whether it happens in a timely manner is not somet\
hing the diagram states."@en ;
dct:source "GDPR Article 32(1)(c)"@en ;
rdfs:seeAlso <https://eur-lex.europa.eu/eli/reg/2016/679/art_32/o\
j> ;
al:review-status al:Reviewed ; dct:date "2026-08-10"^^xsd:date .

#
# d3f:DefensiveTechnique skos:relatedMatch dpv:TechnicalOrganisationalM\
easure .
#
d3f:RestoreDatabase skos:relatedMatch dpv:DataBackupProtocols .

```

Un'altra possibile mappatura consiste nell'associare ad ogni `dpv:TechnicalOrganisationalMeasure` una `d3f:DefensiveTactic` creando un allineamento a maglia larga, per finalità, piuttosto che di equivalenza.

d3f:Defensive Tactic	d3f:DefensiveTechnique	dpv:TechnicalMeasure	dpv:Organisational Measure
d3f:Model	d3f:AssetInventory, d3f:SystemMapping, d3f:NetworkMapping, d3f:OperationalActivity Mapping	dpv:Vulnerability TestingMethods (parziale)	dpv:RecordsOf Activities, dpv:Governance Procedures, dpv:Asset Management Procedures
d3f:Harden	d3f:AgentAuthentication, d3f:Application Hardening, d3f:CredentialHardening, d3f:Message Hardening, d3f:PlatformHardening, d3f:Source CodeHardening	dpv:Encryption, dpv:Cryptographic Methods, dpv:Authentication Protocols, dpv:Security Method	dpv:Information SecurityPolicy, dpv:SecurityProcedure, dpv:DesignStandard
d3f:Detect	d3f:FileAnalysis, d3f:IdentifierAnalysis, d3f:MessageAnalysis, d3f:NetworkTraffic Analysis, d3f:PhysicalAccessMonitoring, d3f:PlatformMonitoring, d3f:ProcessAnalysis, d3f:UserBehaviorAnalysis	dpv:ActivityMonitoring, dpv:IntrusionDetection System	dpv:MonitoringPolicy, dpv:LoggingPolicy
d3f:Isolate	d3f:AccessMediation, d3f:AccessPolicy Administration, d3f:ContentFiltering, d3f:ExecutionIsolation, d3f:NetworkIsolation	dpv:InformationFlow Control, dpv:Access ControlMethod, dpv:Authorisation Protocols	dpv:Authorisation Procedure, dpv:Permission Management, dpv:SecureProcessing Environment
d3f:Deceive d3f:Evict	d3f:DecoyObject, d3f:DecoyEnvironment d3f:CredentialEviction, d3f:ProcessEviction, d3f:ObjectEviction	nessuna dpv:DataSanitisation Technique (parziale)	nessuna dpv:Incident Management Procedures, dpv:Credential Management, dpv:Data DeletionPolicy
d3f:Restore	d3f:RestoreObject, d3f:RestoreAccess	dpv:DataBackup Protocols, dpv:FailSafe Protocols	dpv:DisasterRecovery Procedures, dpv:Data RestorationPolicy

Notiamo che sebbene ci siano delle differenze tra `dpv:TechnicalMeasure` e `dpv:OrganisationalMeasure`, entrambe le categorie possono essere mappate su tattiche difensive di D3FEND, sebbene con tecniche diverse, ad esempio: la `d3f:Harden` $\sqsubseteq$ `d3f:Tactic` può essere abilitata sia da `dpv:TechnicalMeasure` che da `dpv:OrganisationalMeasure` quali `dpv:DesignStandard`.

Inoltre, vediamo che non riusciamo a mappare nessuna delle `dpv:TechnicalOrganisationalMeasure` su `d3f:Deceive`: questo può essere dovuto al fatto che DPV ed EU-GDPR non individuano un quadro giuridico per questo tipo di tattiche.

3 Diagrammi architetturali

Abbiamo visto che la conoscenza può essere rappresentata tramite grafi. Allo stesso modo, possiamo rappresentare un'infrastruttura informatica come un insieme di nodi interconnessi.

I nodi possono indicare sia elementi fisici che digitali, nonché degli attori (e.g., utenti). Le connessioni invece possono rappresentare:

- flussi di dati;
- azioni e/o eventi;
- ma anche relazioni generiche.

Oggi i diagrammi architetturali vengono rappresentati in modo semplificato, sono spesso ritagliati su contesti specifici - quali quelli dei cloud provider (e.g., Amazon Web Services, Microsoft Azure, Google Cloud Platform) o delle piattaforme di orchestrazione standard come Kubernetes (The Kubernetes Authors 2014), e più che per finalità di progettazione, vengono utilizzati per finalità di documentazione e/o marketing.

Si usano quindi notazioni ritagliate intorno alle funzionalità specifiche degli ambienti. Le relazioni tra i nodi sono spesso implicite, e/o di alto livello, inefficaci per dettagliare in modo indipendente dal vendor della piattaforma il funzionamento dell'architettura.

In questo documento rappresenteremo le architetture IT utilizzando la sintassi Mermaid.js (Mermaid Contributors 2014) per i grafi (i.e., flowchart). Si tratta di una rappresentazione sufficientemente flessibile, adatta a rappresentare sia le architetture IT che i grafi in formato RDF - seppure con qualche limitazione: non è possibile rappresentare dei nodi presenti in più sottografi (i.e., i sottografi sono tutti disgiunti). E' sempre possibile aggirare questa limitazione utilizzando al posto dei sottografi, delle relazioni di inclusione.

Questo approccio è utilizzato anche per rappresentare gli oggetti Kubernetes, come mostrato nella Kubernetes Diagram Style Guide (The Kubernetes Authors s.d.).

Inoltre, la sintassi Mermaid.js è supportata da diversi strumenti di documentazione e piattaforme di code hosting, come GitHub e GitLab: questo permette di integrare facilmente i diagrammi architetturali nelle documentazioni dei progetti software, senza ricorrere a strumenti esterni.

Un primo prototipo basato su questo approccio - implementato usando Mermaid.js e PyScript - è stato presentato nel 2023 a PyCon Italia (Polli 2023) e condiviso sul forum di D3FEND il 4 febbraio 2023¹¹. Da quel prototipo è derivata l'applicazione Design & D3FEND (Polli 2026), descritta in sez. 5, *Implementazioni*.

3.1 Altre modalità di rappresentazione delle architetture

3.1.1 D3FEND CAD

D3FEND Cyber Attack-Defense (CAD) (MITRE Corporation 2026) è uno strumento grafico creato dal MITRE per rappresentare gli oggetti di D3FEND e le loro relazioni in formato visuale, generando un grafo in formato RDF.

Una beta è stata pubblicata nel 2024 con il nome di “graph-studio” mentre la prima release stabile è del 2025¹². Sebbene il prototipo di Design & D3FEND presentato nel 2023 sia precedente al CAD, il lavoro fatto durante questo project work ha cercato di integrare alcune delle funzionalità del CAD - come quella di filtrare le risorse in base a delle classificazioni.

D3FEND CAD è uno strumento molto granulare, utile a descrivere nel dettaglio gli attacchi informatici anche da un punto di vista temporale.

Tramite il CAD si possono creare e connettere diversi tipi di nodi in modo assistito, e navigare e filtrare le risorse in base a diverse classificazioni, come:

- Action: `d3f:Action`, `d3f:OffensiveTechnique (Attack)`, `d3f:DefensiveTechnique (Countermeasure)`

¹¹Si veda l'annuncio sullo Slack Space di D3FEND <https://mitre-d3fend.slack.com/archives/C02QRK1RRUL/p1675517395384789>. Consultato il 26 agosto 2026.

¹²<https://www.mitre.org/news-insights/news-release/mitre-launches-d3fend-10-milestone-cybersecurity-ontology>. Consultato il 26 agosto 2026.

- Object: `d3f:DigitalArtifact`, `d3f:Agent`
- State: `d3f:Weakness`, `d3f:Condition`
- Misc: Thing, Note (queste entità, non mappate in D3FEND, possono essere associate a `rdfs:Class` prese da ontologie esterne).

La differenza principale tra i due strumenti è che il CAD lavora ad un livello di dettaglio molto più basso, disaccoppiato dalla rappresentazione delle architetture IT, e destinato ad un pubblico di esperti di sicurezza informatica.

Design & D3FEND invece è uno strumento che lavora ad un livello più alto, integrando la rappresentazione delle architetture IT con la conoscenza di D3FEND, e destinato ad un pubblico più ampio, che include anche gli architetti IT e gli sviluppatori.

Nonostante queste differenze, entrambi rappresentano la conoscenza in formato RDF: Design & D3FEND permette - con diverse limitazioni - di “importare” un grafo creato con D3FEND CAD e visualizzarlo.

3.1.2 UML

Fino alla metà degli anni 2000, UML (Object Management Group 2017) era lo standard de facto per la rappresentazione delle architetture software. Col tempo, questo si è dimostrato poco efficace per la rappresentazione delle infrastrutture. Viene quindi utilizzato solo in ambiti molto specifici, o nell’ambiente accademico: questo anche perché la variabilità dei requisiti funzionali e non funzionali, nonché la disponibilità di componenti software sempre più diversificati nel tempo e/o “as a service”, non giustifica l’adozione di un formato particolarmente ingessato come UML.

Si noti inoltre come, sebbene in molti casi UML sia utilizzato per modellare ontologie in formato RDF, questi due formati non sono isomorfi (si veda ad es. l’annesso normativo al mapping tra i due formalismi (Object Management Group 2014), che comunque risale al 2014).

3.1.3 C4 Model e Mermaid Architecture Diagram

C4 Model(Brown 2018) è un'alternativa ad UML per la rappresentazione di architetture software. Si tratta di un approccio pragmatico, che si concentra su quattro livelli di astrazione:

- **Contesto:** rappresenta il sistema nel suo insieme, e le interazioni con gli attori esterni.
- **Contenitore:** rappresenta i principali componenti del sistema, e le loro interazioni.
- **Componente:** rappresenta i componenti interni del sistema, e le loro interazioni.
- **Codice:** rappresenta il codice sorgente del sistema, e le sue interazioni con i componenti esterni.

Non è sufficientemente generico però, ed obbliga a far rientrare ogni componente in uno dei quattro livelli.

Un altro modello è l'architecture diagram, che si focalizza sulla rappresentazione dei componenti e delle loro interazioni, raggruppando i componenti in livelli con una semantica definita di volta in volta. Questo modello non è sufficientemente espressivo per essere usato in ambito cyber.

3.1.4 Esempio di diagramma architetturale di AWS

Il diagramma in figura 33 sintetizza l'architettura di una piattaforma su AWS.

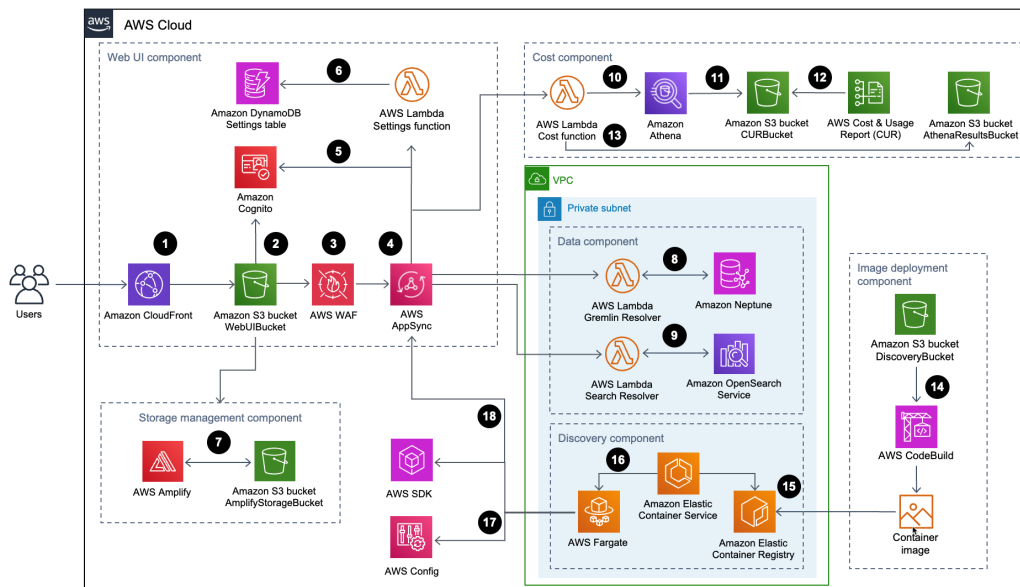


Figura 33: Architettura di una piattaforma su AWS

Analizzando il flusso informativo che parte dall'utente,

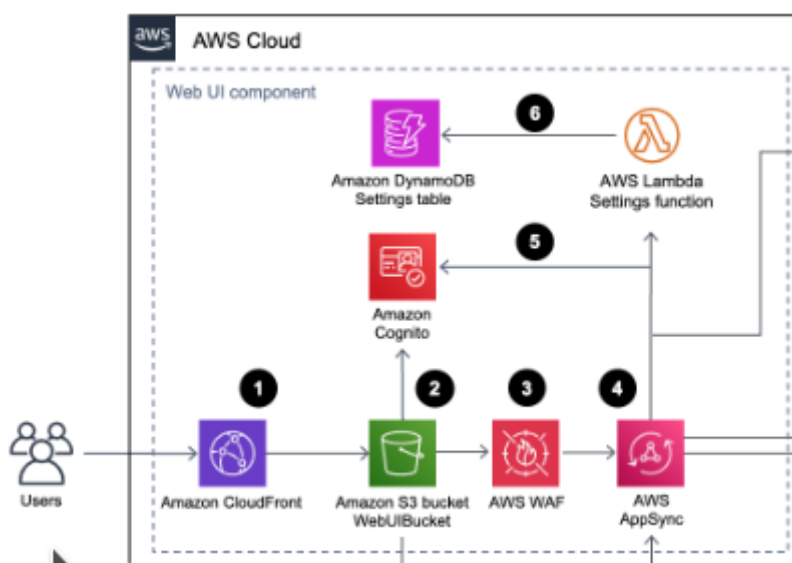


Figura 34: Flusso informativo dell'utente attraverso la CDN e il WAF

vediamo come questa rappresentazione sia ambigua sotto diversi punti di vista.

1. L'utente si connette alla Content Delivery Network (CDN) pubblicata dal servizio CloudFront, che restituisce una pagina web recuperata da un bucket S3.

2. Il diagramma mostra quindi due archi che partono dal bucket S3: una verso l'identity server ed una verso il Web Application Firewall (WAF).

Se provassimo ad associare ad ogni connessione un predicato, ci troveremmo in difficoltà, poiché non esiste una relazione diretta tra il bucket S3 e l'identity server, né tra il bucket S3 e il WAF.

Leggendo la descrizione del flusso informativo associata all'immagine ¹³

Step 1 HTTP Strict-Transport-Security (HSTS) adds security headers to each response from the Amazon CloudFront distribution.

Step 2 An Amazon Simple Storage Service (Amazon S3) bucket hosts the web UI, which is distributed with Amazon CloudFront. Amazon Cognito authenticates user access to the web UI.

Step 3 AWS WAF protects the AppSync API from common exploits and bots that can affect availability, compromise security, or consume excessive resources.

scopriamo che è la CDN ad inoltrare le richieste dell'utente verso il WAF.

Usiamo la terminologia D3FEND per rappresentare in modo preliminare le relazioni tra i nodi, limitandoci ai predicati:

- `d3f:accesses`: The subject `x` takes the action of reading from, writing into, or executing the stored information in the object `y`. Reads, writes, and executes are specific cases of accesses.
- `d3f:reads`: The subject `x` takes the action of reading the stored information in the object `y`.
- `d3f:mediates-access-to`: The entity `x` controls and brokers requests to reach entity `y`, enforcing the access rules that allow or deny it.

¹³<https://web.archive.org/web/20260802164203/https://docs.aws.amazon.com/solutions/workload-discovery-on-aws/>. Consultato 2 agosto 2026.

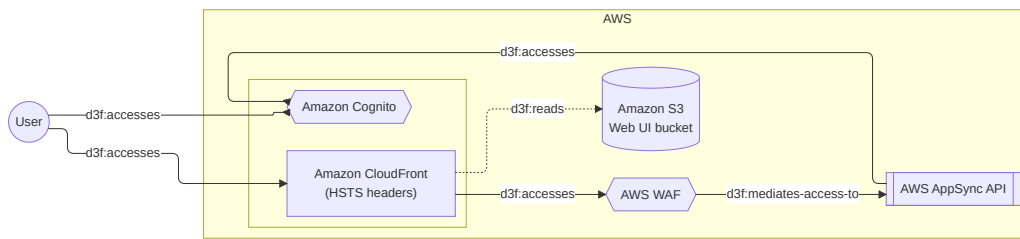


Figura 35: Disambiguazione dei flussi informativi della WebUI.

Si noti come anche questa rappresentazione sia parziale, poiché non include dettagli relativi ai `d3f:DigitalArtifact`, in particolare quelli collegati al networking (i.e., gli HTTP message (Fielding, Nottingham, e Reschke 2022, sez. 3.4), i JSON Web Token (JWT), il traffico di rete, ecc.).

4 Progettazione sicura con grafi architetturali basati su D3FEND

Abbiamo visto che D3FEND permette di rappresentare in modo più preciso le relazioni tra i nodi di un'infrastruttura informatica.

Nella fase di progettazione, ci si concentra sulle componenti funzionali, ma vanno considerati anche i requisiti in termini di risorse, verificando la presenza di componenti che modellano:

- risorse computazionali (CPU, RAM, ecc.);
- networking (traffico, elementi di rete, ecc.);
- storage (dischi, spazio cloud, ecc.).

In questa sezione partiremo da un esempio di architettura minimale ed andremo a dettagliare progressivamente tutte le sue componenti utilizzando RDF.

Per prima cosa connettiamo i due grafi, quello architetturale e quello di D3FEND, utilizzando la property `rdf:type` - con il suo alias `a`.

Questo ci permette di integrare le informazioni di D3FEND.

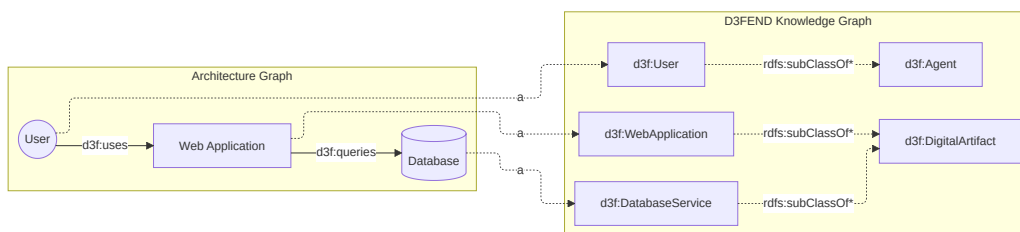


Figura 36: Diagramma minimale di una Web Application

Ricordiamo inoltre l'uso dei termini:

- **diagramma**: per la rappresentazione in formato `mermaid.js` di un'architettura informatica;
- **grafo**: per la rappresentazione in formato RDF di un'architettura informatica.

4.1 Usare la sintassi Mermaid Flowchart per rappresentare i grafi architetturali

In questo documento, utilizzeremo la sintassi Mermaid.js (Mermaid Contributors 2014) flowchart per rappresentare i grafi architetturali. Una descrizione dettagliata della sintassi è disponibile nella sezione relativa del sito.¹⁴

4.1.1 Rappresentazione dei grafi

E' possibile trasformare un diagramma in un grafo RDF. Il nome del grafo è specificato nel campo `id` dell'intestazione (frontmatter) del diagramma, mentre il titolo del grafo è specificato nel campo `title`.

```
---
id: 040-webapp-minimal
---

token[Access Token d3f:WebAccessToken]
```

diventa

```
@prefix d3f: <http://d3fend.mitre.org/ontologies/d3fend.owl#> .
@base : <urn:example:> .

:040-webapp-minimal {
  token a d3f:WebAccessToken ;
  rdfs:label "Access Token"
}
```

Se il grafo non è specificato, viene usato un valore di default.

4.1.2 Rappresentazione dei nodi

- ogni nodo è rappresentato da un nodo mermaid, il cui `id` è l'identificativo del nodo stesso;
- la label del nodo è composta da una stringa;

Esempio:

¹⁴Vedi <https://mermaid.js.org/syntax/flowchart.html>. Consultato il 26 agosto 2026.

```
%% L'id di questo nodo è "token",
%%  mentre la label è "Access Token d3f:WebAccessToken"
token[Access Token d3f:WebAccessToken]
```

oppure

```
token@{label: "Access Token d3f:WebAccessToken" }
```

- i commenti non hanno significato semantico, ma servono a documentare il diagramma; non vengono riportati nel grafo.

```
%% This is a comment
browser[Firefox d3f:Browser]
```

- la label contiene una o più classi D3FEND, separate da spazi, che rappresentano l'`rdf:type` del nodo, similmente a quanto visto nei Labeled Property Graph (LPG) (vedi [Labeled Property Graphs e RDF 1.2](#));

Esempio:

```
%% Un nodo che è sia una d3f:WebApplication che un d3f:PythonPackage.
webapp[Web Application d3f:WebApplication d3f:PythonPackage]
```

- eventuali annotazioni aggiuntive specificate con la sintassi Mermaid.js (Mermaid Contributors 2014) sono un dettaglio di serializzazione e non hanno alcun significato semantico.

Esempio:

```
===== NOTE: '\' line wrapping per RFC 8792 =====

%% shape ed icon vengono ignorate.
webapp[Web Application d3f:WebApplication d3f:PythonPackage]@{shape: ca\
rd, icon: "https://d3fend.mitre.org/cad/images/d3fend-icon-web-applicat\
ion.svg"}
```

4.1.3 Rappresentazione delle relazioni

Le `rdf:Property` che rappresentano delle relazioni tra i nodi sono rappresentate come archi direzionali.

L'architettura viene rappresentata quindi:

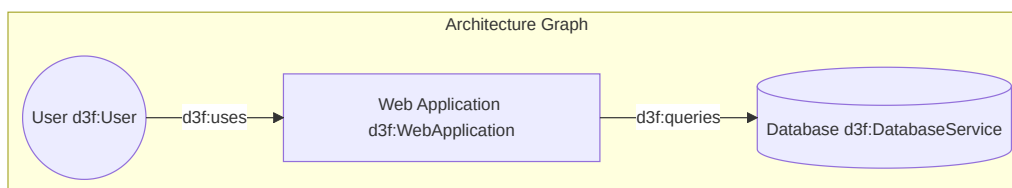


Figura 37: Diagramma annotato di una Web Application

4.1.4 Relazioni transitive

Una relazione transitiva (e.g., `d3f:contains` che rappresenta una generica relazione di appartenenza sia fisica che logica. Possiamo usarla per indicare l'appartenenza di un nodo ad una rete) può essere rappresentata come arco diretto tra due nodi,

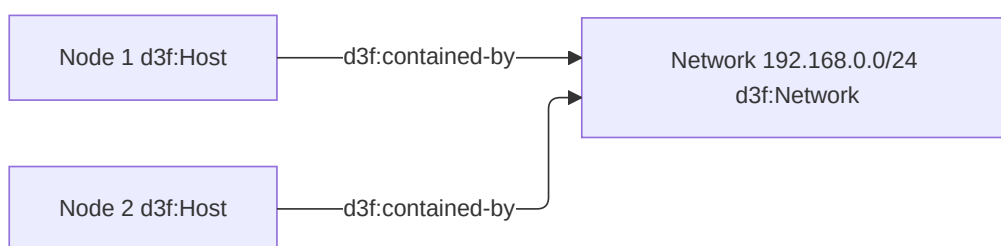


Figura 38: Relazione transitiva rappresentata come archi

oppure tramite la funzionalità `subgraph`.

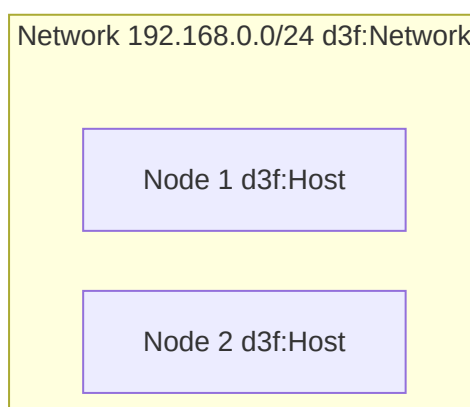


Figura 39: Relazione transitiva rappresentata come sotto-diagramma

I precedenti diagrammi generano quindi lo stesso grafo RDF (modulo il nome del grafo, generato a partire dall'id).

```
@prefix d3f: <http://d3fend.mitre.org/ontologies/d3fend.owl#> .
@base : <urn:example:> .
:040-webapp-minimal-transitive {
  network a d3f:Network ;
    rdfs:label "Network 192.168.0.0/24" ;
    d3f:contains :node-1, :node-2 .
  node-1 a d3f:Host ;
    rdfs:label "Node 1" .
  node-2 a d3f:Host ;
    rdfs:label "Node 2" .
}
```

4.1.5 Rappresentazione di d3f:DefensiveTechnique

Come visto in sez. 2.2, *MITRE D3FEND*, esistono `d3f:DigitalArtifact` che rappresentano delle tecniche difensive (`d3f:DefensiveTechnique`), ma l'ontologia non definisce relazioni tra di essi.

Per una rappresentazione visivamente più semplice, possiamo annotare un nodo con le `d3f:DefensiveTechnique` associate. In questo documento, per semplicità, consideriamo lo stesso nodo del grafo come una `d3f:DefensiveTechnique` e `d3f:DigitalArtifact` allo stesso tempo, delegando a un lavoro futuro una migliore traduzione in RDF.

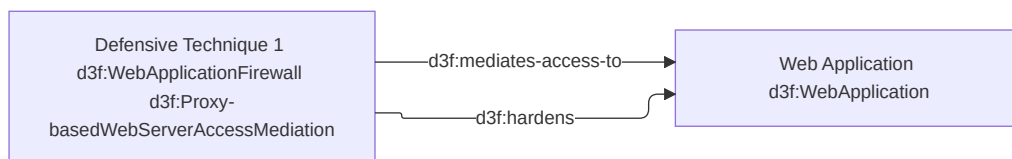


Figura 40: Diagramma minimale di una Web Application con le tecniche difensive

Diviene quindi

```
===== NOTE: '\' line wrapping per RFC 8792 =====

@prefix d3f: <http://d3fend.mitre.org/ontologies/d3fend.owl#> .
@base : <urn:example:> .
:040-webapp-minimal-defensive-techniques {
  waf a d3f:WebApplicationFirewall, d3f:Proxy-basedWebServerAccessMedia\
tion ;
    rdfs:label "Defensive Technique 1" ;
    d3f:mediates-access-to :app ;
    d3f:hardens :app .
}
```

```

app a d3f:WebApplication ;
    rdfs:label "Web Application" .
}

```

4.1.6 Rappresentazione di flussi informativi

Per rappresentare i flussi informativi tra i nodi, possiamo usare le proprietà D3FEND `d3f:accesses` e la sua inversa `d3f:accessed-by`, ad esempio per una data pipeline, sia focalizzandoci sui processi:



Figura 41: Una data pipeline

sia dettagliando gli oggetti scambiati: un client che fa delle richieste HTTP ad un server può essere rappresentato come segue:

- le query vengono prodotte dal client e processate dal server, quindi si può usare `d3f:executed-by`;
- per le transazioni HTTP, la classe più specifica $\sqsubseteq$ `d3f:DigitalInformation` è `d3f:RemoteCommand`, useremo quindi `d3f:WebResourceAccess` $\sqsubseteq$ `d3f:DigitalInformation`

La scelta di separare o meno i vari artefatti digitali dipende dal livello di dettaglio che si vuole rappresentare.

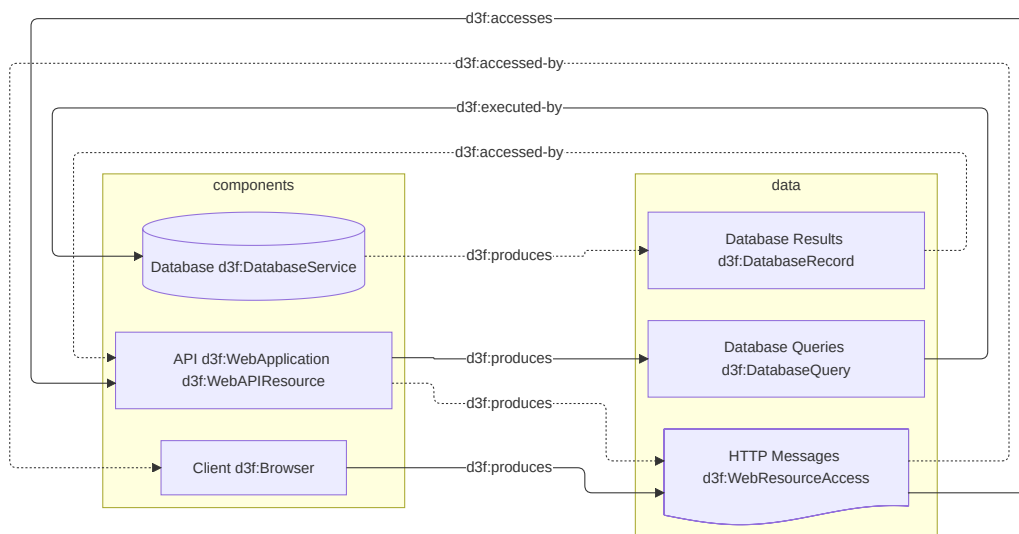


Figura 42: Diagramma minimale di una Web Application con i flussi informativi

4.2 Modellare un'infrastruttura: il caso d'uso di una web application

Per una rappresentazione efficace e realistica di un'infrastruttura informatica, vanno considerate tutte le componenti del sistema: ciò che non è rappresentato viene trascurato.

Il KG ci viene in aiuto suggerendo una serie di relazioni da considerare, a partire da un nodo di interesse, ma è sempre necessario integrare eventuali informazioni mancanti in termini di:

- risorse computazionali (CPU, RAM, ecc.);
- networking (traffico, elementi di rete, ecc.);
- storage (dischi, spazio cloud, ecc.).

In questa sezione vedremo il caso d'uso di una web application minimale, e di come D3FEND ci aiuti nell'individuare:

- le componenti mancanti, e quindi le relazioni da aggiungere al grafo;
- le tecniche offensive applicabili, e quindi le contromisure da adottare;

- le tecniche difensive da adottare, e quindi le contromisure da implementare.

Design & D3FEND integra nell'interfaccia grafica le query SPARQL che useremo di seguito per estrarre dal KG le relazioni e gli oggetti mancanti.

4.2.1 Componenti implicite

In questo ci vengono in aiuto le query SPARQL che mostrano, a partire da un nodo, le relazioni d'intorno già presenti nell'ontologia (vedi sez. 2.2.4, *Intorno di una classe*).

Con un primo giro di intorno, riusciamo già ad integrare degli elementi mancanti come:

- le ACL per limitare le azioni dell'utente;
- il server web che esegue l'applicazione;
- le query al database.

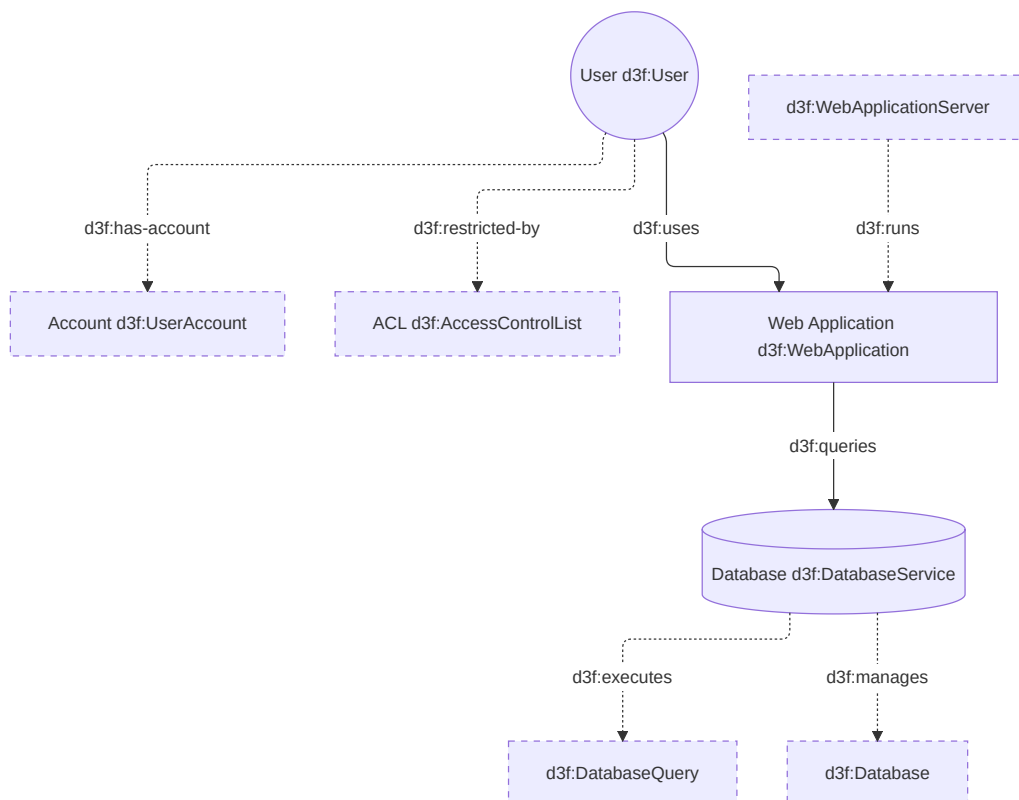


Figura 43: Diagramma di intorno di una Web Application

Guardando la gerarchia delle classi, vediamo anche che `d3f:RemoteDatabaseQuery` $\sqsubseteq$ `d3f:DatabaseQuery` rappresenta meglio una query eseguita via rete, mentre non vogliamo ancora dettagliare le modalità di autorizzazione: generalizziamo quindi `d3f:AccessControlConfiguration` $\sqsubseteq$ `d3f:AccessControlList`.

Da quanto visto in sez. 4.1.6, *Rappresentazione di flussi informativi*, introduciamo il concetto di `d3f:WebResourceAccess` per rappresentare gli scambi tra la Web Application e l'utente.

Inoltre chiariamo che la Web Application deve supportare dei meccanismi di accesso, associando `d3f:AccessControlConfiguration` e `d3f:WebApplication`.

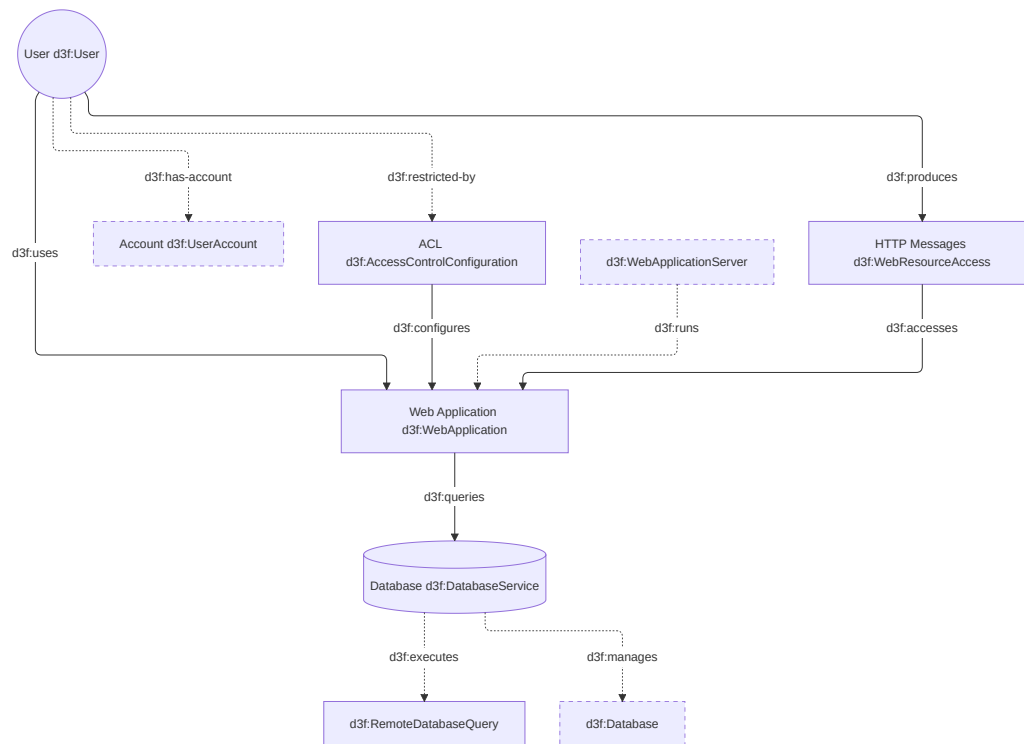


Figura 44: Diagramma di intorno di una Web Application con i meccanismi di accesso

Un ulteriore dettaglio potrebbe essere quello di inserire un nodo per rappresentare le singole `d3f:WebResource` esposte dalla Web App.

4.2.2 Risorse computazionali, networking e storage

Una descrizione architetturale include anche le risorse computazionali, di networking e di storage.

Un modo per individuarle è quello di navigare il KG tramite le relazioni transitive di contenimento.

Questa query elenca tutte le classi che contengono un nodo, attraversando le relazioni `d3f:contains` e `d3f:may-contain` che indichiamo col simbolo $\hookrightarrow$ presenti anche nelle eventuali sottoclassi.

```
===== NOTE: '\' line wrapping per RFC 8792 =====  
  
SELECT DISTINCT ?c0 ?container  
WHERE {  
  { SELECT DISTINCT ?g WHERE { GRAPH ?g {} FILTER(!STRSTARTS(STR(?g), S\  
TR(K:))) } }  
  GRAPH ?g { ?this a ?c0 . }  
  
  ?c0 (rdfs:subClassOf*/^(d3f:contains|d3f:may-contain))+ ?container .  
  FILTER(?container != ?c0)  
}
```

Otteniamo per esempio:

- per `d3f:Database: d3f:DatabaseFile  $\hookrightarrow$  d3f:FileSystem  $\hookrightarrow$  d3f:Storage;`
- per `d3f:WebApplicationServer: d3f:LocalAreaNetwork`
- per `d3f:RemoteDatabaseQuery: d3f:NetworkTraffic`
- per `d3f:WebApplication: d3f:Server  $\sqsubseteq$  d3f:WebApplicationServer`

che integreremo nel diagramma.

Gli elementi di rete, storage e computing non rappresentati vanno invece ricercati e aggiunti manualmente: per analogia con `d3f:NetworkTraffic  $\hookrightarrow$  d3f:RemoteDatabaseQuery` quindi introduciamo `d3f:InboundInternetNetworkTraffic  $\sqsubseteq$  d3f:NetworkTraffic  $\hookrightarrow$  d3f:WebResourceAccess.`

Allo stesso modo, chiariamo che le query al db sono prodotte dalla Web Application.

`d3f:WebApplication d3f:produces d3f:RemoteDatabaseQuery`

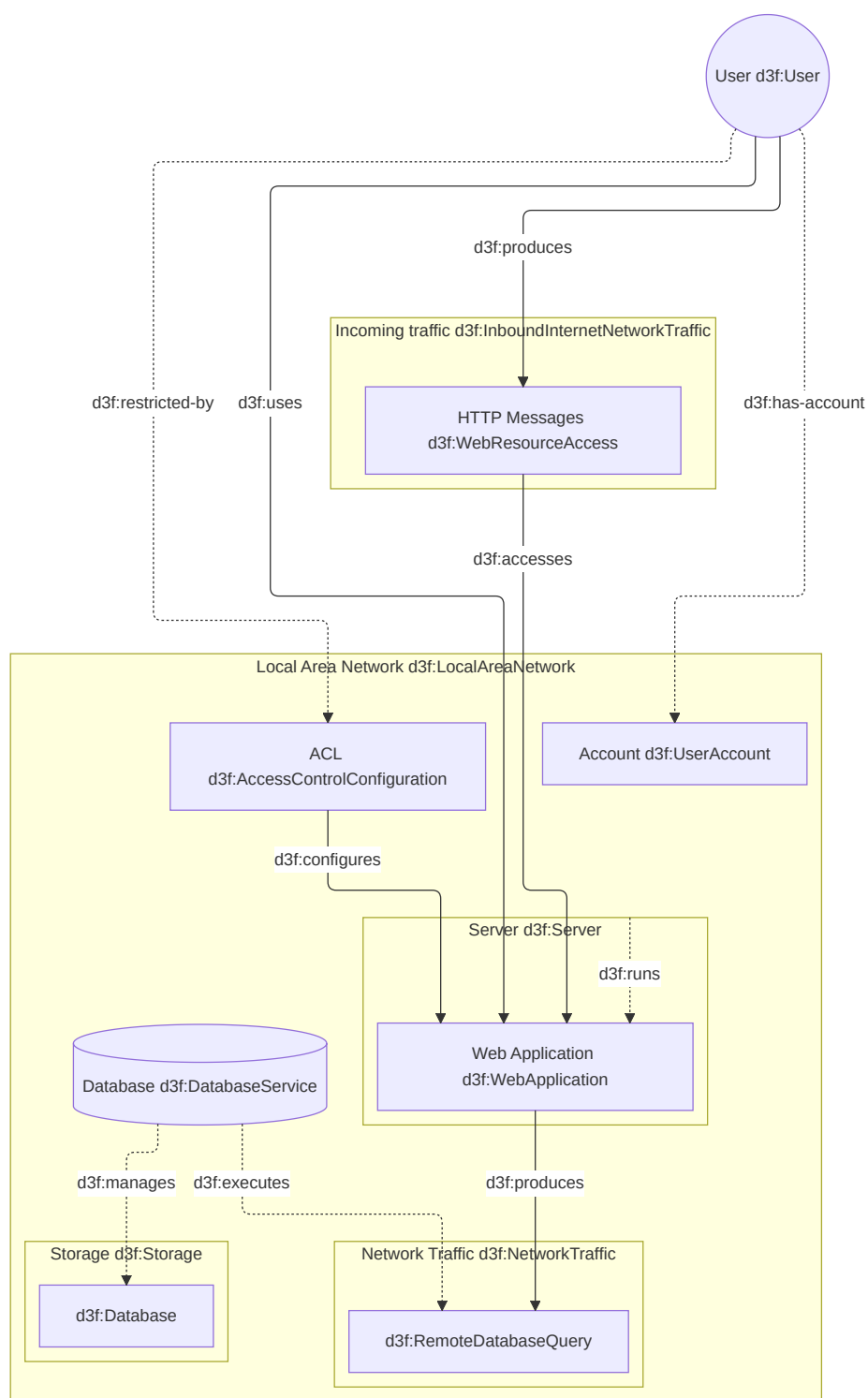


Figura 45: Risorse contenute nella Web Application

A questo punto abbiamo una rappresentazione iniziale dell'architettura con un dettaglio sufficiente per ragionare sulle tecniche offensive impattanti e sulle contromisure

da adottare.

Questa infrastruttura è il caso di studio che utilizzeremo per l'individuazione delle tecniche offensive.

4.3 Ulteriori Esempi

4.3.1 Architettura minimale

Iniziamo a modellare un'architettura informatica minimale con D3FEND, partendo da un utente che interagisce con una Web Application.

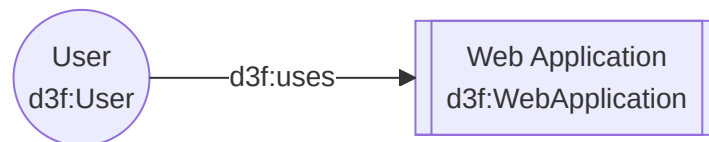


Figura 46: Un utente che accede a una Web Application

Se questo modello chiarisce bene la prospettiva funzionale, è invece troppo generico per ragionare intorno alle tecniche offensive applicabili: non compaiono riferimenti geografici, né informazioni sui flussi informativi.

Già a questo livello possiamo arricchire il modello dettagliando cosa accade intorno all'utente:

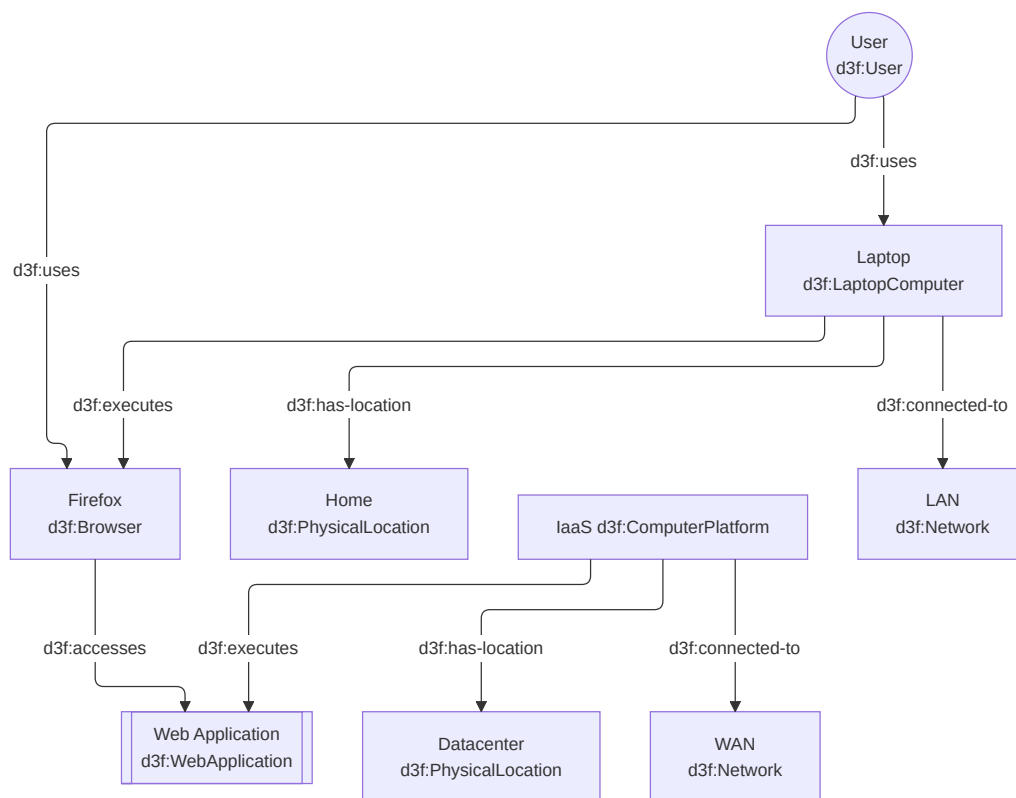


Figura 47: Un utente che accede a una Web Application, con dispositivi e collocazione fisica

Così come abbiamo indicato l'`rdf:type` nella label dei nodi, possiamo rappresentare il predicato

`d3f:contains`: `x contains y`: A core relation that holds between a whole `x` and its part `y`. Equivalent to relational concept 'has part' and thus transitive. (T)

con un sotto-diagramma.

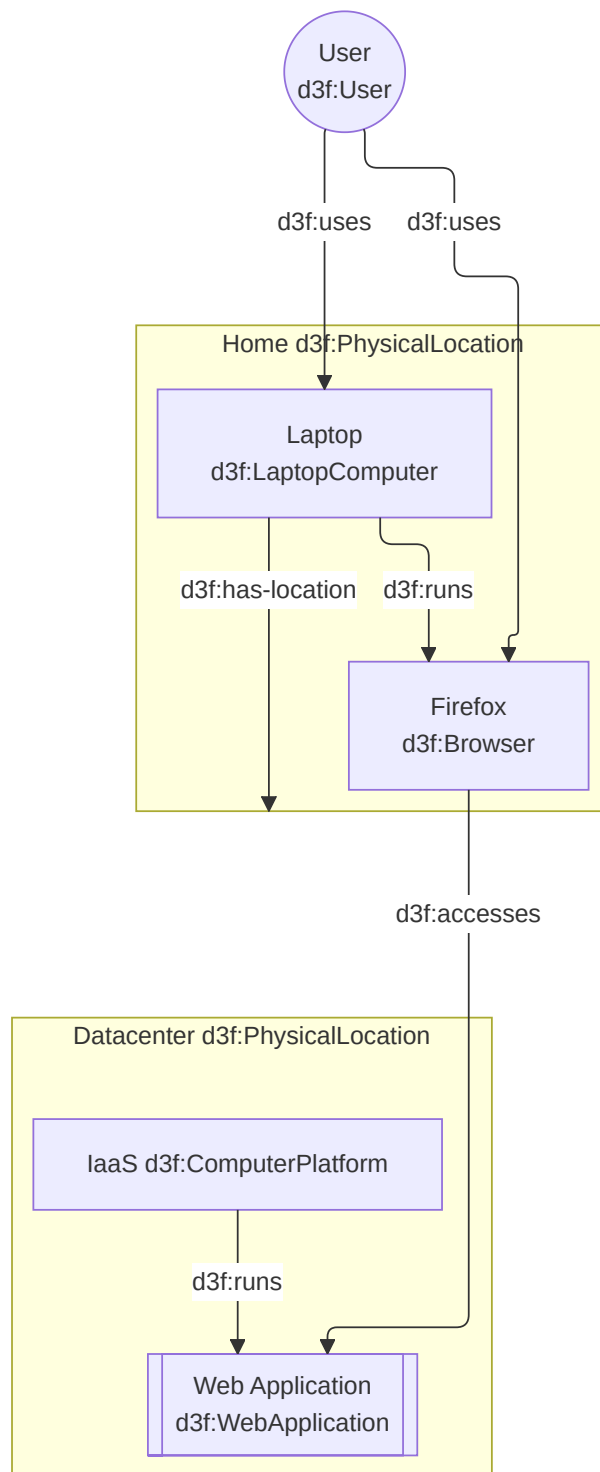


Figura 48: Un utente che accede a una Web Application, con la collocazione come sotto-diagramma

Possiamo dettagliare il diagramma precedente inserendo gli oggetti che rappresen-

tano le query e i risultati. Per direzionare i flussi informativi, utilizziamo proprietà inverse (e.g., `d3f : produced-by` è la relazione inversa di `d3f : produces`).

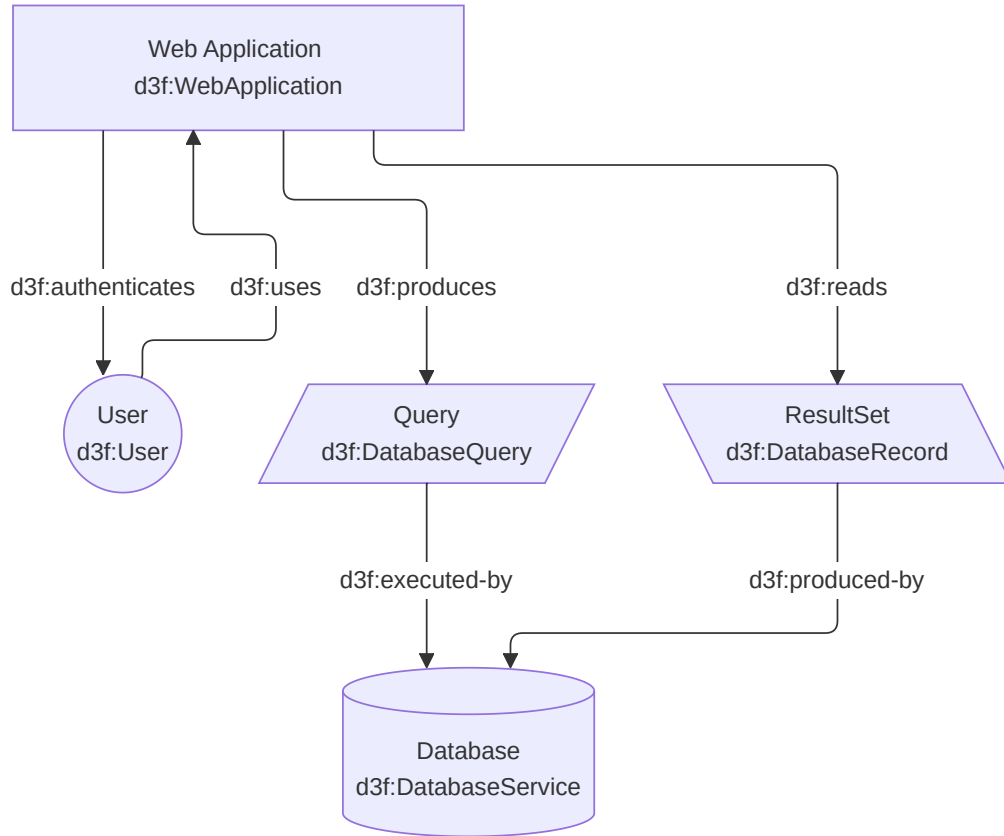


Figura 49: Una Web Application con una query al database

4.3.2 Individuare le tecniche offensive applicabili

Annotando i nodi di questo grafo usando classi e proprietà di D3FEND

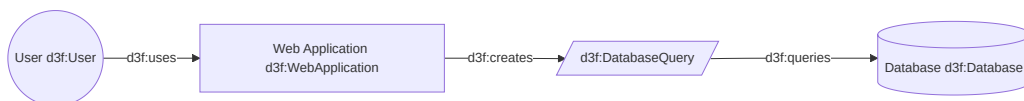


Figura 50: Web Application annotata con classi e proprietà D3FEND

possiamo interrogare il resto del KG per trovare le tecniche offensive applicabili, focalizzandoci su ogni singolo nodo.

```

SELECT DISTINCT ?a ?label ?attacks ?cursor
WHERE {

```

```

?s d3f:attack-id ?a ;
  rdfs:label ?label ;
  rdfs:subClassOf* d3f:OffensiveTechnique ;
  attacks ?cursor .
?cursor rdfs:subClassOf* d3f:DatabaseQuery .
}

```

?a	?label	?attacks	?cursor
T1190	Exploit Public-Facing Application	injects	DatabaseQuery

Allo stesso modo, possiamo interrogare il KG per individuare le tecniche difensive associate ai vari `d3f:Artifact`.

```

SELECT DISTINCT ?s ?protects ?cursor
WHERE {
  ?s rdfs:subClassOf* d3f:DefensiveTechnique ;
    protects ?cursor .
  ?cursor rdfs:subClassOf* d3f:DatabaseQuery .
}

```

?s	?protects	?cursor
DatabaseQueryStringAnalysis	analyzes	DatabaseQuery

Il dataset di D3FEND quindi è un ottimo punto di partenza per supportare la progettazione sicura.

Creando diagrammi di questo tipo, eventualmente componendoli, possiamo individuare le tecniche offensive applicabili già in fase di progettazione.

5 Implementazioni

Questi progetti sono stati sviluppati per mostrare come rendere accessibili le informazioni contenute nei knowledge graph collegati agli ambiti di cybersecurity.

5.1 D3FEND Language

D3FEND Language è un'estensione per Visual Studio Code che permette di accedere alle informazioni pubblicate dall'ontologia D3FEND direttamente dall'editor di codice.

Il progetto mostra un possibile uso delle informazioni presenti in un KG per creare una progettazione coerente e sicura di infrastrutture digitali.

L'estensione è sviluppata in TypeScript e utilizza le API di Visual Studio Code per fornire funzionalità come l'autocompletamento e la navigazione tra le entità e le loro relazioni.

`d3f:WebApplicationFirewall ->|d3f:mediates-access-to| d3f:WebApplication`

L'implementazione si basa su un parser che processa l'ontologia distribuita in formato RDF e genera un file JSON contenente

- classi;
- proprietà;
- relazioni esplicite e inferite.

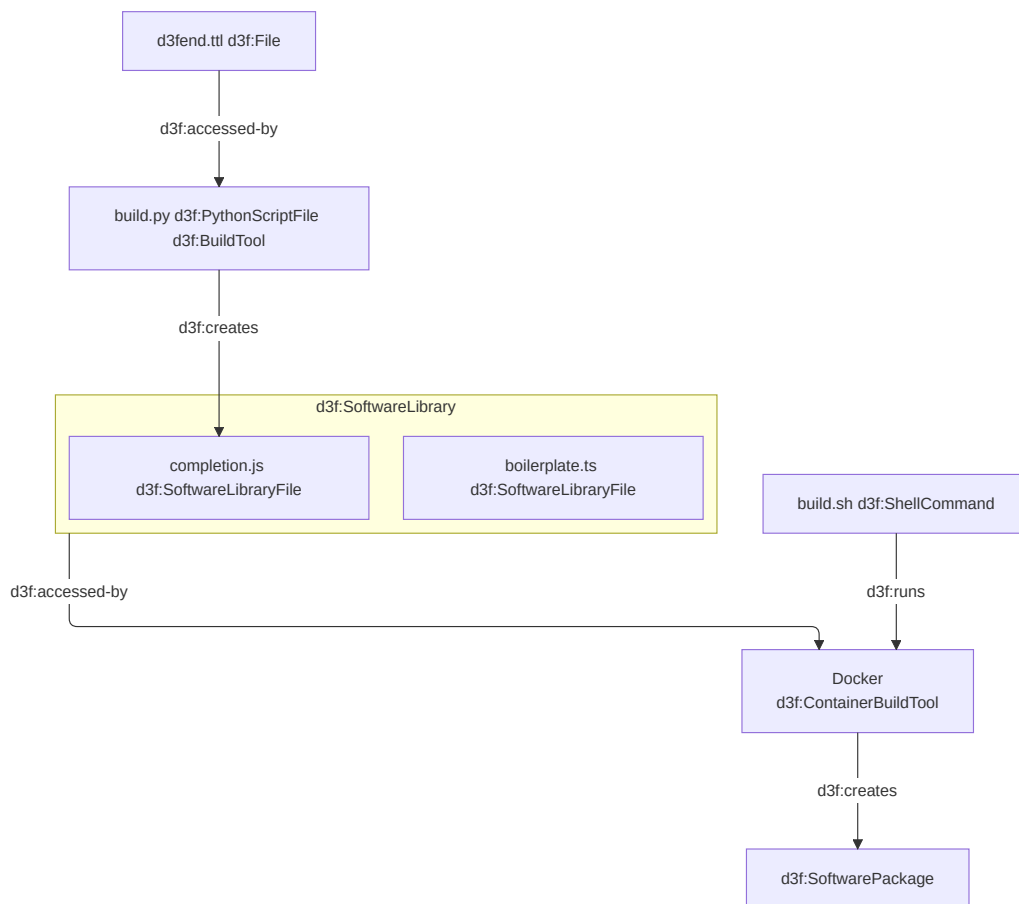


Figura 51: Architettura di D3FEND Language

Una volta installato, è possibile autocompletare le entità D3FEND all'interno di un file .md visualizzando i dettagli di tutte le possibili scelte.

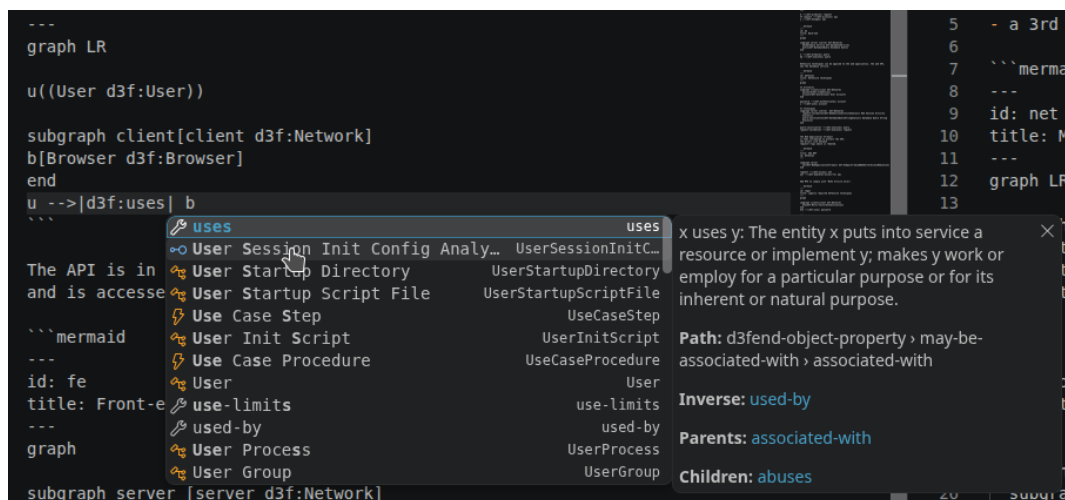


Figura 52: Autocompletamento di entità D3FEND

Quando il mouse è posizionato su un'entità D3FEND all'interno di un file .md, l'estensione mostra un tooltip con le informazioni principali, sia che l'entità sia all'interno di un blocco di codice che all'interno di un testo normale.

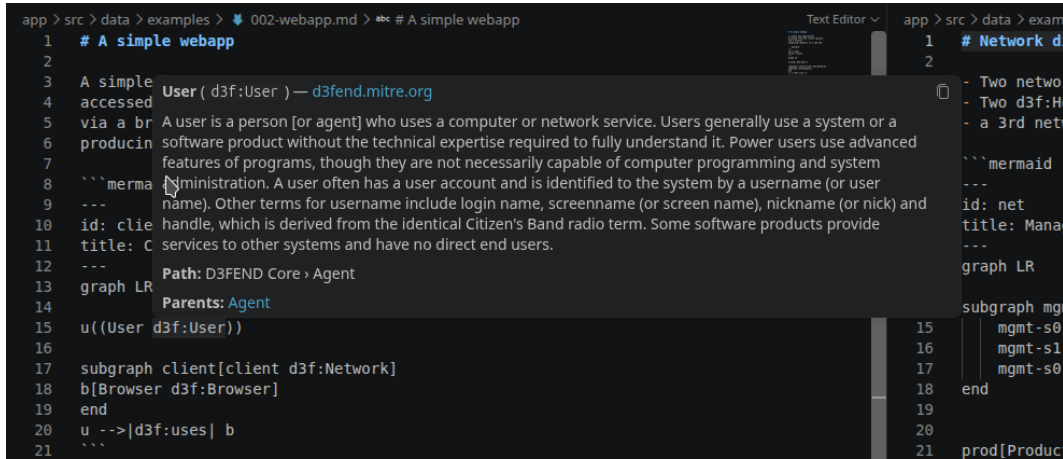


Figura 53: Tooltip su un'entità D3FEND

Cliccando su un'entità mostrata nel tooltip, si può navigare la sua gerarchia.

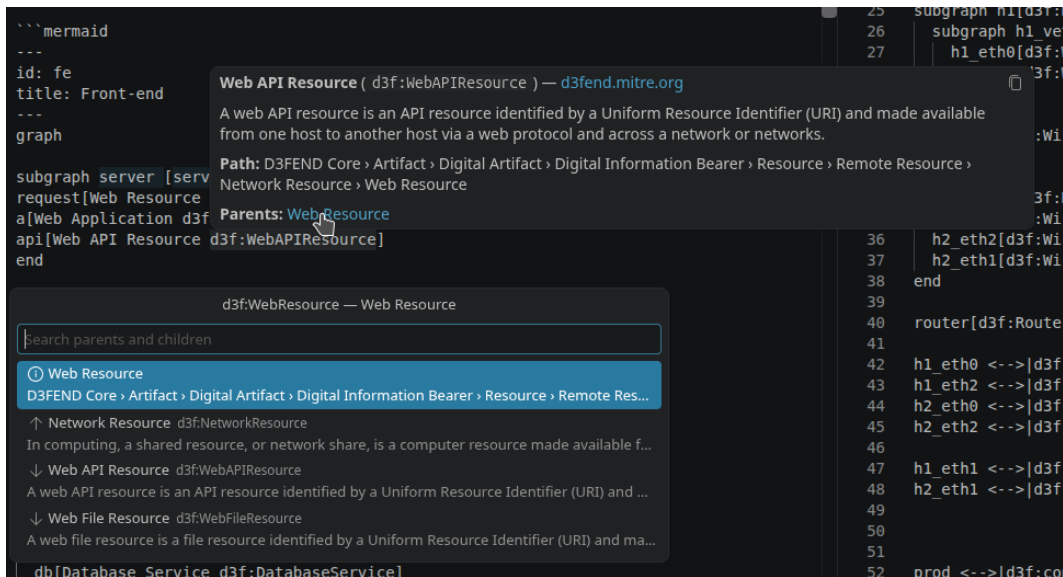


Figura 54: Gerarchia di un'entità D3FEND

5.2 GDPR Language

GDPR Language è un'estensione per Visual Studio Code che permette di accedere alle informazioni pubblicate dalle ontologie DPV ed EU GDPR direttamente dall'IDE.

Tecnicamente, si basa sulla stessa architettura di D3FEND Language, ma si interfaccia con dei KG giuridici.

```
---
title: GDPR Language
---
graph
Citizen[Citizen d3f:Person dpv:NaturalPerson eu-gdpr:DataSubject]
```

L'immagine in figura 55 mostra cosa accade posizionando il mouse su un'entità DPV o EU GDPR all'interno di un file .md.

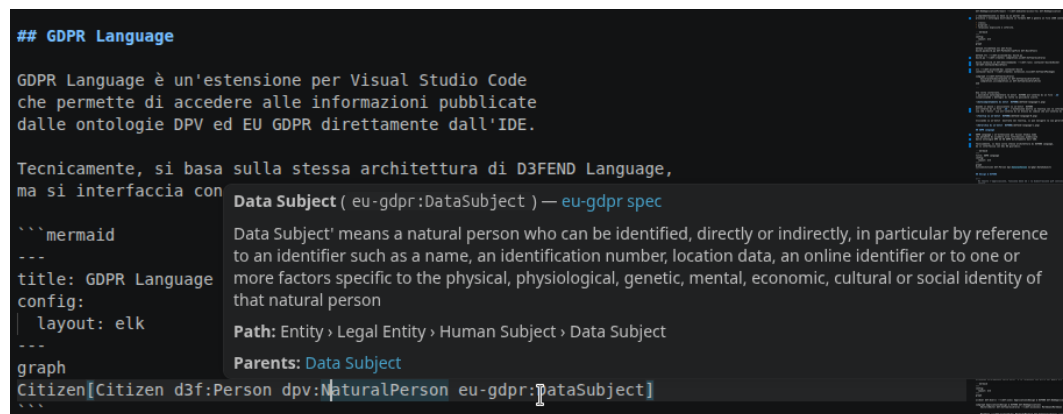


Figura 55: Hover su un'entità DPV o EU GDPR

5.3 Design & D3FEND

Design & D3FEND (Polli 2026) è una Single Page Application (SPA) che mostra come le informazioni contenute nell'ontologia D3FEND possano essere utilizzate per progettare infrastrutture digitali sicure.

Partendo da un file di progetto in formato markdown, e contenente una serie di diagrammi in formato Mermaid.js opportunamente annotati come mostrato in 4, l'applicazione:

- crea un KG contenente le entità e le loro relazioni;
- mostra un diagramma interattivo che permette di navigare tra le entità e le loro relazioni;

- permette di interrogare il KG via SPARQL per ottenere informazioni sulle entità e le relazioni con altri KG, quali DPV ed EU GDPR.

L'applicazione inoltre include strumenti di supporto all'editing, come l'auto-completamento e l'anteprima delle informazioni contenute nei KG di riferimento.

L'applicazione è accessibile all'indirizzo <https://ioggstream.github.io/d3sign/>, il codice sorgente è pubblicato con licenza Apache-2.0 su <https://github.com/ioggstream/d3sign> ed è archiviato con un riferimento permanente e citabile su Zenodo (Polli 2026).

Il primo prototipo di Design & D3FEND è stato sviluppato nel 2023, presentato a PyCon Italia (Polli 2023) e condiviso sul forum di D3FEND il 4 febbraio 2023¹⁵.

I diversi tentativi di migliorare il prototipo, principalmente basati sulla creazione di un'estensione omnicomprendensiva per Visual Studio Code, però si sono scontrati con delle difficoltà tecniche:

- Mermaid.js è un progetto molto attivo e complesso: modificare il modo di renderizzare i nodi ed i link richiedeva il rebuild periodico della libreria;
- la recente integrazione di Mermaid.js in Visual Studio Code ha ulteriormente complicato l'override delle funzioni di rendering;
- la possibilità di manipolare i diagrammi delegando esclusivamente a Mermaid.js il rendering definitivo limitava le possibilità di personalizzazione.

La versione attuale di Design & D3FEND quindi ha puntato su una separazione netta tra la fase di editing (basata su Mermaid.js) e quella di rendering, basata esclusivamente sulle informazioni presenti nel KG.

5.3.1 Architettura

L'applicazione è interamente client-side, sviluppata in JavaScript. Le librerie principali sono:

¹⁵Si veda l'annuncio sullo Slack Space di D3FEND <https://mitre-d3fend.slack.com/archives/C02QRK1RRUL/p1675517395384789>. Consultato il 26 agosto 2026.

- [Mermaid.js](#) per il rendering dei diagrammi;
- [Cytoscape.js](#) per il rendering del grafo RDF;
- [Codemirror](#) per l'editing del markdown e dei diagrammi, nonché per l'auto-completamento e la navigazione tra le entità;
- [n3.js](#) per la manipolazione di grafi RDF in formato Turtle e TriG.

L'architettura generale è la seguente:

1. l'utente usa l'Editor per scrivere un file di progetto in formato markdown, contenente una serie di diagrammi in formato Mermaid.js opportunamente annotati;
2. il parser trasforma il markdown in un grafo RDF, che viene visualizzato tramite Cytoscape.js;
3. l'interfaccia utente (User Interface, UI) basata su Cytoscape.js permette di navigare, selezionare e interrogare le entità e le loro relazioni;
4. inoltre l'utente può interrogare il grafo RDF tramite SPARQL, ottenendo informazioni sulle entità e le relazioni con altri KG, quali DPV ed EU GDPR - che vengono caricati su richiesta.

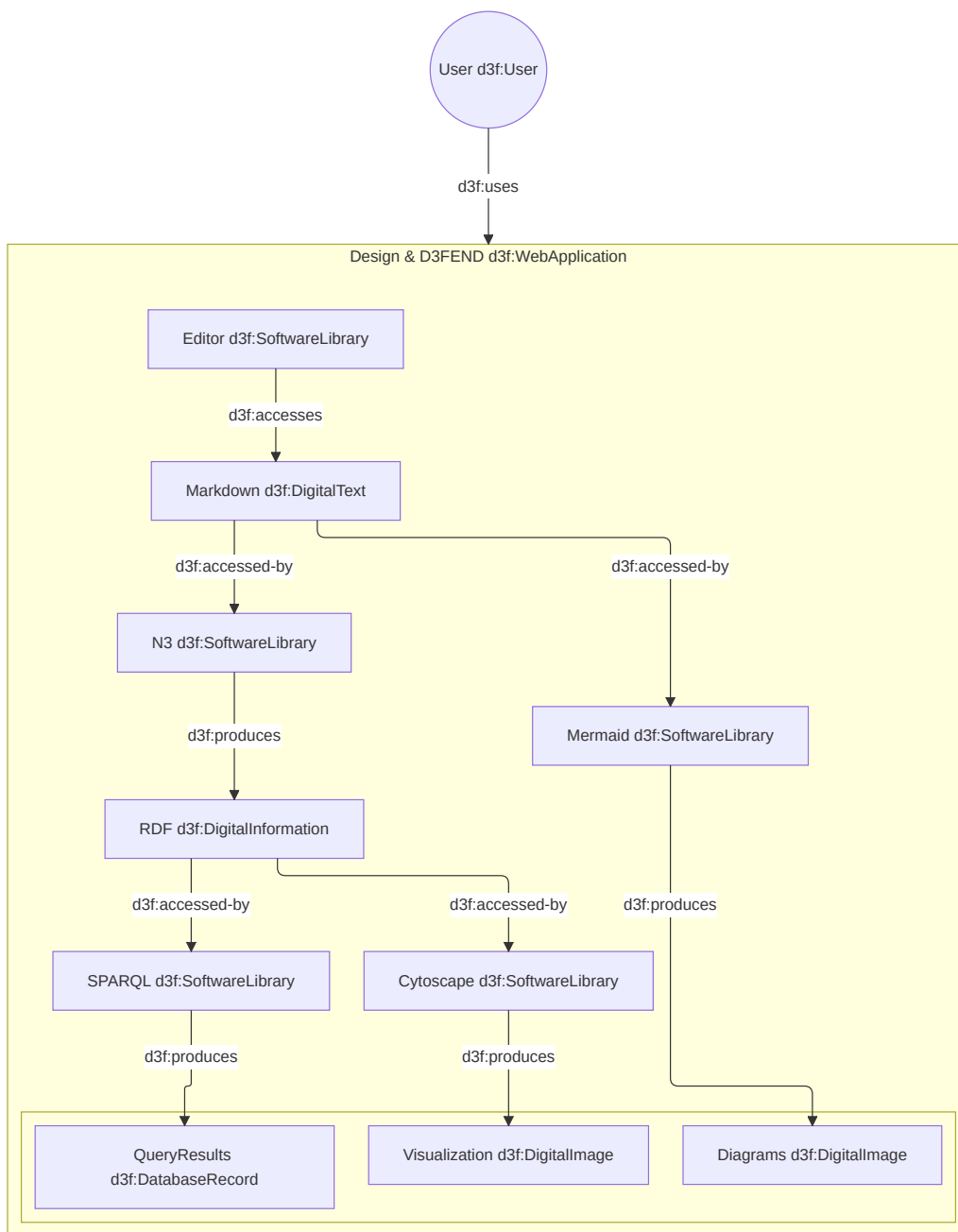


Figura 56: Architettura di Design & D3FEND

5.3.2 Interfaccia

L'interfaccia è essenziale, basata su due colonne:

- a sinistra l'editor dove scrivere il file di progetto;
- a destra la visualizzazione generata a partire dal grafo RDF.

L'editor supporta l'auto-completamento e la navigazione tra le entità e le loro relazioni, similmente a quanto avviene con D3FEND Language per Visual Studio Code 57.

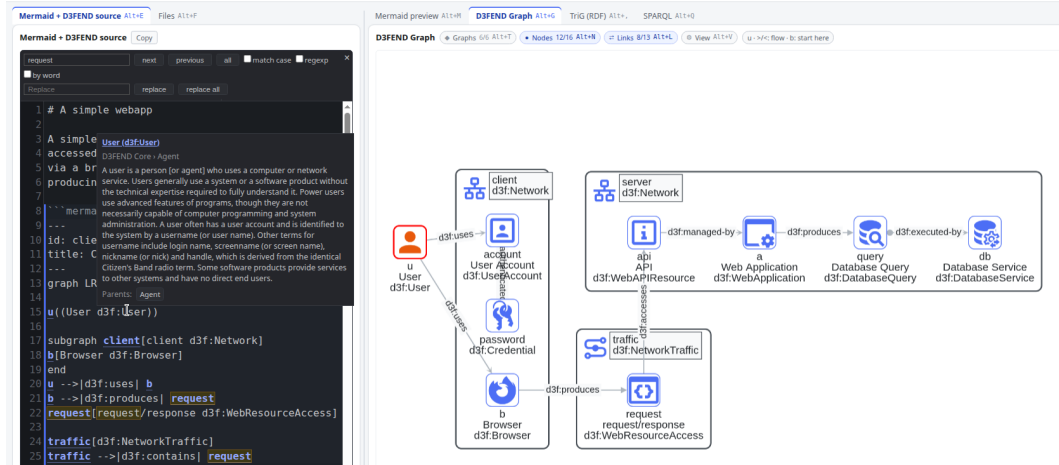


Figura 57: Interfaccia di Design & D3FEND

Figura 58 mostra come già in fase di progettazione è possibile - selezionando un nodo - vedere la descrizione della sua classe, nonché tutte le tecniche offensive e difensive associate ad esso ed alle sue super-classi, anche senza conoscere SPARQL. La visualizzazione può essere alleggerita nascondendo temporaneamente le etichette dei link.

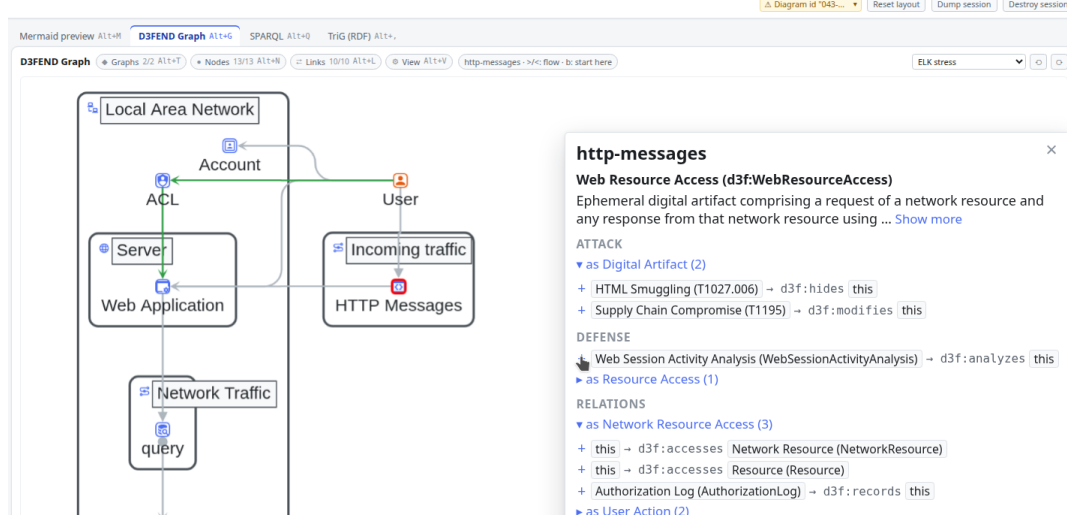


Figura 58: d3f:OffensiveTechnique e d3f:DefensiveTechnique associate ad un oggetto

Seguono alcune delle funzionalità attivabili tramite scorciatoie da tastiera.

Preview dei diagrammi Mermaid.js (Figura 59).

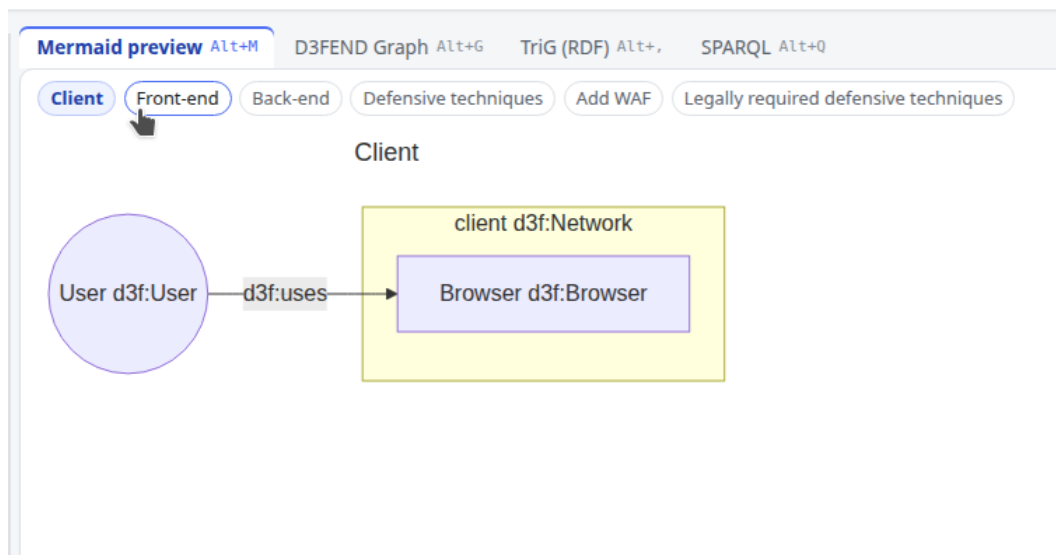


Figura 59: Interfaccia di Design & D3FEND - preview dei diagrammi Mermaid.js

Contenuto del KG generato a partire dal file di progetto (Figura 60).

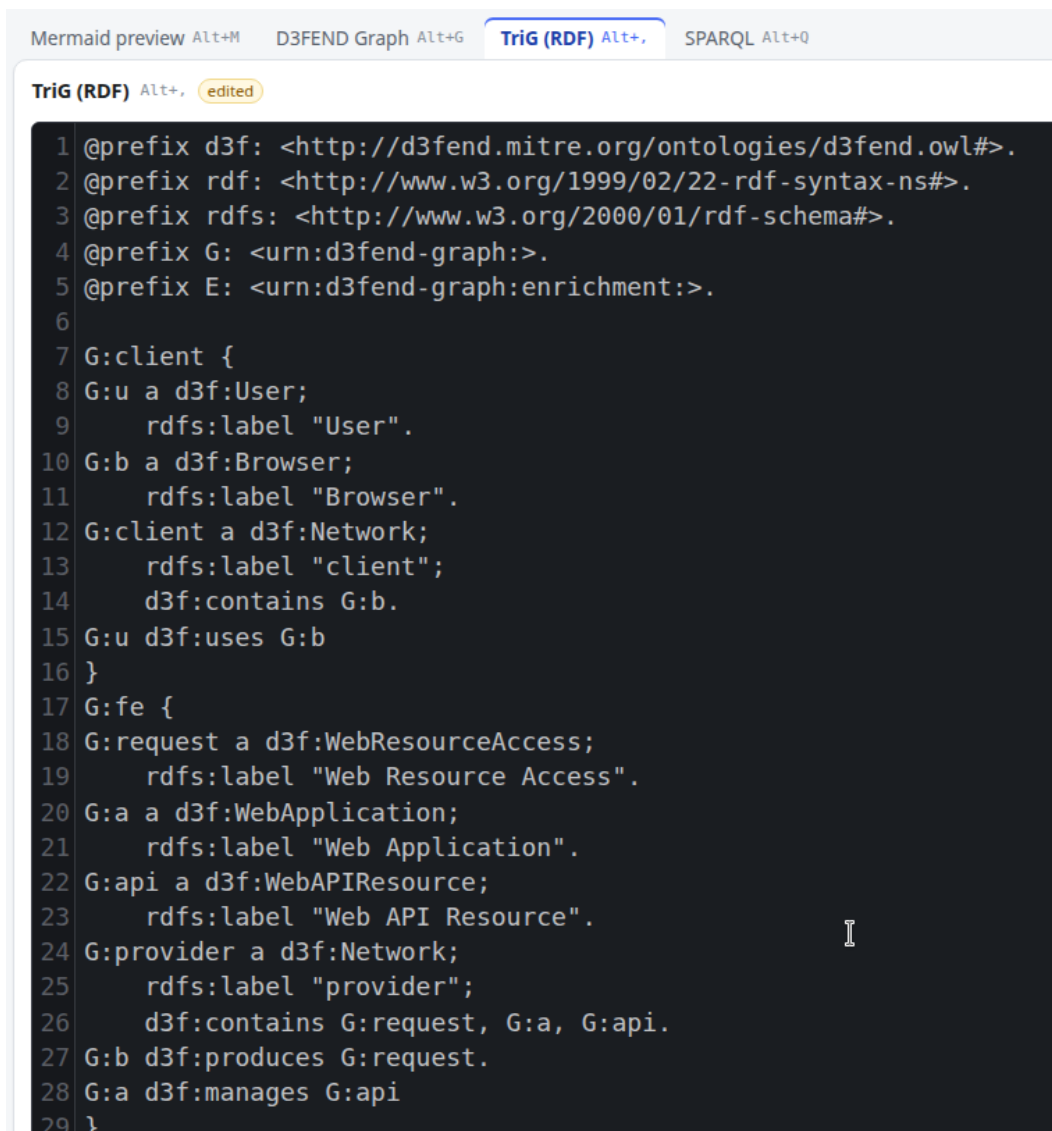


Figura 60: Interfaccia di Design & D3FEND - contenuto del KG generato

Interfaccia SPARQL per interrogare il grafo RDF generato a partire dal file di progetto, con incluse delle query utili (Figura 61).

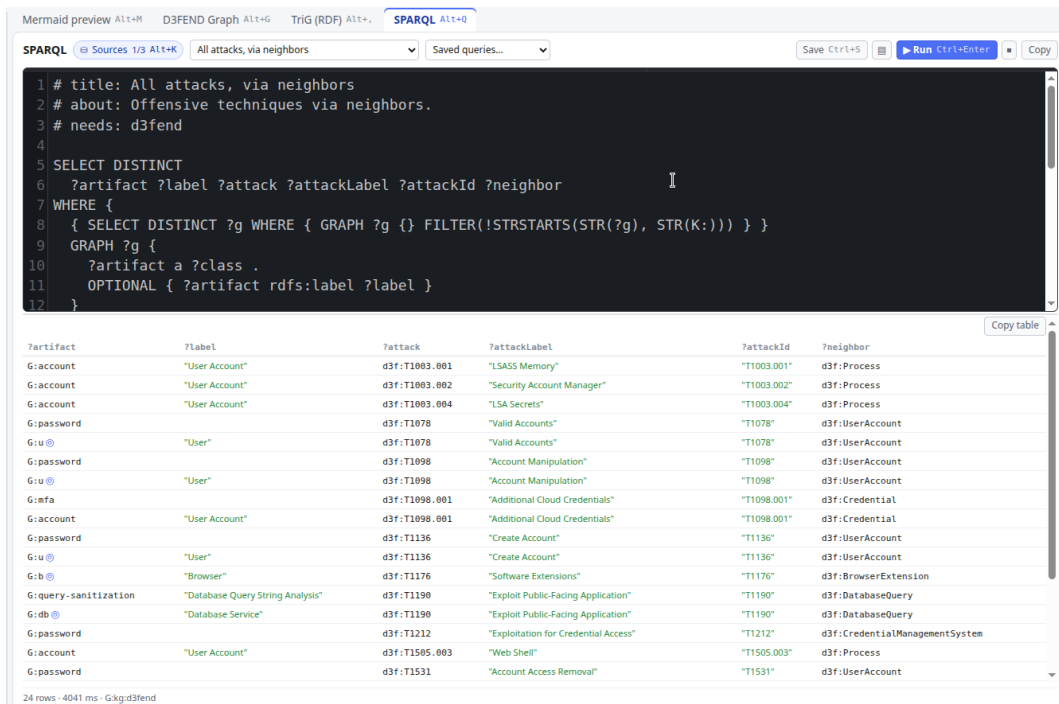


Figura 61: Interfaccia di Design & D3FEND - interfaccia SPARQL

5.4 Caso d'uso: individuare le tecniche ATT&CK applicabili ad un'architettura annotata con D3FEND

Per validare l'efficacia della metodologia proposta, consideriamo la web application minimale modellata in sez. 4.2, *Modellare un'infrastruttura: il caso d'uso di una web application*, e che riportiamo in formato testuale in sez. 5.4.7, *Mermaid.js Diagram*.

Vediamo quanti dei rischi menzionati nella OWASP API Security Top 10 2023 (OWASP Foundation 2023) vengono individuati in relazione agli artefatti digitali annotati estratti tramite il processo di modellazione descritto.

Abbiamo scelto la OWASP API Security Top 10 2023 per la validazione perché - avendo un numero minore di CWE e d3f:DigitalArtifact associati - c'è una minore probabilità di ottenere falsi positivi.

Ricordiamo che la OWASP API Security Top 10 2023 ha l'obiettivo di individuare i rischi critici per le API, e non di fornire un quadro completo dei rischi o delle

d3f:OffensiveTechnique applicabili.

Le query di validazione sono state tutte eseguite tramite Design & D3FEND, con l'obiettivo di permettere una più semplice revisione dell'architettura annotata, seguendo la catena Rischi $\xrightarrow{CWE}$ d3f:OffensiveTechnique $\rightarrow$ d3f:DigitalArtifact.

I grafi coinvolti sono:

- K:d3fend, contenente il KG di D3FEND;
- gli altri grafi, contenenti i dati specifici dell'architettura annotata da cui il frammento seguente estrae l'elenco dei d3f:DigitalArtifact.

```
===== NOTE: '\' line wrapping per RFC 8792 =====  
  
{ SELECT DISTINCT ?g WHERE { GRAPH ?g {} FILTER(!STRSTARTS(STR(?g), STR\  
(K:))) } }  
GRAPH ?g { [] a ?a }
```

Con un server SPARQL dedicato, o un graph database come Neo4j è possibile eseguire query più complesse per ottenere un quadro più ricco: ciò è particolarmente utile in contesti dove l'architettura è complessa, o a valle della fase progettuale.

Una versione successiva di Design & D3FEND potrebbe integrarsi con un database dedicato.

Anticipiamo qui che tutte le diverse strategie usate per individuare i rischi della OWASP API Security Top 10 2023 hanno portato agli stessi risultati finali:

- tutte le strategie hanno individuato i seguenti rischi della OWASP API Security Top 10 2023:
 - A01: Broken Object Level Authorization
 - A02: Broken Authentication
 - A04: Unrestricted Resource Consumption
 - A05: Broken Function Level Authorization
 - A10: Unsafe Consumption of APIs

5.4.1 Ricerca diretta degli artefatti impattati

Con questa query troviamo gli artefatti impattati tramite ricerca diretta.

```
===== NOTE: '\' line wrapping per RFC 8792 =====

prefix top10: <https://owasp.org/Top10/2025/>
prefix top10_api: <https://owasp.org/API-Security/editions/2023/en/>

SELECT DISTINCT
?owasp
(COUNT(distinct ?cwe) as ?ncwe)
(COUNT(distinct ?technique) as ?ntechnique)
?a
?cursor
WHERE {
  { SELECT DISTINCT ?g WHERE { GRAPH ?g {} FILTER(!STRSTARTS(STR(?g), S\
TR(K:))) } }
  GRAPH ?g { [] a ?a }

  ?o a top10_api:Risk ;
    rdfs:label ?owasp ;
    d3f:associated-with ?cwe .

  {
    ?capec a d3f:CommonAttackPattern ;
      d3f:related ?t ;
      d3f:related ?cwe .
      FILTER (STRSTARTS(STR(?t), STR(d3f:T)))

    ?t ?attacks ?a ;
      rdfs:label ?technique .
  }
  UNION
  {
    ?cwe (d3f:weakness-of|d3f:may-be-weakness-of) ?a .
  }
}
group by ?owasp ?a ?cursor
order by ?owasp
```

?owasp	?ncwe	?ntechnique	?a	?cursor
A01: Broken Object Level Authorization	1	1	d3f:NetworkTraffic	
A02: Broken Authentication	1	1	d3f:UserAccount	
A04: Unrestricted Resource Consumption	1	3	d3f:InboundInternetNetworkTraffic	
A05: Broken Function Level Authorization	1	1	d3f:NetworkTraffic	
A10: Unsafe Consumption of APIs	1	1	d3f:NetworkTraffic	

5.4.2 Ricerca per sottoclasse

```
===== NOTE: '\' line wrapping per RFC 8792 =====

prefix top10: <https://owasp.org/Top10/2025/>
prefix top10_api: <https://owasp.org/API-Security/editions/2023/en/>
```

```

SELECT DISTINCT
?owasp
(COUNT(distinct ?cwe) as ?ncwe)
(COUNT(distinct ?technique) as ?ntechnique)
?a
?cursor
WHERE {
  { SELECT DISTINCT ?g WHERE { GRAPH ?g {} FILTER(!STRSTARTS(STR(?g), S\
TR(K:))) } }
  GRAPH ?g { [] a ?a }

  ?o a top10_api:Risk ;
    rdfs:label ?owasp ;
    d3f:associated-with ?cwe .

  ?cursor rdfs:subClassOf* ?a .

  {
    ?capec a d3f:CommonAttackPattern ;
      d3f:related ?t ;
      d3f:related ?cwe .
      # ?t rdfs:subClassOf* d3f:OffensiveTechnique .
      FILTER (STRSTARTS(STR(?t), STR(d3f:T)))

      ?t ?attacks ?cursor ;
      rdfs:label ?technique .
    }
  UNION
  {
    ?cwe (d3f:weakness-of|d3f:may-be-weakness-of) ?cursor .
  }
}
group by ?owasp ?a ?cursor
order by ?owasp

```

owasp	ncwe	ntechnique	a	cursor
A01: Broken Object Level Authorization	1	1	d3f:NetworkTraffic	d3f:NetworkTraffic
A01: Broken Object Level Authorization	1	1	d3f:Credential	d3f:SessionCookie
A01: Broken Object Level Authorization	1	1	d3f:Credential	d3f:AccessToken
A01: Broken Object Level Authorization	1	1	d3f:NetworkTraffic	d3f:WebNetworkTraffic
A02: Broken Authentication	1	2	d3f:NetworkTraffic	d3f:IntranetAdministrativeNetworkTraffic
A02: Broken Authentication	1	1	d3f:UserAccount	d3f:UserAccount
A02: Broken Authentication	1	2	d3f:Credential	d3f:KerberosTicket
A02: Broken Authentication	1	2	d3f:Credential	d3f:Password
A04: Unrestricted Resource Consumption	1	3	d3f:NetworkTraffic	d3f:InboundInternetNetworkTraffic
A04: Unrestricted Resource Consumption	1	3	d3f:InboundInternetNetworkTraffic	d3f:InboundInternetNetworkTraffic
A05: Broken Function Level Authorization	1	1	d3f:NetworkTraffic	d3f:WebNetworkTraffic
A05: Broken Function Level Authorization	1	1	d3f:NetworkTraffic	d3f:NetworkTraffic
A05: Broken Function Level Authorization	1	1	d3f:Credential	d3f:SessionCookie
A05: Broken Function Level Authorization	1	1	d3f:Credential	d3f:AccessToken
A10: Unsafe Consumption of APIs	1	1	d3f:NetworkTraffic	d3f:NetworkTraffic

owasp	ncwe	ntechnique	a	cursor
A10: Unsafe Consumption of APIs	1	1	d3f:NetworkTraffic	d3f:WebNetworkTraffic
A10: Unsafe Consumption of APIs	2	2	d3f:Credential	d3f:SessionCookie
A10: Unsafe Consumption of APIs	1	1	d3f:Credential	d3f:AccessToken

Anche qui troviamo gli stessi rischi individuati in sez. 5.4.1, *Ricerca diretta degli artefatti impattati*, ma con una grana più fine, che permette sia di dettagliare meglio l'architettura che di individuare ulteriori tipi di attacco (e.g., usando una classe più specifica di `d3f:Credential` quale `d3f:AccessToken`).

5.4.3 Ricerca per superclasse

Una ricerca per superclasse conferma il quadro dei rischi individuati:

===== NOTE: '\' line wrapping per RFC 8792 =====

```

prefix top10: <https://owasp.org/Top10/2025/>
prefix top10_api: <https://owasp.org/API-Security/editions/2023/en/>

SELECT DISTINCT
?owasp
(COUNT(distinct ?cwe) as ?ncwe)
(COUNT(distinct ?technique) as ?ntechique)
?a
?cursor
WHERE {
  { SELECT DISTINCT ?g WHERE { GRAPH ?g {} FILTER(!STRSTARTS(STR(?g), S\
TR(K:))) } }
  GRAPH ?g { [] a ?a }

  ?o a top10_api:Risk ;
    rdfs:label ?owasp ;
    d3f:associated-with ?cwe .
  ?a rdfs:subClassOf* ?cursor .

  {
    ?capec a d3f:CommonAttackPattern ;
      d3f:related ?t ;
      d3f:related ?cwe .
    # ?t rdfs:subClassOf* d3f:OffensiveTechnique .
    FILTER (STRSTARTS(STR(?t), STR(d3f:T)))

    ?t ?attacks ?cursor ;
      rdfs:label ?technique .
  }
  UNION
  {
    ?cwe (d3f:weakness-of|d3f:may-be-weakness-of) ?cursor .
  }
}
group by ?owasp ?a ?cursor
order by ?owasp

```

?owasp	?ncwe	?ntechnique	?a	?cursor
A01: Broken Object Level Authorization	1	1	d3f:NetworkTraffic	d3f:NetworkTraffic
A01: Broken Object Level Authorization	1	1	d3f:InboundInternetNetworkTraffic	d3f:NetworkTraffic
A01: Broken Object Level Authorization	1	2	d3f:WebApplication	d3f:ServiceApplication
A02: Broken Authentication	1	1	d3f:UserAccount	d3f:UserAccount
A04: Unrestricted Resource Consumption	1	3	d3f:InboundInternetNetworkTraffic	d3f:InboundInternetNetworkTraffic
A05: Broken Function Level Authorization	1	1	d3f:NetworkTraffic	d3f:NetworkTraffic
A05: Broken Function Level Authorization	1	1	d3f:InboundInternetNetworkTraffic	d3f:NetworkTraffic
A05: Broken Function Level Authorization	1	2	d3f:WebApplication	d3f:ServiceApplication
A10: Unsafe Consumption of APIs	1	1	d3f:InboundInternetNetworkTraffic	d3f:NetworkTraffic
A10: Unsafe Consumption of APIs	1	1	d3f:NetworkTraffic	d3f:NetworkTraffic

La ricerca per superclasse conferma i risultati ottenuti con le due strategie precedenti, ma con un dettaglio minore di tecniche e artefatti impattati, dovuto ad un livello di astrazione maggiore.

5.4.4 Ricerca per intorno di classe

Anche usando l'intorno di classe otteniamo risultati simili.

===== NOTE: '\ ' line wrapping per RFC 8792 =====

```

prefix top10: <https://owasp.org/Top10/2025/>
prefix top10_api: <https://owasp.org/API-Security/editions/2023/en/>

SELECT DISTINCT
?owasp
(COUNT(distinct ?cwe) as ?ncwe)
(COUNT(distinct ?technique) as ?ntechnique)
?a
?cursor
WHERE {
  { SELECT DISTINCT ?g WHERE { GRAPH ?g {} FILTER(!STRSTARTS(STR(?g), S\
TR(K:))) } }
  GRAPH ?g { [] a ?a }

  ?o a top10_api:Risk ;
    rdfs:label ?owasp ;
    d3f:associated-with ?cwe .

  { ?a ?isCloseTo ?cursor . }
UNION
{ ?cursor ?isCloseTo ?a . }

?isCloseTo rdfs:subPropertyOf+ d3f:d3fend-object-property .
?cursor rdfs:subClassOf* d3f:DigitalArtifact .

{

```

```

?capec a d3f:CommonAttackPattern ;
d3f:related ?t ;
d3f:related ?cwe .
# ?t rdfs:subClassOf* d3f:OffensiveTechnique .
FILTER (STRSTARTS(STR(?t), STR(d3f:T)))

?t ?attacks ?cursor ;
rdfs:label ?technique .
}
UNION
{
?cwe (d3f:weakness-of|d3f:may-be-weakness-of) ?cursor .
}
}
group by ?owasp ?a ?cursor
order by ?owasp

```

?owasp	?ncwe	?ntechnique	?a	?cursor
A01: Broken Object Level Authorization	1	1	d3f:InboundInternetNetworkTraffic	d3f:NetworkTraffic
A02: Broken Authentication	1	1	d3f:AuthenticationFunction	d3f:UserAccount
A02: Broken Authentication	1	1	d3f:User	d3f:UserAccount
A02: Broken Authentication	1	1	d3f:Credential	d3f:UserAccount
A04: Unrestricted Resource Consumption	9	9	d3f:NetworkTraffic	d3f:InboundInternetNetworkTraffic
A05: Broken Function Level Authorization	1	1	d3f:InboundInternetNetworkTraffic	d3f:NetworkTraffic
A10: Unsafe Consumption of APIs	1	1	d3f:InboundInternetNetworkTraffic	d3f:NetworkTraffic

I risultati ottenuti con la ricerca per intorno di classe sono in linea con gli altri risultati, indicando che un'ulteriore estensione all'intorno delle classi individuate non porta a scoprire ulteriori rischi della OWASP API Security Top 10 2023.

5.4.5 Ricerca delle misure di sicurezza

Le d3f:DefensiveTechnique possono essere ricercate nel grafo verificandone l'associazione con le classi degli oggetti dell'architettura. Diamo qui un esempio per ricerca diretta, né superclassi né sottoclassi - ampiamente sufficiente a mostrare la ricchezza di D3FEND in ambito difensivo.

```

===== NOTE: '\' line wrapping per RFC 8792 =====

SELECT DISTINCT

?class ?measure
WHERE {
  { SELECT DISTINCT ?g WHERE { GRAPH ?g {} FILTER(!STRSTARTS(STR(?g), S\
TR(K:))) } }

```

```

GRAPH ?g {
  [] a ?class .
}

?measure ?verb ?class ;
      rdfs:subClassOf* d3f:DefensiveTechnique .
?verb rdfs:subPropertyOf* d3f:d3fend-object-property .
}
ORDER BY ?class ?measure

```

?measure	?class
d3f:InboundSessionVolumeAnalysis	d3f:InboundInternetNetworkTraffic
d3f:UserGeolocationLogonPatternAnalysis	d3f:NetworkTraffic
d3f:RemoteTerminalSessionDetection	d3f:NetworkTraffic
d3f:ProtocolMetadataAnomalyDetection	d3f:NetworkTraffic
d3f:PerHostDownload-UploadRatioAnalysis	d3f:NetworkTraffic
d3f:NetworkTrafficSignatureAnalysis	d3f:NetworkTraffic
d3f:NetworkTrafficFiltering	d3f:NetworkTraffic
d3f:NetworkTrafficCommunityDeviation	d3f:NetworkTraffic
d3f:Client-serverPayloadProfiling	d3f:NetworkTraffic
d3f:ApplicationProtocolCommandAnalysis	d3f:NetworkTraffic
d3f:LANAccessMediation	d3f:LocalAreaNetwork
d3f:ConnectedHoneynet	d3f:LocalAreaNetwork
d3f:DiskEncryption	d3f:Storage
d3f:WebSessionActivityAnalysis	d3f:WebResourceAccess
d3f:RestoreDatabase	d3f:Database
d3f:DataInventory	d3f:Database
d3f:NetworkTrafficPolicyMapping	d3f:AccessControlConfiguration
d3f:AccessModeling	d3f:AccessControlConfiguration
d3f:UserAccountPermissions	d3f:UserAccount
d3f:UnlockAccount	d3f:UserAccount
d3f:RestoreUserAccountAccess	d3f:UserAccount
d3f:ChangeDefaultPassword	d3f:UserAccount
d3f:AgentAuthentication	d3f:UserAccount
d3f:AccountLocking	d3f:UserAccount
d3f:AccessModeling	d3f:UserAccount
d3f:DecoyPersona	d3f:User

5.4.6 Conclusioni

L'approccio descritto in questo documento, ed implementato in Design & D3FEND, permette di individuare 5 su 7 dei rischi della OWASP API Security Top 10 2023 attualmente indicizzabili tramite i KG di D3FEND e CAPEC.

Restano esclusi:

- i rischi per cui non è definito un percorso che passi dalle CWE e da CAPEC;
- i rischi A07 - *Server Side Request Forgery* e A08 - *Security Misconfiguration*, per i quali riportiamo i percorsi verso i d3f:DigitalArtifact raggiungibili, che non sono attualmente diagrammati nell'architettura annotata. Una

rappresentazione più completa dell'architettura potrebbe comunque includerli (e.g., dettagliando l'implementazione della `d3f:WebApplication` o delle varie `d3f:WebAPIResource`).

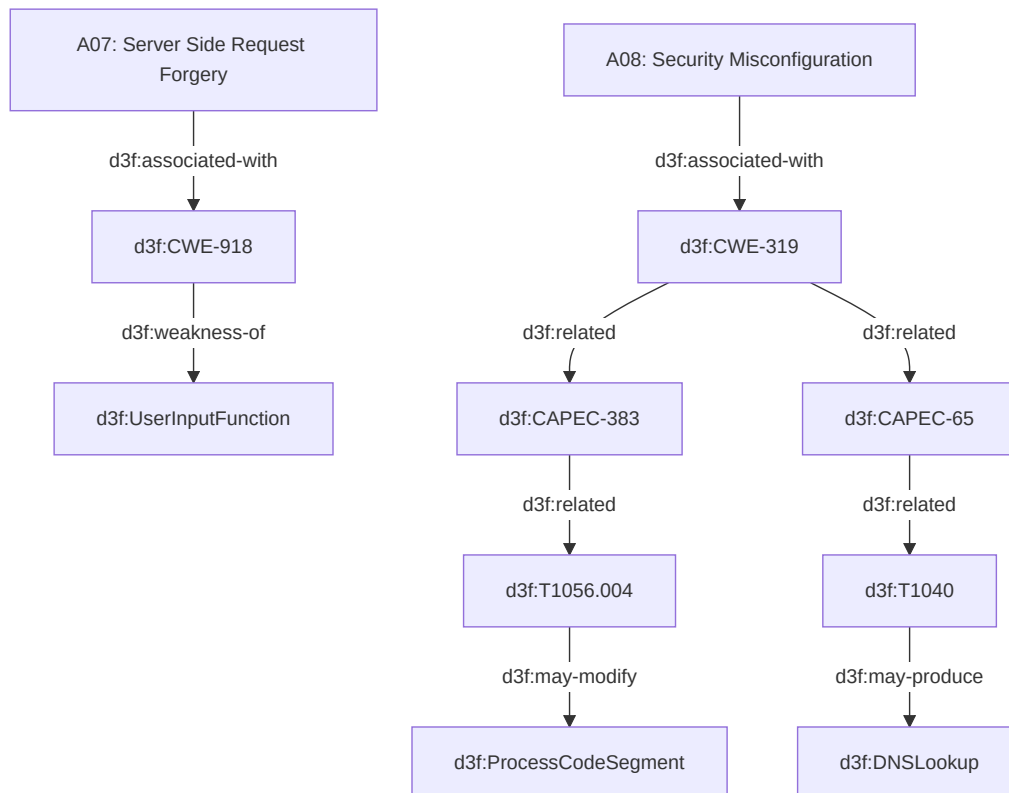


Figura 62: Percorsi verso i rischi A07 e A08 della OWASP API Security Top 10 2023

5.4.7 Mermaid.js Diagram

Di seguito è riportato il diagramma Mermaid.js corrispondente al caso d'uso mini- male:

```

===== NOTE: '\' line wrapping per RFC 8792 =====

---
id: 043-webapp-minimal-containing-resources
title: Risorse contenute nella Web Application
config:
  layout: elk
---
graph
u((User d3f:User))

```

```

A[Web Application d3f:WebApplication]
u -->|d3f:uses| A
A -->|d3f:produces| query

dbms[(Database d3f:DatabaseService)]

%% Intorno di d3f:User
u -. ->|d3f:has-account| account

acl[ACL d3f:AccessControlConfiguration]
u -. ->|d3f:restricted-by| acl

%% Intorno di d3f:WebApplication
server[d3f:WebApplicationServer]
server -. ->|d3f:runs| A

%% Intorno di d3f:DatabaseService
query[d3f:RemoteDatabaseQuery]
dbms -. ->|d3f:executes| query

subgraph storage[Storage d3f:Storage]
db[d3f:Database]
end
dbms -. ->|d3f:manages| db

u -->|d3f:produces| http-messages[HTTP Messages d3f:WebResourceAccess] \
-->|d3f:accesses| A

acl -->|d3f:configures| A

subgraph LAN[Local Area Network d3f:LocalAreaNetwork]
  account["Account d3f:UserAccount"]
  server
  dbms
  storage
  acl
  intranet-traffic
end

subgraph intranet-traffic[Network Traffic d3f:NetworkTraffic]
  query
end

subgraph server[Server d3f:Server]
  A
end

subgraph inbound-traffic[Incoming traffic d3f:InboundInternetNetworkTra\
ffic]
  http-messages
end

```

5.4.8 Knowledge Graph

Di seguito il KG generato a partire dal caso d'uso minimale modellato in sez. 4.2,
Modellare un'infrastruttura: il caso d'uso di una web application.

===== NOTE: '\ ' line wrapping per RFC 8792 =====

@prefix d3f: <http://d3fend.mitre.org/ontologies/d3fend.owl#>.
 @prefix rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#>.

```

@prefix rdfs: <http://www.w3.org/2000/01/rdf-schema#>.
@prefix G: <urn:d3fend-graph:>.
@prefix E: <urn:d3fend-graph:enrichment:>.

G:043-webapp-minimal-containing-resources {
  G:u a d3f:User;
      rdfs:label "User".
  G:A a d3f:WebApplication;
      rdfs:label "Web Application".
  G:query a d3f:RemoteDatabaseQuery.
  G:dbms a d3f:DatabaseService;
      rdfs:label "Database".
  G:account a d3f:UserAccount;
      rdfs:label "Account".
  G:acl a d3f:AccessControlConfiguration;
      rdfs:label "ACL".
  G:server a d3f:WebApplicationServer.
  G:db a d3f:Database.
  G:http-messages a d3f:WebResourceAccess;
      rdfs:label "HTTP Messages".
  G:storage a d3f:Storage;
      rdfs:label "Storage".
  G:LAN a d3f:LocalAreaNetwork;
      rdfs:label "Local Area Network".
  G:intranet-traffic a d3f:NetworkTraffic;
      rdfs:label "Network Traffic".
  G:server a d3f:Server;
      rdfs:label "Server".
  G:inbound-traffic a d3f:InboundInternetNetworkTraffic;
      rdfs:label "Incoming traffic".
  G:storage d3f:contains G:db.
  G:LAN d3f:contains G:dbms, G:account, G:acl, G:server, G:storage, G:i\
ntranet-traffic.
  G:intranet-traffic d3f:contains G:query.
  G:server d3f:contains G:A.
  G:inbound-traffic d3f:contains G:http-messages.
  G:u d3f:uses G:A.
  G:A d3f:produces G:query.
  G:u d3f:has-account G:account;
      d3f:restricted-by G:acl.
  G:server d3f:runs G:A.
  G:dbms d3f:executes G:query;
      d3f:manages G:db.
  G:u d3f:produces G:http-messages.
  G:http-messages d3f:accesses G:A.
  G:acl d3f:configures G:A
}

```

6 Conclusioni e sviluppi futuri

Questo elaborato ha esplorato la “democratizzazione” dell’uso dei Cybersecurity Knowledge Graph (KG) illustrando come sia possibile utilizzare le informazioni contenute in D3FEND anche senza una solida conoscenza del Web Semantico, tramite delle applicazioni user-friendly che supportano la progettazione sicura delle infrastrutture e delle applicazioni IT, rendendo disponibili in tempo reale le informazioni contenute nei KG collegati agli ambiti di cybersecurity.

Tali strumenti richiedono semplicemente la conoscenza di Markdown e Mermaid.js - tool ampiamente diffusi e consolidati tra sviluppatori ed architetti IT: gli utenti possono quindi partire dalla loro base documentale per aggiungere informazioni di sicurezza, ricercandole tramite le funzionalità di autocompletamento e navigazione.

L’interfaccia utente di “Design & D3FEND” (Polli 2026) permette di visualizzare in modo immediato sia i vari livelli di dettaglio dei diagrammi architetturali, “ripiegando” o “espandendo” le entità principali, che i diversi tipi di flussi informativi, creando quindi le varie viste a partire da un unico set informativo.

Le sfide principali sono state quelle relative alla modellazione delle infrastrutture IT e dell’individuazione delle relazioni tra le entità, riconoscendo comunque che qualunque rappresentazione è soggetta a delle scelte.

I criteri valutati sono i seguenti:

1. Criterio 1 (notazione testuale annotata, traducibile in un grafo RDF interrogabile): raggiunto; vedi sez. 4.1, *Usare la sintassi Mermaid Flowchart per rappresentare i grafi architetturali* e sez. 4.2, *Modellare un’infrastruttura: il caso d’uso di una web application*.
2. Criterio 2 (individuare le `d3f:DefensiveTechnique` applicabili, in modalità assistita e via SPARQL): raggiunto; vedi sez. 5.4.5, *Ricerca delle misure di sicurezza*.
3. Criterio 3 (derivare le `d3f:OffensiveTechnique` senza intervento manua-

le): raggiunto; con l'individuazione - mostrata in sez. 5.4, *Caso d'uso: individuare le tecniche ATT&CK applicabili ad un'architettura annotata con D3FEND* - di 5 rischi OWASP API Security Top 10 su 7 raggiungibili tramite una mappatura delle CWE associate verso D3FEND.

6.1 Valutazione critica

I risultati relativi ai rischi OWASP API Security Top 10 mostrano come i diversi approcci di ricerca delle tecniche offensive applicabili convergono: nella sua semplicità quindi, l'approccio mostrato risulta valido e coerente. Questo avviene in particolare quando il grafo è adeguatamente "chiuso" rispetto agli intorni degli oggetti di base.

La copertura dei rischi OWASP dipende interamente dai dataset CAPEC e D3FEND e dalla catena di mappatura verso i `d3f:DigitalArtifact`. L'impossibilità di mappare il rischio A06 della OWASP API Security Top 10 è un limite strutturale derivante dall'assenza di una CWE associata, e collegato al concetto - tecnicamente vago - di "Sensitive Business Flows". In questi termini ci potrebbe venire in aiuto un'integrazione tra OWASP e DPV, dove si potrebbe creare un collegamento - comunque arbitrario - con `dpv:SensitiveData`.

6.2 Limiti

L'individuazione di `d3f:OffensiveTechnique` tramite SPARQL si presta alla presenza di falsi positivi. Questo non è un problema: le tecniche individuate sono un elenco di candidati da valutare, non un'analisi finale. L'importante è poter avere quest'elenco in tempo reale già durante la fase di progettazione.

La validazione mostrata si basa su un caso d'uso minimale: questa è stata una scelta consapevole, volta a limitare il numero di CWE e `d3f:OffensiveTechnique` individuabili. Un caso più complesso avrebbe portato ad una copertura maggiore dei rischi OWASP API Security Top 10. D'altro canto avrebbe richiesto degli skill più avanzati da parte degli annotatori, riducendo così la platea target.

I risultati in termini di copertura dipendono comunque dalla capacità di modellare l'architettura: questo è chiaramente illustrato nel processo di annotazione del caso d'uso visto in sez. 4.2, *Modellare un'infrastruttura: il caso d'uso di una web application*. Le funzionalità di Design & D3FEND supportano quindi un processo di modellazione strutturato, che passa per l'individuazione degli intorni e delle gerarchie degli oggetti; senza di esso i risultati rischierebbero invece di essere modesti.

E' stato deciso invece di lasciare fuori la complessità legata alla specializzazione delle relazioni sia per la mancanza di librerie consolidate, sia per l'analisi relativa alle interazioni tra LPG e RDF 1.2.

Gli sviluppi futuri si possono concentrare su diversi aspetti, sia implementativi che di ricerca applicata.

6.3 Sviluppi implementativi

6.3.1 Integrazione con dataset esterni

L'integrazione con dei dataset contenenti informazioni sui prodotti (e.g., OpenShift, MySQL, Valgrind, Linux, etc.), una loro classificazione D3FEND, eventuali vulnerabilità note, in modo da semplificare i diagrammi architetturali risultanti. Il dataset [Common Platform Enumeration \(CPE\)](#) del NIST può essere un punto di partenza per associare i prodotti a d3f:DigitalArtifact e d3f:DefensiveTechnique. Questo dataset associa ad ogni prodotto Hardware, Software o Sistema Operativo un URI univoco - seppure composto da 11 campi obbligatori. Ad esempio, il seguente URI rappresenta il database MySQL versione 8.0.33:

```
cpe:2.3:a:oracle:mysql:8.0.33::::::::*
```

mentre Ubuntu Linux versione 26.04 è rappresentato da:

```
cpe:2.3:o:canonical:ubuntu_linux:26.04::::::::*
```

Un'interfaccia utente potrebbe definire degli "alias" per questi URI, basandosi su un dataset locale (quello completo è troppo vasto) e ricomporre la stringa corretta per l'URI.

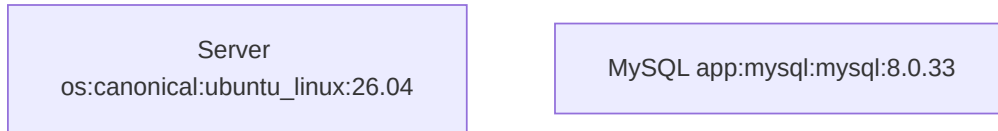


Figura 63: Diagramma con alias CPE per sistema operativo e applicazione

genererebbe quindi il seguente grafo RDF:

```

@prefix app: <cpe:2.3:a:> .
@prefix hw: <cpe:2.3:h:> .
@prefix os: <cpe:2.3:o:> .

app:mysql:mysql:8.0.33:****:*:*:*
  rdfs:subClassOf d3f:DatabaseApplication .

os:canonical:ubuntu_linux:26.04:****:*:*:*
  rdfs:subClassOf d3f:OperatingSystem .

mysql
  a cpe:2.3:a:oracle:mysql:8.0.33:****:*:*:* ;
  rdfs:label "MySQL" ;

server
  a cpe:2.3:o:canonical:ubuntu_linux:26.04:****:*:*:* ;
  rdfs:label "Ubuntu Linux" .
  
```

Si noti la scelta di rappresentare il server come un sistema operativo, funzionale al livello di dettaglio con cui si è deciso di rappresentare l'infrastruttura IT in questo contesto.

Lo stesso approccio si può adottare per associare a dei prodotti delle `d3f:DefensiveTechnique`.

6.3.2 DevSecOps e Integrazione con IaC

L'approccio shift-left consiste nell'inserire la sicurezza fin dalle prime fasi del ciclo di vita delle piattaforme IT, incluse quelle descritte tramite Infrastructure as Code (IaC) (National Institute of Standards and Technology s.d.a).

La creazione di un KG a partire dalle fasi di progettazione permette di inserire dei meccanismi di validazione e verifica della sicurezza già prima delle fasi di imple-

mentazione. È l'approccio che viene utilizzato anche nella metodologia API-First in cui le specifiche di un'API vengono definite prima dell'implementazione, e le sue operazioni, parametri e schemi dati validati tramite strumenti automatici (e.g., Spectral <https://github.com/stoplightio/spectral>).

Questo approccio viene raccomandato anche dal Governo Italiano, che ha pubblicato sia delle Linee Guida per la progettazione delle API, che dei ruleset per la validazione machine-readable delle specifiche OpenAPI (e.g., <https://github.com/italia/api-oas-checker-rules>).

In maniera analoga, sarebbe possibile creare dei workflow di Continuous Integration (CI) che, partendo dai markdown architetturali, prima generano il grafo RDF dell'architettura; quindi tramite una serie di query SPARQL verificano la presenza di misure di sicurezza, enumerando attacchi e definendo delle soglie di accettazione.

6.4 Sviluppi di ricerca applicata

6.4.1 Identificazione della superficie d'attacco

Questo lavoro ha mostrato come sia possibile individuare le tecniche offensive applicabili a un'architettura annotata con D3FEND. Un possibile sviluppo futuro è quello di creare un meccanismo automatico per individuare la superficie d'attacco di un'infrastruttura IT, basandosi sulle entità e le relazioni presenti nel grafo RDF, e rivalutare le tecniche offensive applicabili in base ad essa.

6.4.2 Integrazione con vocabolari giuridici

Lo sviluppo più complesso è quello relativo ad una maggiore integrazione con i vocabolari giuridici, in modo da avere una rappresentazione più esaustiva e precisa dei riferimenti normativi e delle loro implicazioni sulla progettazione.

Questo farebbe da ponte anche verso dei meccanismi di supporto all'analisi del rischio e alla compliance in generale.

6.4.3 Piattaforma integrata per la progettazione sicura

Una possibile evoluzione è quella di creare una piattaforma integrata, capace di utilizzare dei KG remoti (e.g., via API) e di importare all'interno del KG delle infrastrutture a partire da IaC. Anche questa ipotesi si rivela particolarmente sfidante, non tanto dal punto di vista implementativo - poiché si tratta di parsare dei file di testo quali Kubernetes, Ansible, Terraform, etc. - quanto dal punto di vista della rappresentazione delle entità e delle loro relazioni.

Nel caso dell'IaC ogni file rappresenta diverse componenti, ed è inerentemente polisemico: tutto infatti è una `d3f:ConfigurationResource` ma implica la presenza di altri `d3f:DigitalArtifact`.

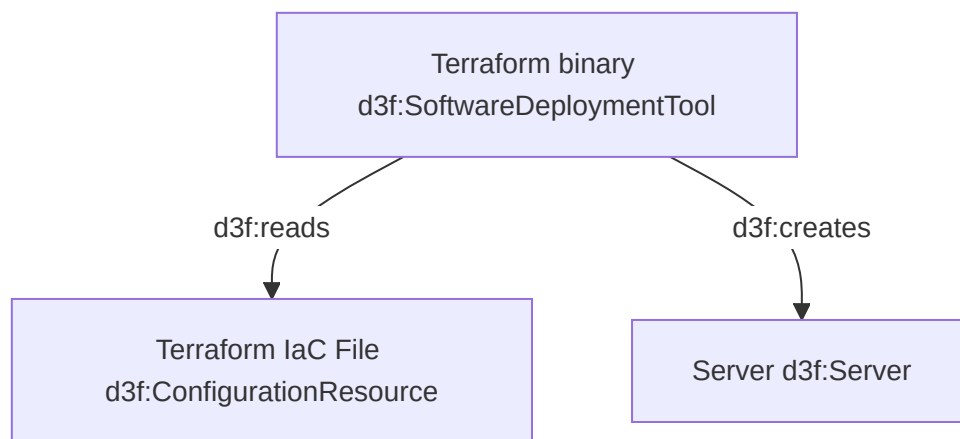


Figura 64: Oggetti digitali impliciti in un file Terraform

Andrebbe quindi fatto un lavoro preventivo di modellazione per verificare l'usabilità di tali rappresentazioni.

Un meccanismo analogo può essere applicato alla creazione di file OpenAPI che descrivono i servizi esposti da un'infrastruttura IT, ad esempio in merito alla tipologia dei dati trattati.

Bibliografia

- Amazon Web Services, Inc. s.d. «Amazon Web Services». Consultato 26 agosto 2026. <https://aws.amazon.com/>.
- Beckett, David, Tim Berners-Lee, Eric Prud'hommeaux, e Gavin Carothers. 2014. «RDF 1.1 Turtle: Terse RDF Triple Language». Recommendation. W3C. <https://www.w3.org/TR/turtle/>.
- Berners-Lee, Tim, Roy T. Fielding, e Larry Masinter. 2005. «Uniform Resource Identifier (URI): Generic Syntax». RFC 3986. IETF. <https://doi.org/10.17487/RFC3986>.
- Birbeck, Mark, e Shane McCarron. 2010. «CURIE Syntax 1.0: A syntax for expressing Compact URIs». Working Group Note. W3C. <https://www.w3.org/TR/curie/>.
- Bray, Tim. 2017. «The JavaScript Object Notation (JSON) Data Interchange Format». RFC 8259. IETF. <https://doi.org/10.17487/RFC8259>.
- Bray, Tim, Jean Paoli, C. M. Sperberg-McQueen, Eve Maler, e François Yergeau. 2008. «Extensible Markup Language (XML) 1.0 (Fifth Edition)». Recommendation. W3C. <https://www.w3.org/TR/xml/>.
- Brickley, Dan, e R. V. Guha. 2014. «RDF Schema 1.1». Recommendation. W3C. <https://www.w3.org/TR/rdf-schema/>.
- Brown, Simon. 2018. «The C4 Model for Visualising Software Architecture». <https://c4model.com/>.
- Carothers, Gavin, e Andy Seaborne. 2014. «RDF 1.1 TriG: RDF Dataset Language». Recommendation. W3C. <https://www.w3.org/TR/trig/>.
- CIS Sapienza, e Laboratorio Nazionale di Cybersecurity CINI. 2025. «Framework Nazionale per la Cybersecurity e la Data Protection — Edizione 2025 (v2.1)». Centro di ricerca CIS, Sapienza Università di Roma e Laboratorio Nazionale di Cybersecurity del CINI. <https://www.cybersecurityframework.it/>.

- Cyganiak, Richard, David Wood, e Markus Lanthaler. 2014. «RDF 1.1 Concepts and Abstract Syntax». Recommendation. W3C. <https://www.w3.org/TR/rdf11-concepts/>.
- Diderot, Denis, e Jean Le Rond d'Alembert. 1751. *Encyclopédie, ou dictionnaire raisonné des sciences, des arts et des métiers*. Paris: Briasson, David, Le Breton, Durand.
- Dürst, Martin, e Michel Suignard. 2005. «Internationalized Resource Identifiers (IRIs)». RFC 3987. IETF. <https://doi.org/10.17487/RFC3987>.
- European Union. 2016. «Regulation (EU) 2016/679 of the European Parliament and of the Council (General Data Protection Regulation)». <https://eur-lex.europa.eu/eli/reg/2016/679/oj>.
- Evans, Eric. 2003. *Domain-Driven Design: Tackling Complexity in the Heart of Software*. Boston: Addison-Wesley.
- Fielding, Roy T., Mark Nottingham, e Julian Reschke. 2022. «HTTP Semantics». RFC 9110. IETF. <https://doi.org/10.17487/RFC9110>.
- Fowler, Martin. 2002. «Domain Model». In *Patterns of Enterprise Application Architecture*. Boston: Addison-Wesley.
- Gandon, Fabien, e Guus Schreiber. 2014. «RDF 1.1 XML Syntax». Recommendation. W3C. <https://www.w3.org/TR/rdf-syntax-grammar/>.
- Harris, Steve, e Andy Seaborne. 2013. «SPARQL 1.1 Query Language». Recommendation. W3C. <https://www.w3.org/TR/sparql11-query/>.
- Hartig, Olaf, Pierre-Antoine Champin, Gregg Kellogg, e Andy Seaborne. 2026. «RDF 1.2 Concepts and Abstract Data Model». Candidate Recommendation Snapshot. W3C. <https://www.w3.org/TR/rdf12-concepts/>.
- Hogan, Aidan, Eva Blomqvist, Michael Cochez, Claudia d'Amato, Gerard de Melo, Claudio Gutierrez, Sabrina Kirrane, et al. 2021. «Knowledge Graphs». *ACM Computing Surveys* 54 (4): 1–37. <https://doi.org/10.1145/3447772>.
- International Organization for Standardization. s.d.a. «ISO». Consultato 26 agosto

2026. <https://www.iso.org/>.
- . s.d.b. «ISO 3166 — Codes for the representation of names of countries and their subdivisions». ISO. Consultato 26 agosto 2026. <https://www.iso.org/iso-3166-country-codes.html>.
- International Organization for Standardization, e International Electrotechnical Commission. 2018. «ISO/IEC 27000:2018 — Information technology — Security techniques — Information security management systems — Overview and vocabulary». Standard. 5^a ed. ISO. <https://www.iso.org/standard/73906.html>.
- ISO/IEC. s.d. «Information technology — Database languages — SQL». Standard 9075. ISO/IEC.
- Jones, Michael B., John Bradley, e Nat Sakimura. 2015. «JSON Web Token (JWT)». RFC 7519. IETF. <https://doi.org/10.17487/RFC7519>.
- Kellogg, Gregg, Pierre-Antoine Champin, e Dave Longley. 2020. «JSON-LD 1.1: A JSON-based Serialization for Linked Data». Recommendation. W3C. <https://www.w3.org/TR/json-ld11/>.
- Library of Congress. s.d. «Library of Congress Subject Headings (LCSH)». Consultato 26 agosto 2026. <https://id.loc.gov/authorities/subjects.html>.
- Mell, Peter, e Timothy Grance. 2011. «The NIST Definition of Cloud Computing». Special Publication 800-145. National Institute of Standards; Technology. <https://doi.org/10.6028/NIST.SP.800-145>.
- Mermaid Contributors. 2014. «Mermaid.js». <https://mermaid.js.org/>.
- Miles, Alistair, e Sean Bechhofer. 2009. «SKOS Simple Knowledge Organization System Reference». Recommendation. W3C. <https://www.w3.org/TR/skos-reference/>.
- MITRE Corporation. 2021. «D3FEND: A Knowledge Graph of Cybersecurity Countermeasures». <https://d3fend.mitre.org/>.
- . 2024. «D3FEND Ontology: CAPEC to RDF Conversion Tool (update_capec.py)». <https://github.com/d3fend/d3fend-ontology/pull/258>.

- . 2026. «D3FEND CAD User Guide». <https://d3fend.mitre.org/cad/docs/>.
- . s.d.a. «CAPEC: Common Attack Pattern Enumeration and Classification». Consultato 4 settembre 2026. <https://capec.mitre.org/>.
- . s.d.b. «CWE: Common Weakness Enumeration». Consultato 4 settembre 2026. <https://cwe.mitre.org/>.
- . s.d.c. «MITRE». Consultato 26 agosto 2026. <https://www.mitre.org/>.
- . s.d.d. «MITRE ATLAS: Adversarial Threat Landscape for Artificial-Intelligence Systems». Consultato 2 settembre 2026. <https://atlas.mitre.org/>.
- . s.d.e. «MITRE ATT&CK». Consultato 26 agosto 2026. <https://attack.mitre.org/>.
- Mozilla Developer Network. s.d. «SPA (Single-page application)». MDN Web Docs Glossary. Consultato 26 agosto 2026. <https://developer.mozilla.org/en-US/docs/Glossary/SPA>.
- National Institute of Standards and Technology. 2020. «Security and Privacy Controls for Information Systems and Organizations». Special Publication 800-53 Revision 5. National Institute of Standards; Technology. <https://doi.org/10.6028/NIST.SP.800-53r5>.
- . 2024. «The NIST Cybersecurity Framework (CSF) 2.0». Cybersecurity White Paper CSWP 29. National Institute of Standards; Technology. <https://doi.org/10.6028/NIST.CSWP.29>.
- . 2026. «NIST Special Publication 1347: Cybersecurity Framework 2.0». Special Publication. National Institute of Standards; Technology. <https://doi.org/10.6028/NIST.SP.1347>.
- . s.d.a. «Computer Security Resource Center Glossary». Consultato 26 agosto 2026. <https://csrc.nist.gov/glossary>.
- . s.d.b. «Security Control — Computer Security Resource Center Glossary». Consultato 26 agosto 2026. <https://csrc.nist.gov/glossary/term/>

[security_control](#).

National Security Agency. s.d. «National Security Agency/Central Security Service». Consultato 26 agosto 2026. <https://www.nsa.gov/>.

Neo4j, Inc. s.d. «Neo4j Graph Database». Consultato 26 agosto 2026. <https://neo4j.com/>.

Nottingham, Mark. 2017. «Web Linking». RFC 8288. IETF. <https://doi.org/10.17487/RFC8288>.

Object Management Group. 2014. «Ontology Definition Metamodel (ODM), Version 1.1, Annex D: Mapping UML to OWL». formal/2014-09-02. OMG. <https://www.omg.org/spec/ODM/1.1/>.

———. 2017. «Unified Modeling Language (UML), Version 2.5.1». formal/2017-12-05. OMG. <https://www.omg.org/spec/UML/2.5.1/>.

OWASP Foundation. 2023. «OWASP API Security Top 10 2023». <https://owasp.org/API-Security/editions/2023/en/0x00-header/>.

———. 2025. «OWASP Top 10 Web Application Security Risks 2025». <https://owasp.org/Top10/>.

Pandit, Harshvardhan J., Beatriz Esteves, Delaram Golpayegani, Georg P. Krog, Julian Flake, e Paul Ryan. 2026. «Data Privacy Vocabulary (DPV), Version 2.3». Final Community Group Report. W3C Data Privacy Vocabularies; Controls Community Group (DPVCG). <https://w3id.org/dpv/2.3>.

Pandit, Harshvardhan J., Beatriz Esteves, Efstratios Koulterakis, Georg P. Krog, e Paul Ryan. 2026. «EU GDPR concepts for DPV, Version 2.3». Final Community Group Report. W3C Data Privacy Vocabularies; Controls Community Group (DPVCG). <https://w3id.org/dpv/2.3/legal/eu/gdpr>.

Pandit, Harshvardhan J., Beatriz Esteves, e Georg P. Krog. s.d. «Data Privacy Vocabulary (DPV) Primer, Version 2.3». Consultato 26 agosto 2026. <https://w3id.org/dpv/primer>.

Polli, Roberto. 2023. «Cybersecurity in the Semantic Web with MITRE D3FEND».

- Presentazione, PyCon Italia 2023, Firenze, Italia. <https://2023.pycon.it/en/event/cybersecurity-in-the-semantic-web-with-mitre-d3fend>.
- . 2026. «Design & D3FEND». Software, Zenodo. <https://doi.org/10.5281/zenodo.22125372>.
- The Aerospace Corporation. s.d. «SPARTA: Space Attack Research and Tactic Analysis». Consultato 2 settembre 2026. <https://sparta.aerospace.org/>.
- The CVE Program. s.d. «CVE: Common Vulnerabilities and Exposures». Consultato 4 settembre 2026. <https://www.cve.org/>.
- The Kubernetes Authors. 2014. «Kubernetes». <https://kubernetes.io/>.
- . s.d. «Diagram Guide». Consultato 26 agosto 2026. <https://kubernetes.io/docs/contribute/style/diagram-guide/>.
- Unified Cyber Ontology. s.d. «Unified Cyber Ontology (UCO)». Consultato 26 agosto 2026. <https://unifiedcyberontology.org/>.
- URI Planning Interest Group, W3C/IETF. s.d. «URIs, URLs, and URNs: Clarifications and Recommendations 1.0». Consultato 26 agosto 2026. <https://www.w3.org/TR/uri-clarification/>.
- W3C OWL Working Group. 2012. «OWL 2 Web Ontology Language Document Overview (Second Edition)». Recommendation. W3C. <https://www.w3.org/TR/owl2-overview/>.
- Watsen, Kent, Erik Auerswald, Adrian Farrel, e Qin Wu. 2020. «Handling Long Lines in Content of Internet-Drafts and RFCs». RFC 8792. IETF. <https://doi.org/10.17487/RFC8792>.
- Wikidata. s.d. «Wikidata». Consultato 26 agosto 2026. <https://www.wikidata.org/>.
- World Wide Web Consortium. s.d. «W3C». Consultato 26 agosto 2026. <https://www.w3.org/>.